

# ARM PrimeCell

## Vectored Interrupt Controller (PL192)

### **Design Documentation**

## ARM PrimeCell Vectored Interrupt Controller (PL192) Design Documentation

© Copyright ARM Limited 2002. All rights reserved.

### Proprietary notice

ARM, the ARM powered logo, Thumb and StrongARM are registered trademarks of ARM Limited.

The ARM logo, PrimeCell, AMBA, Angel, ARMulator, EmbeddedICE, ModelGen, Multi-ICE, ARM7TDMI, ARM9TDMI, TDMI and STRONG are trademarks of ARM Limited.

All other products or services mentioned herein may be trademarks of their respective owners.

Neither the whole nor any part of the information contained in, or the product described in, this document may be adapted or reproduced in any material form except with the prior written permission of the copyright holder.

The product described in this document is subject to continuous developments and improvements. All particulars of the product and its use contained in this document are given by ARM Limited in good faith. However, all warranties implied or expressed, including but not limited to implied warranties or merchantability, or fitness for purpose, are excluded.

This document is intended only to assist the reader in the use of the product. ARM Limited shall not be liable for any loss or damage arising from the use of any information in this document, or any error or omission in such information, or any incorrect use of the product.

### Document confidentiality status

This document is ARM Confidential (Distributed to ARM staff, product licensees & NDA signatories only).

### Product status

The information in this document is Final **(information on a developed product)**

### Web address

<http://www.arm.com>

## Feedback

ARM Limited welcomes feedback on both the product, and the documentation.

### Feedback on this document

If you have any comments on about this document, please send email to [errata@arm.com](mailto:errata@arm.com) giving:

- The document title
- The documents number
- The page number(s) to which your comments refer
- A concise explanation of your comments

General suggestion for additions and improvements are also welcome.

### Feedback on the ARM PrimeCell Vectored Interrupt Controller

If you have any comments or suggestions about this product, please contact your supplier giving:

- The Product name
- A concise explanation of your comments.

# Contents

|            |                                                            |           |
|------------|------------------------------------------------------------|-----------|
| <b>1</b>   | <b>ABOUT THIS DOCUMENT</b>                                 | <b>4</b>  |
| <b>1.1</b> | <b>Change control</b>                                      | <b>4</b>  |
| 1.1.1      | Current status and anticipated changes                     | 4         |
| 1.1.2      | Change history                                             | 4         |
| <b>1.2</b> | <b>References</b>                                          | <b>4</b>  |
| <b>1.3</b> | <b>Terms and abbreviations</b>                             | <b>4</b>  |
| <b>2</b>   | <b>SCOPE</b>                                               | <b>6</b>  |
| <b>3</b>   | <b>INTRODUCTION</b>                                        | <b>7</b>  |
| <b>4</b>   | <b>BLOCK DESCRIPTION</b>                                   | <b>8</b>  |
| <b>4.1</b> | <b>Signal Description</b>                                  | <b>8</b>  |
| <b>4.2</b> | <b>Programmer's Model</b>                                  | <b>9</b>  |
| 4.2.1      | Functional Mode Registers                                  | 10        |
| 4.2.2      | Integration Test Mode Registers                            | 10        |
| 4.2.3      | Identification Registers                                   | 11        |
| <b>4.3</b> | <b>Block Partitioning</b>                                  | <b>12</b> |
| <b>4.4</b> | <b>VIC-AHB Interface (VicAhbif) top level block</b>        | <b>13</b> |
| 4.4.1      | Input signals                                              | 14        |
| 4.4.2      | Output signals                                             | 15        |
| <b>4.5</b> | <b>Interrupt Processing (VicInterrupt) top level block</b> | <b>17</b> |
| 4.5.1      | Input signals                                              | 18        |
| 4.5.2      | Output signals                                             | 19        |
| <b>4.6</b> | <b>VIC-CPU Interface (VicCpuif) block description</b>      | <b>20</b> |
| 4.6.1      | Input signals                                              | 21        |
| 4.6.2      | Output signals                                             | 22        |
| <b>4.7</b> | <b>Revision Designator Block</b>                           | <b>23</b> |
| 4.7.1      | Input signals                                              | 24        |
| 4.7.2      | Output signals                                             | 24        |
| <b>5</b>   | <b>VIC IMPLEMENTATION DETAILS</b>                          | <b>25</b> |
| <b>5.1</b> | <b>Hierarchy</b>                                           | <b>25</b> |
| <b>5.2</b> | <b>AHB Interface Block (VicAhbif)</b>                      | <b>25</b> |
| 5.2.1      | AHB Signal decode                                          | 25        |
| 5.2.2      | Registers write                                            | 26        |
| 5.2.3      | Register Read                                              | 27        |

|            |                                                    |           |
|------------|----------------------------------------------------|-----------|
| 5.2.4      | Slave response generation                          | 30        |
| 5.2.5      | Generation of Control signals                      | 31        |
| <b>5.3</b> | <b>Interrupt Processing Block (VicInterrupt)</b>   | <b>32</b> |
| 5.3.1      | Priority Scheme                                    | 32        |
| 5.3.2      | VicInterrupt Block                                 | 33        |
| 5.3.3      | VicPriority block                                  | 34        |
| 5.3.4      | VicIntResolver block                               | 36        |
| <b>5.4</b> | <b>CPU Interface (VicCpuif)</b>                    | <b>37</b> |
| <b>5.5</b> | <b>Revision Designator block (VicRevAnd)</b>       | <b>40</b> |
| 5.5.1      | Integration test input registers                   | 40        |
| 5.5.2      | Integration test output registers                  | 41        |
| 5.5.3      | Integration test output registers                  | 42        |
| <b>6</b>   | <b>CUSTOMISATION</b>                               | <b>43</b> |
| <b>6.1</b> | <b>Removing the VIC port</b>                       | <b>43</b> |
| <b>6.2</b> | <b>Removing the Daisy Chain Connection support</b> | <b>43</b> |
| <b>7</b>   | <b>VIC VERIFICATION TESTBENCH</b>                  | <b>45</b> |
| <b>7.1</b> | <b>VIC VHDL Verification Testbench</b>             | <b>45</b> |
| 7.1.1      | Unit Under Test                                    | 46        |
| 7.1.2      | Bus Master Protocol Checker                        | 47        |
| 7.1.3      | AHB Master/Arbiter Model                           | 47        |
| 7.1.4      | Decoder                                            | 47        |
| 7.1.5      | Default Slave                                      | 47        |
| 7.1.6      | Protocol Checker                                   | 47        |
| 7.1.7      | Trickbox                                           | 48        |
| 7.1.8      | Simulation environment                             | 48        |
| <b>7.2</b> | <b>VIC Verilog Verification Testbench</b>          | <b>49</b> |
| 7.2.1      | Testbench                                          | 50        |
| 7.2.2      | Unit Under Test                                    | 50        |
| 7.2.3      | Bus Master Protocol Checker                        | 51        |
| 7.2.4      | AHB Master/Arbiter Model                           | 51        |
| 7.2.5      | Decoder                                            | 51        |
| 7.2.6      | Default Slave                                      | 51        |
| 7.2.7      | Protocol Checker                                   | 51        |
| 7.2.8      | Trickbox                                           | 52        |
| 7.2.9      | Simulation Environment                             | 52        |
| <b>7.3</b> | <b>Trickbox Description</b>                        | <b>53</b> |
| 7.3.1      | Trickbox Signal Description                        | 53        |
| 7.3.2      | Trickbox Register Description                      | 55        |
| 7.3.3      | Trickbox Specific Registers Description            | 56        |
| 7.3.4      | Trickbox Mirrored Registers Description            | 58        |
| 7.3.5      | Trickbox Functional Description                    | 61        |
| <b>8</b>   | <b>FUNCTIONAL TESTS</b>                            | <b>66</b> |

|             |                                                                    |            |
|-------------|--------------------------------------------------------------------|------------|
| <b>8.1</b>  | <b>Reset Tests (ResetTests)</b>                                    | <b>66</b>  |
| <b>8.2</b>  | <b>Register Tests (RegisterTests)</b>                              | <b>68</b>  |
| <b>8.3</b>  | <b>Transfer Test (TransferTests)</b>                               | <b>69</b>  |
| <b>8.4</b>  | <b>Error Response Test (ErrorRespTests)</b>                        | <b>69</b>  |
| <b>8.5</b>  | <b>Protection Mode Test (ProtectionTests)</b>                      | <b>69</b>  |
| <b>8.6</b>  | <b>Protection Mode Interrupt Test (ProtModelIntTests)</b>          | <b>70</b>  |
| <b>8.7</b>  | <b>Basic Interrupt Tests</b>                                       | <b>71</b>  |
| 8.7.1       | One Interrupt Test (UserOneIntTests0-7 or VicEnUserIntTests0-7)    | 72         |
| 8.7.2       | Two Interrupt Test (TwoFourIntTests or VicEnTwoFourIntTests)       | 75         |
| 8.7.3       | Four Interrupt Test (TwoFourIntTests or VicEnTwoFourIntTests)      | 77         |
| 8.7.4       | Eight Interrupt Test (EightAllIntTests or VicEnEightAllIntTests)   | 80         |
| 8.7.5       | Sixteen Interrupt Test (EightAllIntTests or VicEnEightAllIntTests) | 84         |
| 8.7.6       | Thirty-Two Interrupt Test (AllIntTest or VicEnAllIntTest)          | 89         |
| 8.7.7       | Special Interrupt Tests                                            | 97         |
| 8.7.8       | Priority Consistency Test (PriConsistency)                         | 103        |
| 8.7.9       | Nested Interrupts Tests                                            | 103        |
| 8.7.10      | Asynchronous Mode Tests                                            | 104        |
| 8.7.10.1    | ClockOffTest                                                       | 104        |
| 8.7.10.2    | VicSyncEnTest                                                      | 104        |
| 8.7.11      | Random Test                                                        | 104        |
| <b>8.8</b>  | <b>Verification Test Vector Generation</b>                         | <b>104</b> |
| <b>9</b>    | <b>INTEGRATION TESTBENCH</b>                                       | <b>105</b> |
| <b>9.1</b>  | <b>VIC VHDL Integration Testbench</b>                              | <b>105</b> |
| 9.1.1       | Unit Under Test                                                    | 106        |
| 9.1.2       | VIC Trickbox (VicTrickbox)                                         | 107        |
| 9.1.3       | Test Vectors                                                       | 107        |
| 9.1.4       | Running Simulations                                                | 107        |
| <b>9.2</b>  | <b>VIC Verilog Integration Testbench</b>                           | <b>107</b> |
| 9.2.1       | Unit Under Test                                                    | 109        |
| 9.2.2       | VIC Trickbox (VicTrickbox)                                         | 110        |
| 9.2.3       | Test Vectors                                                       | 110        |
| 9.2.4       | Running Simulations                                                | 110        |
| <b>10</b>   | <b>INTEGRATION TESTS</b>                                           | <b>111</b> |
| <b>10.1</b> | <b>Integration Testing of AHB pins</b>                             | <b>111</b> |
| <b>10.2</b> | <b>Integration Testing of intra-chip inputs</b>                    | <b>111</b> |
| <b>10.3</b> | <b>Integration Testing of intra-chip outputs</b>                   | <b>112</b> |
| <b>10.4</b> | <b>Integration Test Vector Generation</b>                          | <b>112</b> |

# 1 ABOUT THIS DOCUMENT

## 1.1 Change control

### 1.1.1 Current status and anticipated changes

This document has been reviewed and review comments have been incorporated.

### 1.1.2 Change history

| Issue | Date                             | Change       |
|-------|----------------------------------|--------------|
| A00   | 2 <sup>nd</sup> September, 2002  | First draft  |
| A01   | 16 <sup>th</sup> September, 2002 | Second draft |
| A     | 1 <sup>st</sup> October, 2002    | Final        |

## 1.2 References

This document refers to the following documents.

| Ref. | Doc No              | Title                                                                          |
|------|---------------------|--------------------------------------------------------------------------------|
| 1    | DDI0273A            | ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual |
| 2    | ARM DDI 0138        | Example AMBA System Technical Reference Manual                                 |
| 3    | ARM DUI 0092        | Example AMBA System User Guide                                                 |
| 4    | SCB00-QPRO-0008-A03 | ARM PrimeCell Integration Vector Strategy                                      |
| 5    | ARM DUI 0006        | AMBA Compliance Test Suite User Guide                                          |

## 1.3 Terms and abbreviations

This document uses the following terms and abbreviations.

| Term | Meaning                                    |
|------|--------------------------------------------|
| AMBA | Advanced Micro-controller Bus Architecture |
| AHB  | Advanced High-performance Bus              |
| ASIC | Application Specific Integrated Circuit    |
| HDL  | Hardware Description Language              |
| TIC  | Test Interface Controller                  |
| IRQ  | ARM core interrupt.                        |
| ISR  | Interrupt Service Routine.                 |
| FIQ  | ARM core fast interrupt.                   |

|                    |                                                                                                                            |
|--------------------|----------------------------------------------------------------------------------------------------------------------------|
| VIC                | Vectored Interrupt Controller.                                                                                             |
| Vectored Interrupt | An interrupt where the vector to the interrupt service routine is provided by hardware, normally the interrupt controller. |



## 2 SCOPE

This document describes the design and implementation of the ARM PrimeCell VIC (Vectored Interrupt Controller PL192).

The document is divided into 10 sections.

- Section 1 gives the change status of the document, terms, abbreviations and conventions used in the document
- Section 3 gives an overview of VIC PL192
- Section 4 gives the overview of the PL192 design, block partitioning and list of input and output signals of each block. This section also lists the functional registers of the VIC PL192
- Section 5 describes the implementation details of each block mentioned in the section 4.
- Section 6 describes the customisation of the VIC.
- Section 7 describes the VIC PL192 Functional verification testbench.
- Section 8 describes the Functional Tests for VIC PL192.
- Section 9 describes the VIC PL192 Integration testbench.
- Section 10 describes the VIC PL192 Integration tests.

## 3 INTRODUCTION

The VIC PL192 is a part of a library of reusable IP blocks, which can be easily integrated into a typical ASIC design flow. It provides a software interface to the interrupt system. It resolves the interrupt generated either by hardware or by software, asserts the interrupt and provides the starting vector address of the interrupt service routine.

For further information on this block, refer to the ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual [Ref.1].

The Example AMBA System Technical Reference Manual [Ref.2] and User Guide [Ref.3] describe an example micro-controller based on an ARM processor and the low-power, generic design methodology of AMBA.

## 4 BLOCK DESCRIPTION

### 4.1 Signal Description

The following tables summarise the VIC signal list. **Table 4.1-1** lists the AHB interface signals, while **Table 4.1-2** lists the module-specific functionality related signals. **Table 4.1-3** lists the Scan Test signals.

| Name         | Type | Source or Destination | Description                                                                                                                                                                                                    |
|--------------|------|-----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HCLK         | IN   | Clock Source          | AHB clock                                                                                                                                                                                                      |
| HRESETn      | IN   | Reset Controller      | AHB reset (active low)                                                                                                                                                                                         |
| HADDR[11:2]  | IN   | Master                | This is part of the AHB address bus, which is used for decoding the internal address space                                                                                                                     |
| HSELVIC      | IN   | Decoder               | When HIGH, this signal indicates that VIC has been selected                                                                                                                                                    |
| HTRANS[1:0]  | IN   | Master                | Only the connection HTRANS [1] is used. When HTRANS[1] is HIGH, this signal indicates that the current transfer is a Sequential or Non-Sequential transfer and when LOW, it indicates an Idle or Busy transfer |
| HPROT[3:0]   | IN   | Master                | Only the connection HPROT [1] is used. When HPROT[1] is HIGH, this signal indicates privileged access and when LOW, it indicates the User Access                                                               |
| HSIZE[2:0]   | IN   | Master                | Indicates AHB bus transfer size. The VIC supports 32-bit accesses only                                                                                                                                         |
| HWDATA[31:0] | IN   | Master                | AHB Write data to the VIC                                                                                                                                                                                      |
| HWRITE       | IN   | Master                | AHB bus write signal. When this signal is HIGH, this signal indicates a write into the VIC and when LOW, a read from the VIC                                                                                   |
| HREADYIN     | IN   | External Slave        | AHB bus transfer done indication generated either by the VIC or by an alternate slave                                                                                                                          |
| HRDATA[31:0] | OUT  | Slave                 | AHB data is read out of the VIC via this bus                                                                                                                                                                   |
| HREADYOUT    | OUT  | Slave                 | AHB bus transfer done indication generated by the VIC                                                                                                                                                          |
| HRESP[1:0]   | OUT  | Slave                 | AHB bus transfer response generated by the VIC                                                                                                                                                                 |

**Table 4.1-1 AHB interface signal description**

| Name                | Type | Source or Destination         | Description                                         |
|---------------------|------|-------------------------------|-----------------------------------------------------|
| VICINTSOURCE [31:0] | IN   | Peripheral(s)                 | Interrupt source input from Peripheral(s)           |
| nVICFIQIN           | IN   | External interrupt controller | Daisy chained FIQ input                             |
| nVICIRQIN           | IN   | External interrupt controller | Daisy chained IRQ input                             |
| VICVECTADDRIN[31:0] | IN   | External interrupt controller | Daisy chained interrupt address input               |
| VICIRQACK           | IN   | ARM / Upstream                | Interrupt acknowledge from processor if VIC port is |

| Name              | Type | Source or Destination               | Description                                                                                                                        |
|-------------------|------|-------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
|                   |      | Interrupt Controller                | used (Only for ARM11 and some ARM10 products), or from VICIRQACKOUT from upstream daisy chain VIC                                  |
| VICIRQINREG       | IN   | System                              | Tied HIGH if the user wishes to register the daisy chained nVICIRQIN. Tied LOW if the user does not wish to register the nVICIRQIN |
| VICFIQINREG       | IN   | System                              | Tied HIGH if the user wishes to register the daisy chained nVICFIQIN. Tied LOW if the user does not wish to register the nVICFIQIN |
| nVICSYNCEN        | IN   | System                              | Used to indicate that the VIC port must operate in synchronous mode when nVICSYNCEN is LOW                                         |
| nVICIRQ           | OUT  | ARM / Upstream Interrupt Controller | Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain                                      |
| nVICFIQ           | OUT  | ARM / Upstream Interrupt Controller | Fast Interrupt request to the ARM processor or to the next Interrupt Controller in the Daisy chain                                 |
| VICVECTADDR[31:0] | OUT  | ARM / Upstream Interrupt controller | Interrupt address to the processor or to the next Interrupt Controller in the Daisy chain                                          |
| VICVECTADDRV      | OUT  | ARM                                 | Handshaking feedback to the processor to indicate output on VICVECTADDR[31:0] is stable                                            |
| VICIRQACKOUT      | OUT  | External interrupt controller       | Acknowledge feedback to downstream daisy chained Vectored Interrupt Controller                                                     |

**Table 4.1-2 Module-specific functional signal descriptions.**

| Name        | Type | Source or Destination | Description                        |
|-------------|------|-----------------------|------------------------------------|
| SCANENABLE  | IN   | Test Controller       | Scan test enable                   |
| SCANINHCLK  | IN   | Test Controller       | Scan Input signal for HCLK domain  |
| SCANOUTHCLK | OUT  | Test Controller       | Scan Output signal for HCLK domain |

**Table 4.1-3 Scan Test signal descriptions.**

## 4.2 Programmer's Model

The VIC Functional mode registers are shown in **Table 4.2-1** and the Integration Test Mode registers are shown in **Table 4.2-2**. The VIC Identification registers are shown in **Table 4.2-3**. The base address of the VIC is implementation dependent. For best performance, the VIC should reside at 0xffff\_f000. The registers located at offsets 0x300, 0x304, 0x308, 0x30C, 0x310, 0x314 and 0x318 are reserved for integration testing and are not used during normal operation. For a detailed description of the registers, refer to the ARM PrimeCell Vectored Interrupt Controller (PL192) Technical Reference Manual [Ref. 1].

### 4.2.1 Functional Mode Registers

| Register              | Offset from base | Type | Bit Fields | Reset Value HRESETn | Description                                                  |
|-----------------------|------------------|------|------------|---------------------|--------------------------------------------------------------|
| VICIRQSTATUS          | 0x000            | R    | 31 – 0     | 0x00000000          | Provides the status of IRQ                                   |
| VICFIQSTATUS          | 0x004            | R    | 31 – 0     | 0x00000000          | Provides the status of FIQ                                   |
| VICRAWINTR            | 0x008            | R    | 31 – 0     | -                   | Provides the raw interrupt status                            |
| VICINTSELECT          | 0x00C            | R/W  | 31 – 0     | 0x00000000          | Selects the type of interrupt (IRQ or FIQ)                   |
| VICINTENABLE          | 0x010            | R/W  | 31 – 0     | 0x00000000          | Interrupt enable                                             |
| VICINTENCLEAR         | 0x014            | W    | 31- 0      | -                   | Interrupt enable clear                                       |
| VICSOFTINT            | 0x018            | R/W  | 31 – 0     | 0x00000000          | Generates software interrupt                                 |
| VICSOFTINTCLEAR       | 0x01C            | W    | 31 – 0     | -                   | Clears the software interrupt                                |
| VICPROTECTION         | 0x020            | R/W  | 0          | 0x0                 | Protection enable                                            |
| VICSWPRIORITYMASK     | 0x024            | R/W  | 15 – 0     | 0xFFFF              | Software priority mask                                       |
| VICVECTPRIORITY DAISY | 0x028            | R/W  | 3 – 0      | 0xF                 | Vector priority for Daisy Chain VIC                          |
| VICADDRESS            | 0xF00            | R/W  | 31 – 0     | 0x00000000          | ISR address of the currently servicing interrupt.            |
| VICVECTADDRX          | 0x100 – 0x17C    | R/W  | 31 – 0     | 0x00000000          | Vector addresses of 32 interrupt sources [X = 0 to 31]       |
| VICVECTPRIORITYX      | 0x200 – 0x27C    | R/W  | 3 – 0      | 0xF                 | Vector Priority values of 32 interrupt sources [X = 0 to 31] |

**Table 4.2-1 Functional Mode Registers Memory Map**

### 4.2.2 Integration Test Mode Registers

| Register            | Offset from base | Type | Bit Fields | Reset Value HRESETn | Description                           |
|---------------------|------------------|------|------------|---------------------|---------------------------------------|
| VICITCR             | 0x300            | R/W  | 1 – 0      | 0x0                 | Test control register                 |
| VICITIP1            | 0x304            | R/W  | 10 – 6     | -                   | Test input register1                  |
| VICITIP2            | 0x308            | R/W  | 31 – 0     | -                   | Test input register2                  |
| VICITOP1            | 0x30C            | R/W  | 8 – 6      | 0x000               | Test output register1                 |
| VICITOP2            | 0x310            | R/W  | 31 – 0     | 0x00000000          | Test output register2                 |
| VICINTSSTATUS       | 0x314            | R    | 31 – 0     | 0x00000000          | Sampled interrupt source status       |
| VICINTSSTATUS CLEAR | 0x318            | W    | 31 – 0     | -                   | Sampled interrupt source status Clear |

**Table 4.2-2 Integration Test Mode Registers Memory Map**

### 4.2.3 Identification Registers

| Register     | Offset from base | Type | Bit Fields | Reset value HRESETn | Description                       |
|--------------|------------------|------|------------|---------------------|-----------------------------------|
| VICPERIPHID0 | 0xFE0            | R    | 7 – 0      | 0x92                | Peripheral ID register Bits 7:0   |
| VICPERIPHID1 | 0xFE4            | R    | 15 – 8     | 0x11                | Peripheral ID register Bits 15:8  |
| VICPERIPHID2 | 0xFE8            | R    | 23 – 16    | 0x04                | Peripheral ID register Bits 23:16 |
| VICPERIPHID3 | 0xFEC            | R    | 31 – 24    | 0x00                | Peripheral ID register Bits 31:24 |
| VICPCCELLID0 | 0xFF0            | R    | 7 – 0      | 0x0D                | PrimeCell ID register Bits 7:0    |
| VICPCCELLID1 | 0xFF4            | R    | 15 – 8     | 0xF0                | PrimeCell ID register Bits 15:8   |
| VICPCCELLID2 | 0xFF8            | R    | 23 – 16    | 0x05                | PrimeCell ID register Bits 23:16  |
| VICPCCELLID3 | 0xFFC            | R    | 31 – 24    | 0xB1                | PrimeCell ID register Bits 31:24  |

**Table 4.2-3 Identification Registers Memory Map**

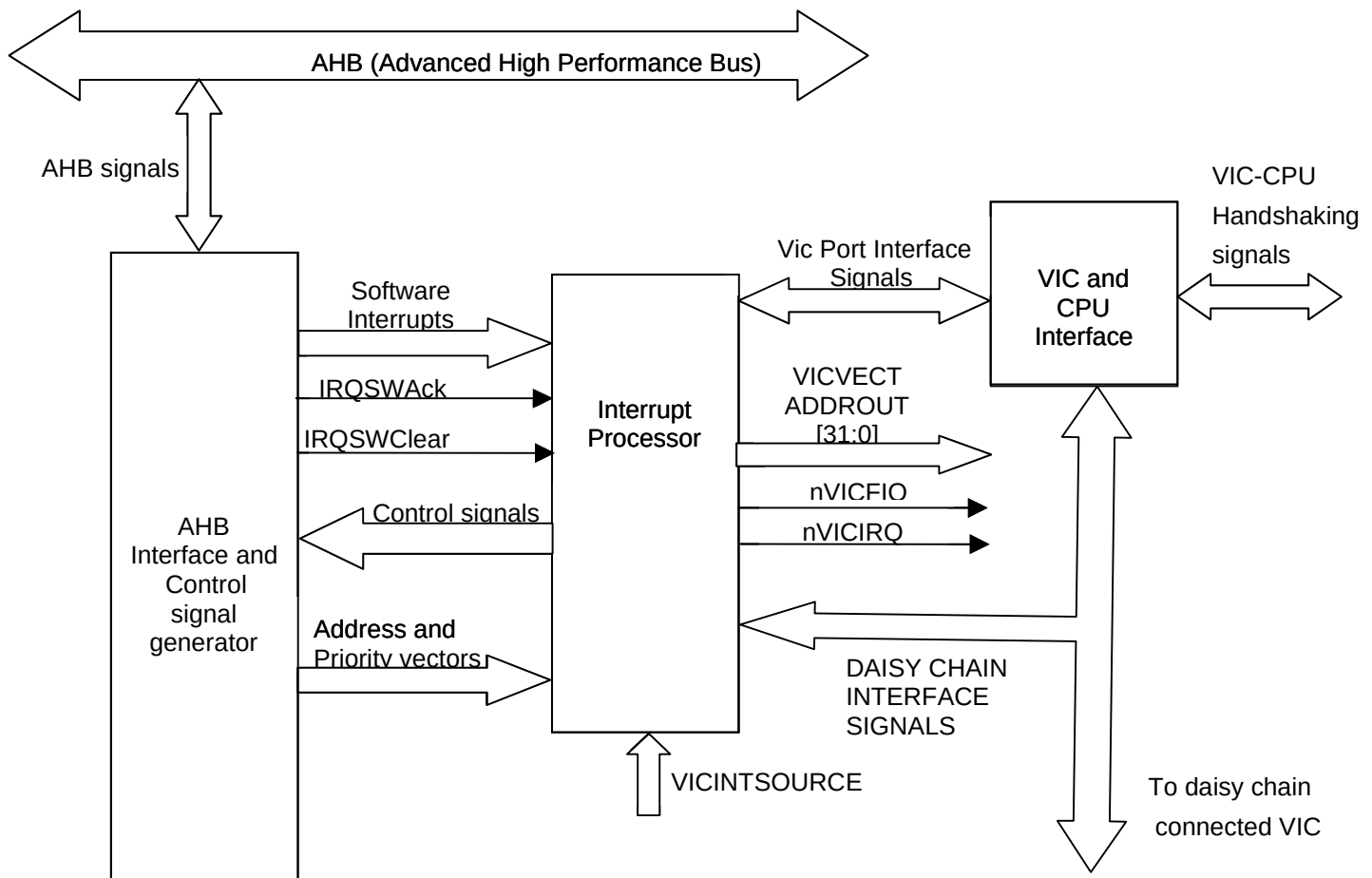
**Note:** Unfilled values in the 'Reset Value' Column of **Table 4.2-1** and **Table 4.2-2** indicates that the reset values are unknown. This is because these registers are fed by external inputs (nVICIRQIN, nVICFIQIN and VICVECTADDRIN). The actual values in these registers will hence depend on the values present on the external inputs.

### 4.3 Block Partitioning

The Vectored Interrupt Controller block diagram is shown in **Figure 4.3-1**. The design contains the following sub-blocks.

- AHB Interface and Register Block (VicAhbif)
- Interrupt processing (VicInterrupt)
- Processor interface block (VicCpuif)

The design of these sub-blocks is detailed in the subsequent sections.



**Figure 4.3-1 Vectored Interrupt Controller Block diagram**

The VicAhbif block is used to interface the AHB bus and to handle the reads and writes to the registers. It also generates signals required for the software interrupt acknowledge (when reading or writing to VICADDRESS register).

The VicInterrupt block accepts interrupt sources and determines the highest priority interrupt to be served. Inside the VicInterrupt block, the interrupt requests are synchronised for the priority encoding logic. At the same time, unsynchronised interrupt requests pass through a masking circuit and reach the processor. Even if HCLK is stopped, the asynchronous interrupt request can still reach the processor. It also stores the interrupt that is currently being serviced.

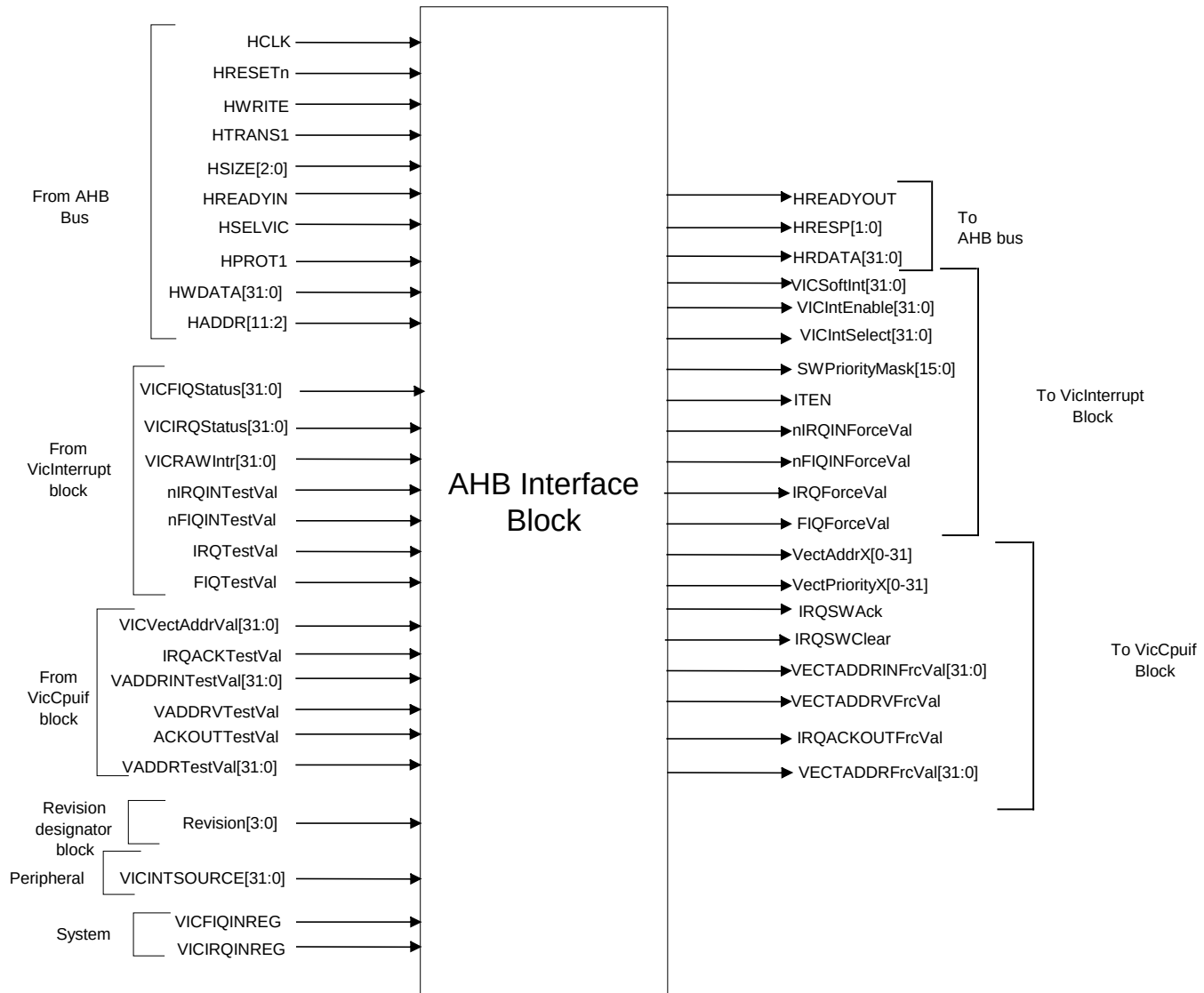
The VicCpuif block generates the handshaking signals required for the VIC port. This block generates the masking control signals, which will be used in the VicInterrupt block. If the interrupt is from a downstream VIC, the signal VICIRQACKOUT is asserted informing the VICs on the daisy chain that the interrupt has been acknowledged.

## 4.4 VIC-AHB Interface (VicAhbif) top level block

**Figure 4.4-1** shows the port level block diagram of the AHB interfaces (VicAhbif). The AHB interface allows the VIC to connect to the AMBA AHB system. The following functionality is implemented in this block

- AHB Address and control signals registering
- Error monitoring to detect the transfers that results in an error response.
- Generation of read and write acknowledge signals

The block diagram of the VicAhbif is as below



**Figure 4.4-1 VicAhbif Block diagram**



#### 4.4.1 Input signals

| Signal Name          | Source                    | Comments                                                                                                                                                                                                                                                            |
|----------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HCLK                 | AHB Master                | AHB clock                                                                                                                                                                                                                                                           |
| HRESETn              | AHB Master                | AHB reset                                                                                                                                                                                                                                                           |
| HSELVIC              | AHB Master                | VIC select signal                                                                                                                                                                                                                                                   |
| HADDR[11:2]          | AHB Master                | AHB address lines                                                                                                                                                                                                                                                   |
| HTRANS1              | AHB Master                | Bit 1 of AHB transfer type                                                                                                                                                                                                                                          |
| HWRITE               | AHB Master                | AHB write                                                                                                                                                                                                                                                           |
| HREADYIN             | AHB Master                | Shared HREADY line                                                                                                                                                                                                                                                  |
| HPROT1               | AHB Master                | Bit 1 of HPROT                                                                                                                                                                                                                                                      |
| HSIZE[2:0]           | AHB Master                | AHB transfer size                                                                                                                                                                                                                                                   |
| HWDATA[31:0]         | AHB Master                | AHB data line                                                                                                                                                                                                                                                       |
| Revision[3:0]        | Revision designator block | Indicates the peripheral revision number. The AHB master through the PERIPHID3 register can read this vector signal.                                                                                                                                                |
| VICFIQStatus[31:0]   | VicInterrupt              | Indicates the status of the interrupts after the FIQ masking. When the particular bit is '0', that interrupt is inactive and if it is '1', interrupt is active                                                                                                      |
| VICIRQStatus[31:0]   | VicInterrupt              | Indicates the status of the interrupts[31:0] after IRQ masking. There is one bit for each interrupt source. When the bit is '0', interrupt is inactive and when it is '1', the interrupt is active                                                                  |
| VICRawIntr[31:0]     | VicInterrupt              | Indicates the status of the interrupts before masking by the enable registers<br><br>There is one bit of the register for each interrupt source<br><br>If the bit is '0', interrupt is inactive before masking and if it is '1', interrupt is active before masking |
| VICINTSOURCE[31:0]   | Peripheral                | Peripheral interrupt source interrupt request                                                                                                                                                                                                                       |
| VICVectAddrVal[31:0] | VicCpuif                  | Vector address from VicCpuif for read by software                                                                                                                                                                                                                   |
| IRQACKTestVal        | VicCpuif                  | Integration test value of VICIRQACK                                                                                                                                                                                                                                 |

| Signal Name          | Source       | Comments                                |
|----------------------|--------------|-----------------------------------------|
| nIRQINTestVal        | VicInterrupt | Integration test value of nVICIRQIN     |
| nFIQINTestVal        | VicInterrupt | Integration test value of nVICFIQIN     |
| VADDRINTestVal[31:0] | VicCpuif     | Integration test value of VICVECTADDRIN |
| VADDRVTestVal        | VicCpuif     | Integration test value of VICVECTADDRV  |
| IRQTestVal           | VicInterrupt | Integration test value of VICIRQ        |
| FIQTestVal           | VicInterrupt | Integration test value of VICFIQ        |
| ACKOUTTestVal        | VicCpuif     | Integration test value of VICIRQACKOUT  |
| VADDRTestVal[31:0]   | VicCpuif     | Integration test value of VICVECTADDR   |
| VICFIQINREG          | System       | Register enable signal for VICFIQIN     |
| VICIRQINREG          | System       | Register enable signal for VICIRQIN     |

**Table 4.4-1 Input signals**

#### 4.4.2 Output signals

| Signal Name        | Destination  | Comments                                                                                                                                                                                                 |
|--------------------|--------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HRESP[1:0]         | AHB Master   | Slave response                                                                                                                                                                                           |
| HREADYOUT          | AHB Master   | Slave ready signal                                                                                                                                                                                       |
| HRDATA[31:0]       | AHB Master   | AHB read data                                                                                                                                                                                            |
| VICSoftInt[31:0]   | VicInterrupt | Indicates the software interrupt<br><br>There is one bit for each interrupt source<br><br>When the bit is '0', the interrupt is inactive<br><br>When the bit is '1', the interrupt is active             |
| VICIntEnable[31:0] | VicInterrupt | Indicates whether the interrupts are enabled or not<br><br>There is one bit for each interrupt source<br><br>When the bit is '0', interrupt is disabled<br><br>When the bit is '1', interrupt is enabled |

| Signal Name                   | Destination               | Comments                                                                                                                                                                                                                              |
|-------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>VICIntSelect[31:0]</b>     | VicInterrupt              | Interrupt select signals<br><br>For each interrupt source, there is one bit in this register<br><br>When '1', the source is FIQ interrupt.<br>When '0', the source is IRQ interrupt.                                                  |
| <b>SWPriorityMask[15:0]</b>   | VicInterrupt              | This register contains the mask value for the interrupt priority levels. There is one bit for each interrupt source. When this bit is '0', the interrupt priority is masked. When this bit is '1', the interrupt level is not masked. |
| <b>VectAddrX[31:0]</b>        | VicCpuif                  | VectAddrX registers contain the ISR vector addresses (X = 0 to 31)                                                                                                                                                                    |
| <b>VectPriorityX[31:0]</b>    | VicCpuif                  | VectPriorityX contains the priority level of each interrupt (X = 0 to 31)                                                                                                                                                             |
| <b>IRQSWAck</b>               | VicCpuif                  | Software IRQ acknowledge                                                                                                                                                                                                              |
| <b>IRQSWClear</b>             | VicCpuif                  | Software IRQ clear                                                                                                                                                                                                                    |
| <b>ITEN</b>                   | VicCpuif and VicInterrupt | Integration test enable                                                                                                                                                                                                               |
| <b>IRQACKForceVal</b>         | VicCpuif                  | Force value for IRQACK. This value will be used during integration tests.                                                                                                                                                             |
| <b>nIRQINForceVal</b>         | VicInterrupt              | Force value for nVICIRQIN. This value will be used during integration tests.                                                                                                                                                          |
| <b>nFIQINForceVal</b>         | VicInterrupt              | Force value for nVICFIQIN. This value will be used during integration tests.                                                                                                                                                          |
| <b>VECTADDRINFrCVal[31:0]</b> | VicCpuif                  | Force value for VECTADDRIN. This value will be used during integration tests.                                                                                                                                                         |
| <b>VECTADDRVFrCVal</b>        | VicCpuif                  | Force value for VECTADDRV. This value will be used during integration tests.                                                                                                                                                          |
| <b>IRQForceVal</b>            | VicInterrupt              | Force value for non-inverted nVICIRQ. This value will be used during integration tests.                                                                                                                                               |
| <b>FIQForceVal</b>            | VicInterrupt              | Force value for non-inverted nVICFIQ. This value will be used during integration tests.                                                                                                                                               |
| <b>IRQACKOUTFrCVal</b>        | VicCpuif                  | Force value for VICIRQACKOUT. This value will be used during integration tests.                                                                                                                                                       |

| Signal Name          | Destination | Comments                                                                             |
|----------------------|-------------|--------------------------------------------------------------------------------------|
| VECTADDRFrcVal[31:0] | VicCpuif    | Force value for VICVETADDRROUT.<br>This value will be used during integration tests. |

**Table 4.4-2 Output signals**

## 4.5 Interrupt Processing (VicInterrupt) top level block

The VicInterrupt block has the following modules:

- Sixteen instantiations of VicPriority module
- Interrupt resolving block (VicIntResolver)

The VicPriority module resolves the interrupts, which have the same priority levels. When two interrupts are asserted simultaneously with the same priority levels, then the hardware priority scheme comes into effect.

The VicIntResolver resolves the interrupts according to the software programmed interrupt levels. It also generates nVICIRQ and nVICFIQ asynchronously.

A block schematic of VicInterrupt is as shown in the **Figure 4.5-1**

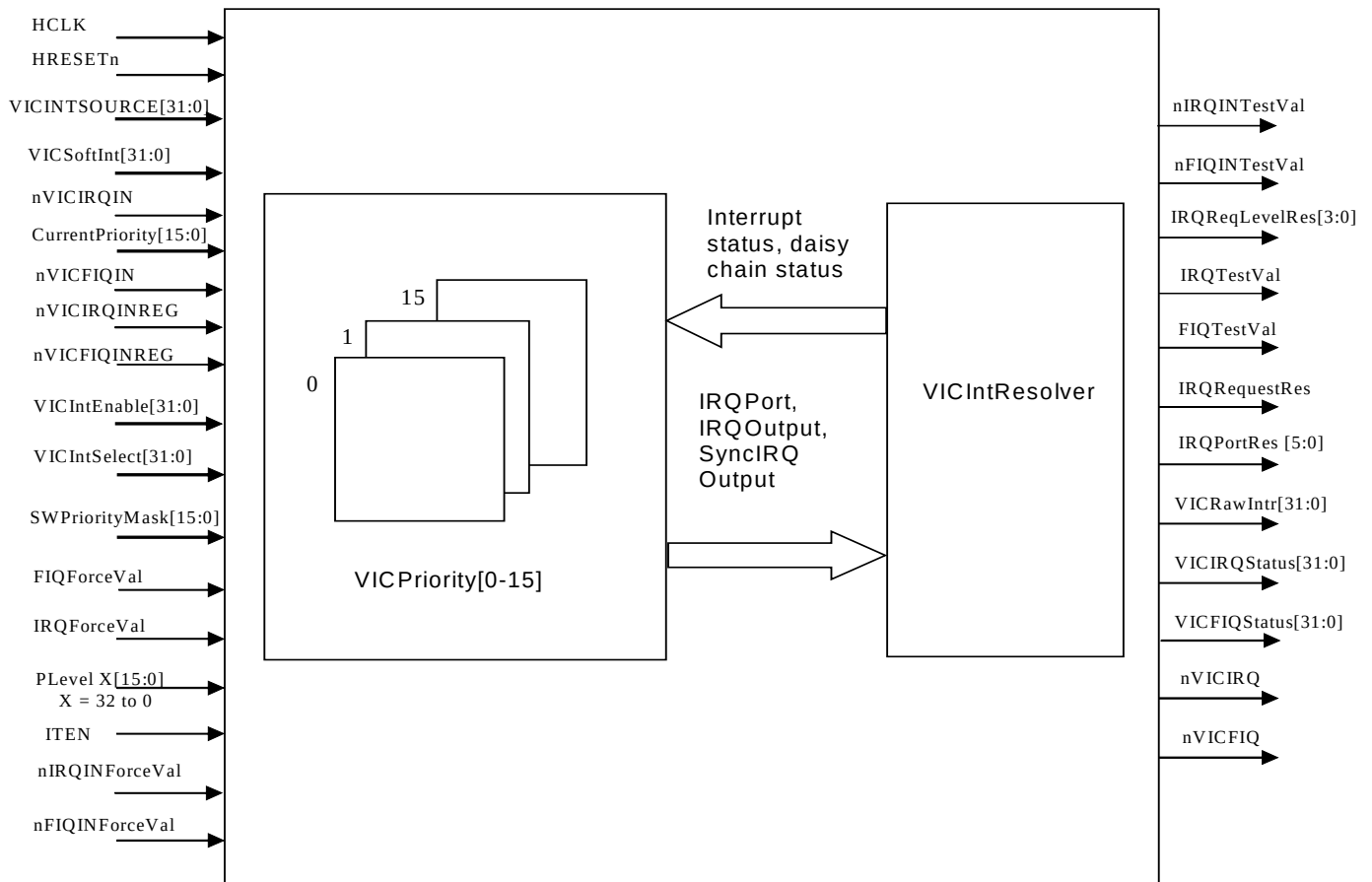


Figure 4.5-1 VicInterrupt Block diagram

#### 4.5.1 Input signals

| Signal Name        | Source                    | Comments                                             |
|--------------------|---------------------------|------------------------------------------------------|
| HCLK               | AHB Master                | AHB clock                                            |
| HRESETn            | AHB Master                | AHB reset                                            |
| VICINTSOURCE[31:0] | Peripheral                | Peripheral interrupt source input request            |
| VICSoftInt[31:0]   | VicAhbif                  | Software interrupt source                            |
| nVICIRQIN          | Daisy chain connected VIC | Daisy chain IRQ input                                |
| nVICFIQIN          | Daisy chain connected VIC | Daisy chain FIQ input                                |
| VICFIQINREG        | System                    | Register enable signal for VICFIQIN                  |
| VICIRQINREG        | System                    | Register enable signal for VICIRQIN                  |
| VICIntEnable[31:0] | VicAhbif                  | Indicates whether the interrupts are enabled or not. |

| Signal Name                  | Source   | Comments                                                                                                                                                                                                                                |
|------------------------------|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                              |          | Each bit is corresponding to each interrupt source.<br><br>When the bit is '0', interrupt is disabled.<br><br>When the bit is '1', interrupt is enabled.                                                                                |
| <b>VICIntSelect[31:0]</b>    | VicAhbif | Interrupt select signals<br><br>There is one bit for each interrupt source.<br><br>When '1', the source is FIQ interrupt<br>When '0', the source is IRQ interrupt                                                                       |
| <b>SWPriorityMask[15:0]</b>  | VicAhbif | This register contains the mask value for the interrupt priority levels. There is one bit for each priority levels.<br>When this bit is '0', the interrupt priority is masked. When this bit is '1', the interrupt level is not masked. |
| <b>PLevelX[3:0]</b>          | VicAhbif | Priority levels of each interrupt source [X = 0 to 32]                                                                                                                                                                                  |
| <b>CurrentPriority[15:0]</b> | VicCpuif | Current interrupt priority                                                                                                                                                                                                              |
| <b>ITEN</b>                  | VicAhbif | Integration test mode enable                                                                                                                                                                                                            |
| <b>nIRQINForceVal</b>        | VicAhbif | Force value for nVICIRQIN                                                                                                                                                                                                               |
| <b>nFIQINForceVal</b>        | VicAhbif | Force value for nVICFIQIN                                                                                                                                                                                                               |
| <b>IRQForceVal</b>           | VicAhbif | Force value for nVICIRQ output                                                                                                                                                                                                          |
| <b>FIQForceVal</b>           | VicAhbif | Force value for nVICFIQ output                                                                                                                                                                                                          |

**Table 4.5-1 Input signals**

#### 4.5.2 Output signals

| Signal Name          | Destination | Comments                                   |
|----------------------|-------------|--------------------------------------------|
| <b>nIRQINTestVal</b> | VicAhbif    | Integration test value of nVICIRQIN        |
| <b>nFIQINTestVal</b> | VicAhbif    | Integration test value of nVICFIQIN        |
| <b>IRQTestVal</b>    | VicAhbif    | Non-inverted VICIRQ output read back value |
| <b>FIQTestVal</b>    | VicAhbif    | Non-inverted VICFIQ output read back value |

| Signal Name         | Destination | Comments                                                                   |
|---------------------|-------------|----------------------------------------------------------------------------|
| nVICFIQ             | Processor   | Interrupt controller Fast interrupt request to processor                   |
| nVICIRQ             | Processor   | Interrupt controller Interrupt request to processor                        |
| IRQRequestRes       | VicCpuif    | Resolved synchronised IRQ request                                          |
| IRQPortRes[5:0]     | VicCpuif    | Resolved IRQ source 0 to 31, VICINTSOURCE (0 to 31), 32, Daisy chain input |
| IRQReqLevelRes[3:0] | VicCpuif    | Resolved priority level of current IRQ request                             |
| VICRawIntr[31:0]    | VicAhbif    | Status of the interrupts before masking                                    |
| VICFIQStatus[31:0]  | VicAhbif    | Status of the FIQ after disabling the interrupt                            |
| VICIRQStatus[31:0]  | VicAhbif    | Status of the IRQ after disabling the interrupt                            |

**Table 4.5-2 Output signals**

## 4.6 VIC-CPU Interface (VicCpuif) block description

The VicCpuif block has the following functionality,

- It generates the handshaking signals for the CPU and the daisy chain VIC.
- Generates the control signals to control the priority logic.
- Generates the address to be put on VICVECTADDR[31:0] lines.

The block diagram of the VicCpuif is as shown in the **figure 4.6-1**.

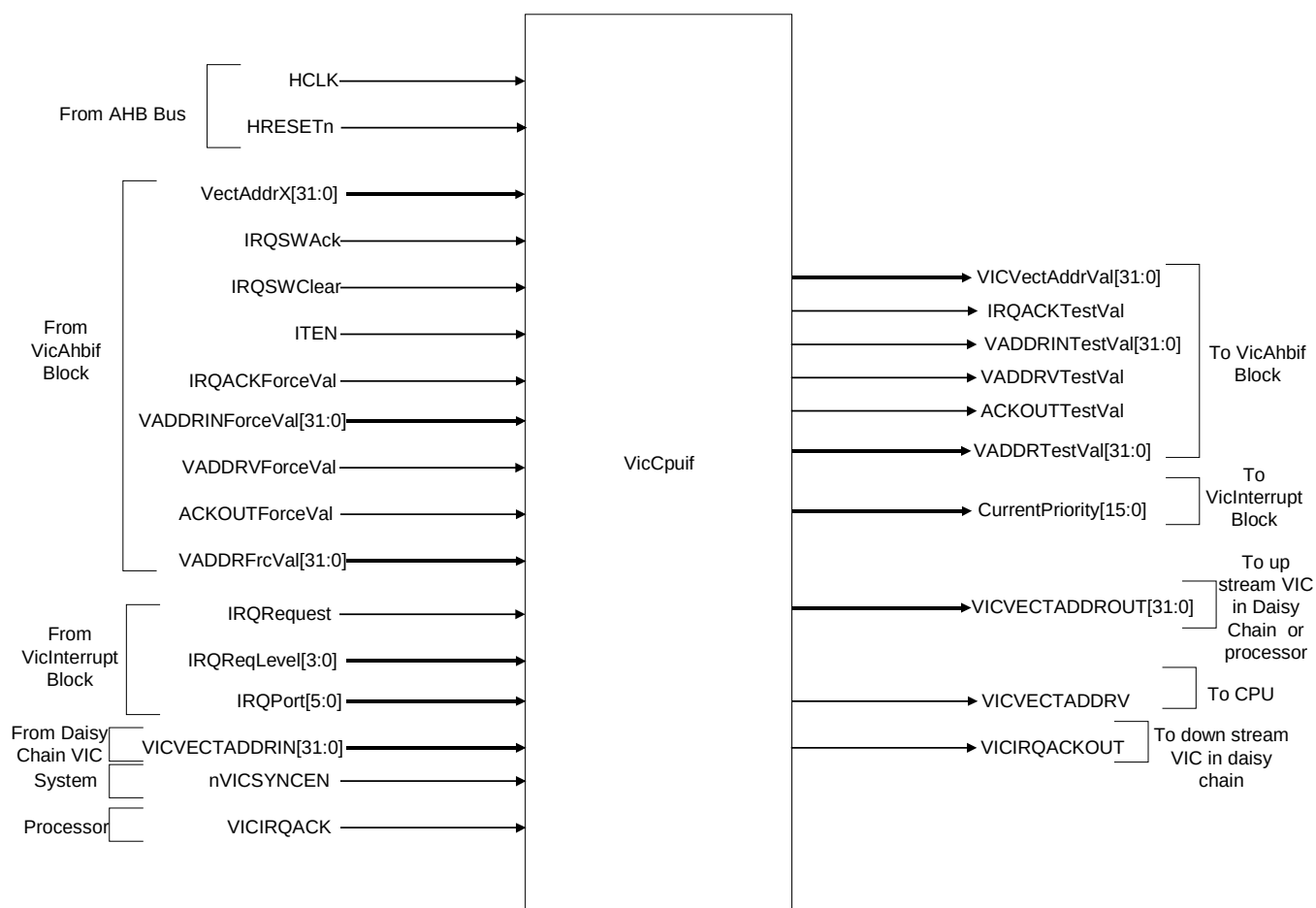


Figure 4.6-1 VicCpuif Block diagram

#### 4.6.1 Input signals

| Signal Name                | Source                    | Comments                                                                  |
|----------------------------|---------------------------|---------------------------------------------------------------------------|
| <b>HCLK</b>                | AHB Master                | AHB clock                                                                 |
| <b>HRESETn</b>             | AHB Master                | AHB reset signal                                                          |
| <b>VectAddrX[31:0]</b>     | VicAhbif                  | ISR address for 32 interrupt sources<br>[X = 0 to 31]                     |
| <b>VICVECTADDRIN[31:0]</b> | Daisy chain connected VIC | ISR address for the interrupt raised<br>by the daisy chain connected VIC. |
| <b>IRQRequest</b>          | VicInterrupt              | Indicates if any interrupt is active or<br>not                            |
| <b>IRQReqLevel[3:0]</b>    | VicInterrupt              | Indicates priority level of the interrupt<br>served                       |
| <b>IRQPort[5:0]</b>        | VicInterrupt              | Indicates the port number of the<br>interrupt being serviced.             |



| Signal Name                  | Source    | Comments                                                                                                                                                                                                                                 |
|------------------------------|-----------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>IRQSWAck</b>              | VicAhbif  | Indicates whether the AHB read is initiated to the VICADDRESS register as an indication that the CPU acknowledges the interrupt.                                                                                                         |
| <b>IRQSWClear</b>            | VicAhbif  | Indicates whether the AHB write is initiated to the VICADDRESS register as an indication that the CPU executes the ISR.                                                                                                                  |
| <b>nVICSYNCE</b>             | System    | Indicates if the VIC port is operated in the synchronous mode or asynchronous mode. When LOW, VIC port signals are synchronised inside the VIC.                                                                                          |
| <b>VICIRQACK</b>             | Processor | Indicates if the CPU is able to take the interrupt. When the nVICIRQ is asserted, the CPU acknowledges the VIC to take the interrupt by asserting the VICIRQACK line. Once the CPU samples the ISR address, it de asserts the VICIRQACK. |
| <b>ITEN</b>                  | VicAhbif  | Indicates the integration test mode. When '1' VIC is in integration test mode. When '0', VIC is in normal mode.                                                                                                                          |
| <b>IRQACKForceVal</b>        | VicAhbif  | Force value for VICIRQACK. This value is used in integration testing.                                                                                                                                                                    |
| <b>VADDRINForceVal[31:0]</b> | VicAhbif  | Force value for VICVECTADDRIN[31:0]. This value is used in integration testing.                                                                                                                                                          |
| <b>VADDRVForceVal</b>        | VicAhbif  | Force value for VICVECTADDRV. This value is used in integration testing.                                                                                                                                                                 |
| <b>ACKOUTForceVal</b>        | VicAhbif  | Force value for VICIRQACKOUT. This value is used in integration testing.                                                                                                                                                                 |
| <b>VADDRForceVal</b>         | VicAhbif  | Force value for VICVECTADDRROUT[31:0]. This value is used in integration testing.                                                                                                                                                        |

**Table 4.6-2 Input signals to VicCpuif block****4.6.2 Output signals**

| Signal Name         | Destination                              | Comments                                                      |
|---------------------|------------------------------------------|---------------------------------------------------------------|
| <b>VICIRQACKOUT</b> | Daisy chain connected to down stream VIC | Indicates if the daisy chain connected VIC is being serviced. |

| Signal Name           | Destination                                         | Comments                                                                              |
|-----------------------|-----------------------------------------------------|---------------------------------------------------------------------------------------|
| CurrentPriority[15:0] | VicInterrupt                                        | Indicates current priority interrupt                                                  |
| VICVectAddrVal[31:0]  | VicAhbif                                            | VIC vector address of the ISR                                                         |
| VICVECTADDRV          | Processor                                           | Indicates if the ISR address on VICVECTADDRROUT is stable.                            |
| VICVECTADDRROUT[31:0] | Daisy chain connected to up stream VIC or processor | ISR address out line.                                                                 |
| IRQACKTestVal         | VicAhbif                                            | Integration test value of VICIRQACK. This value is used in integration testing.       |
| VADDRINTestVal[31:0]  | VicAhbif                                            | Integration test value of VICVECTADDRIN. This value is used in integration testing.   |
| VADDRVTestVal         | VicAhbif                                            | Integration test value of VICADDRV. This value is used in integration testing.        |
| ACKOUTTestVal         | VicAhbif                                            | Integration test value of VICIRQACKOUT. This value is used in integration testing.    |
| VADDRTestVal[31:0]    | VicAhbif                                            | Integration test value of VICVECTADDRROUT. This value is used in integration testing. |

Table 4.6-3 Output signals from VicCpuif block

## 4.7 Revision Designator Block

This module (VicRevAnd) contains a single AND gate to be used as a placeholder cell to mark the revision of the controller. The 2 input pins are tied-off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.

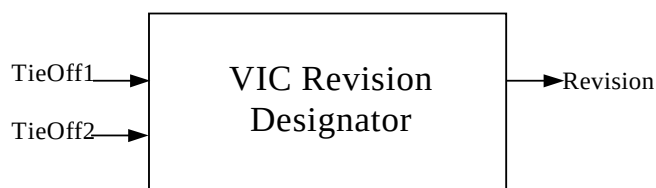


Figure 4.7-1 Block Inputs and Outputs for VicRevAnd

The input and output ports of VicRevAnd are described in **Table 4.7-1** and **Table 4.7-2**.

### 4.7.1 Input signals

| Signal Name  | Source              | Description                                                               |
|--------------|---------------------|---------------------------------------------------------------------------|
| TieOff[2..1] | VIC top-level block | Signals to indicate the values to be used to generate the revision number |

*Table 4.7-1 Block inputs of VicRevAnd*

### 4.7.2 Output signals

| Signal Name   | Source                    | Description                                                                       |
|---------------|---------------------------|-----------------------------------------------------------------------------------|
| Revision[3:0] | Revision Designator block | Revision number information to be used in the PrimeCell identification registers. |

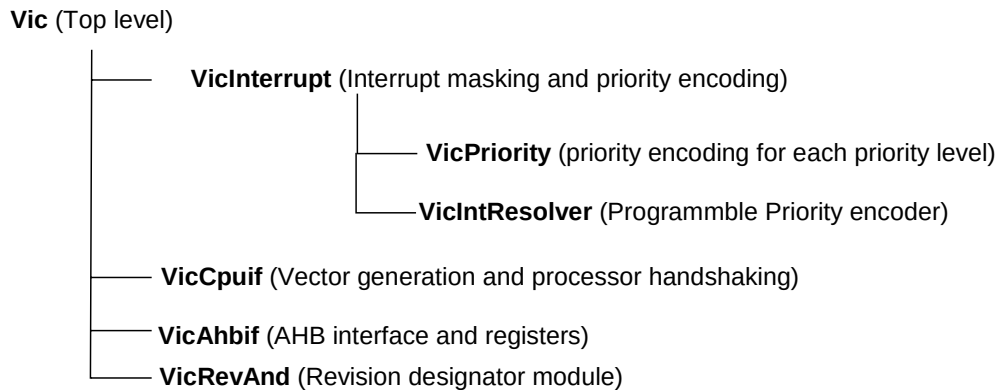
*Table 4.7-2 Block inputs of VicRevAnd*

## 5 VIC IMPLEMENTATION DETAILS

In this chapter the blocks mentioned in the chapter 4 are described.

### 5.1 Hierarchy

The hierarchy in the VIC is shown in **Figure 5.1-1**. A short description about the functionality of each of the sub-modules is also given within the brackets.



**Figure 5.1-1 VIC Design Hierarchy**

### 5.2 AHB Interface Block (VicAhbif)

The AHB Interface (VicAhbif) block consists of registers for the control of the whole vectored interrupt controller, and the AHB interface for these registers. In addition, a protection feature is included. When this feature is enabled, only privileged accesses are allowed.

The VicAhbif is also used to generate an acknowledge signal and interrupt clear signal to the priority control logic. When the VICADDRESS register is read, a signal is sent to the VicInterrupt logic to indicate that the processor has acknowledged the interrupt. This will set the current interrupt priority level to the level of the current interrupt. At the end of the interrupt service routine, the processor performs a write to the VICADDRESS register, resetting the current priority level to the previous state.

#### 5.2.1 AHB Signal decode

The first section of the VicAhbif block is the AHB signal decoding. If a valid transfer is detected, the circuit will generate the registered address and the control information, as well as two registered signals named ReadEnable and WriteEnable. The ReadEnable and WriteEnable signals are used to control the access of registers. If the protection feature is enabled (VicProtection=1) and the access is in user mode, then the ReadEnable and the WriteEnable signals are not asserted.

The vector address registers and priority registers are one wait stated registers. When a read or write operation is initiated to these registers, then HREADYOUT will be pulled low for one clock. Other registers are all zero wait stated registers.

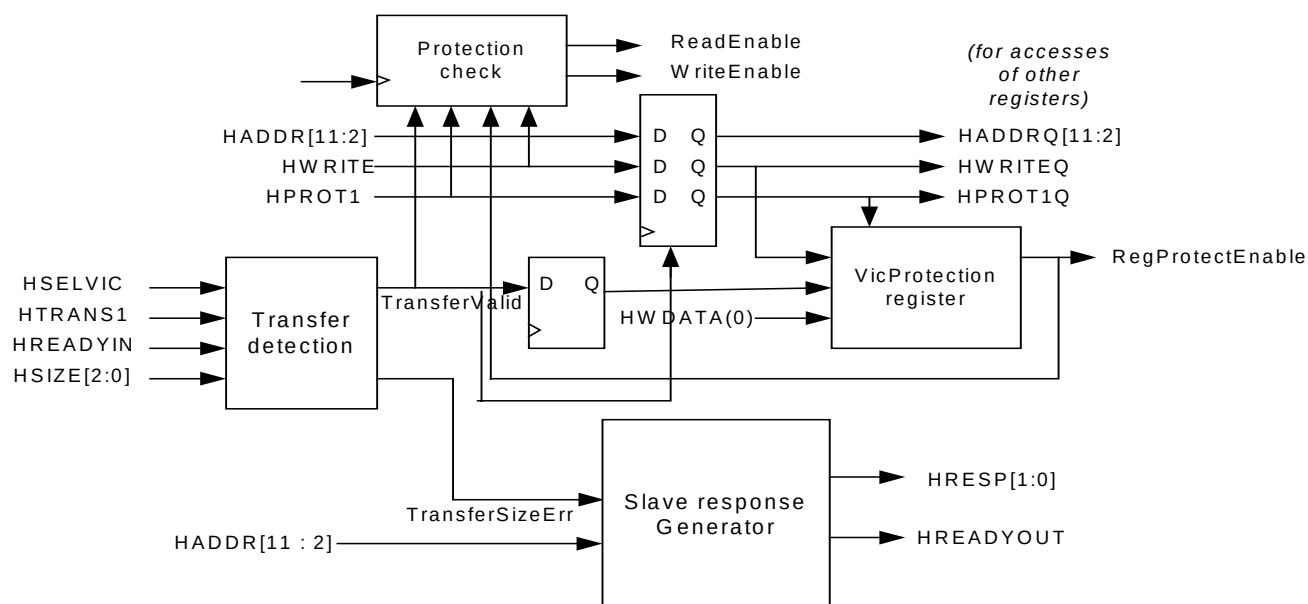


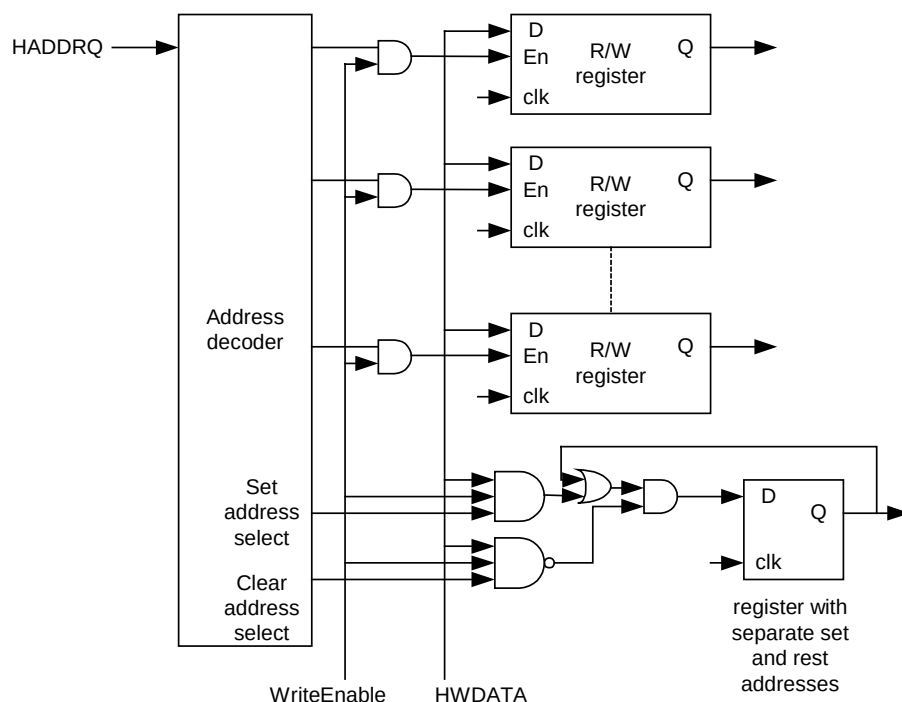
Figure 5.2-1 AHB interface

## 5.2.2 Registers write

The read-write registers in the VIC are divided into two types: simple read-write registers and read-write registers with separate set and clear addresses. The vector address registers (VICVECTADDR0-31) and vector priority registers (VICVECTPRIORITY0-31) are examples of simple read-write registers, and software interrupt registers (VICSOFTINT/VICSOFTINTCLEAR) are examples of registers with separate set and clear addresses.

The registers can also be divided into two types depending on the number of wait states inserted for the access. The vector address registers (VICVECTADDR0-31) and vector priority registers (VICVECTPRIORITY0-31) are one wait stated registers. I.e. when these registers are accessed, HREADYOUT will be pulled low for one clock period. All other registers are zero wait stated registers.

The implementation of a register write is illustrated in the following diagram:

**Figure 5.2-2 Register write**

The value of HWDATA is stored in the read-write registers when the WriteEnable is high and the correct address for that register has been decoded. In case of read-write registers with separate set and clear addresses, the input to the register is gated by the set address select and clear address select.

In case of one wait stated registers, when a write is initiated, HREADYOUT is pulled low and the AHB write is extended by one more clock. Note that, the way of registering the data during register write is same in both zero wait stated and one wait stated registers.

### 5.2.3 Register Read

For a register read, there are two different types of multiplexers depending on the number of wait states inserted. For one wait state registers (VICVECTADDR0-31, VICVECTPRIORITY0-31 registers), when ReadEnable is HIGH and the valid transfer is detected, registered HADDR is decoded and the contents of the registers are clocked and passed on to HRDATA. HREADYOUT is pulled LOW for one clock and the read is extended by one clock.

For zero wait stated registers, when valid read is detected, HADDRQ is decoded, and the corresponding register data is directly passed to the HRDATA without clocking it. In this case, HREADYOUT is kept HIGH throughout the access.

Below table shows the one wait stated registers and zero wait stated registers,

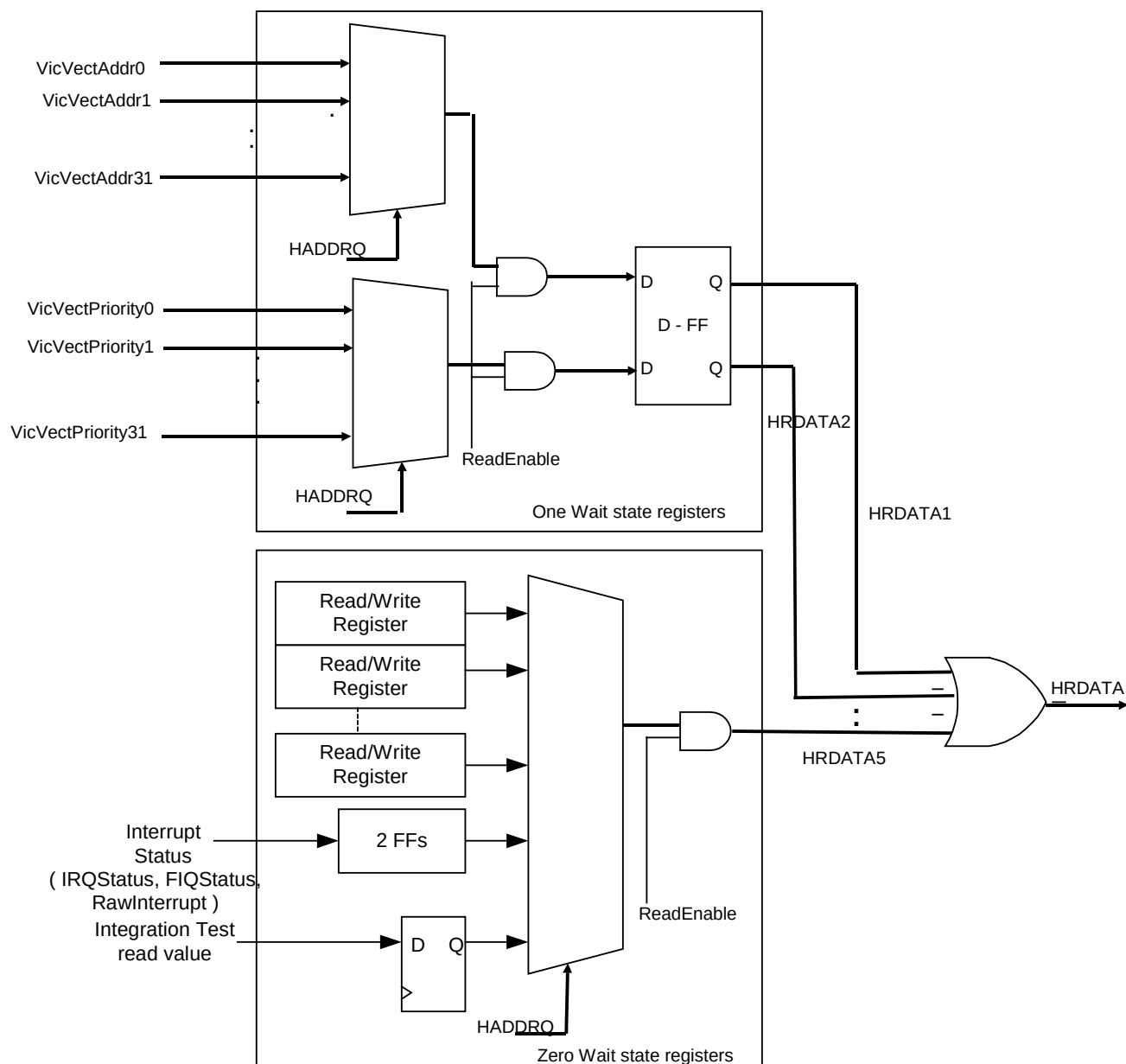
| One wait stated Registers            | Offset Address from Base |
|--------------------------------------|--------------------------|
| VICVECTADDRX[31:0] (X = 0 to 31)     | 0x100 – 0x17C            |
| VICVECTPRIORITYX[31:0] (X = 0 to 31) | 0x200 – 0x27C            |

**Table 5.2-1 One wait stated registers**

| Zero wait stated Registers | Offset Address from Base |
|----------------------------|--------------------------|
| VICIRQSTATUS               | 0x000                    |

| Zero wait stated Registers | Offset Address from Base |
|----------------------------|--------------------------|
| VICFIQSTATUS               | 0x004                    |
| VICRAWINTR                 | 0x008                    |
| VICINTSELECT               | 0x00C                    |
| VICINTENABLE               | 0x010                    |
| VICINTENCLEAR              | 0x014                    |
| VICSOFTINT                 | 0x018                    |
| VICSOFTINTCLEAR            | 0x01C                    |
| VICPROTECTION              | 0x020                    |
| VICSWPRIORITYMASK          | 0x024                    |
| VICVECTPRIORITY<br>DAISY   | 0x028                    |
| VICITCR                    | 0x300                    |
| VICITIP1                   | 0x304                    |
| VICITIP2                   | 0x308                    |
| VICITOP1                   | 0x30C                    |
| VICITOP2                   | 0x310                    |
| VICINTSSTATUS              | 0x314                    |
| VICINTSSTATUSCLEAR         | 0x318                    |
| VICADDRESS                 | 0xF00                    |
| VICPERIPHID0               | 0xFE0                    |
| VICPERIPHID1               | 0xFE4                    |
| VICPERIPHID2               | 0xFE8                    |
| VICPERIPHID3               | 0xFEC                    |
| VICPCCELLID0               | 0xFF0                    |
| VICPCCELLID1               | 0xFF4                    |
| VICPCCELLID2               | 0xFF8                    |
| VICPCCELLID3               | 0xFFC                    |

**Table 5.2-2 Zero wait stated registers**



**Figure 5.2-3 Register read**

There are five multiplexers for HRDATA selection. HRDATA1 and HRDATA2 are the outputs of one wait state register multiplexers. HRDATA3, HRDATA4 and HRDATA5 are the outputs of zero wait state register multiplexers. HRDATA is generated by ORing the output of all five multiplexers.

```
assign HRDATA = (HRDATA1 | HRDATA2 | HRDATA3 | HRDATA4 | HRDATA5);
```

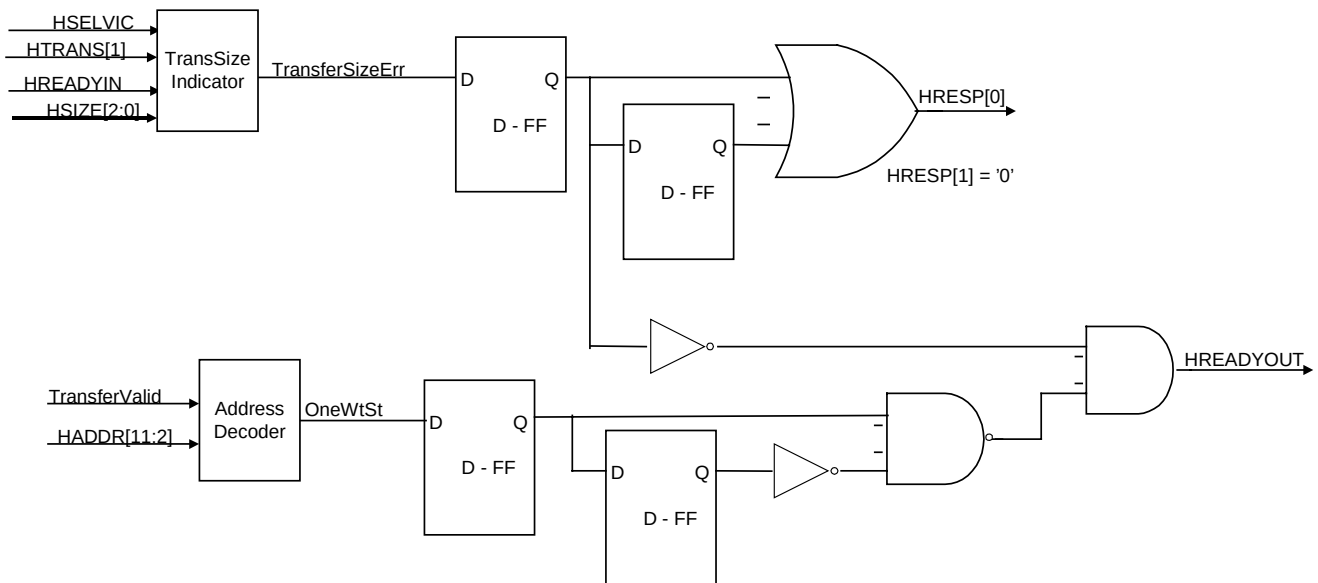
When any of the registers of the particular multiplexers are not selected then the output is driven with zeros to enable the HRDATA to contain the proper data.



The inputs to the interrupt status registers (zero wait stated registers) are synchronised. When a read is initiated to these registers synchronised data will be returned. This avoids metastability problems. Read data for integration tests registers are also synchronised before entering the read data multiplexer to avoid the metastability problems.

### 5.2.4 Slave response generation

The VIC generates the HREADYOUT and HRESP signals according to the access initiated by the master. The slave responses are generated as shown in the below diagram,



**Figure 5.2-4 Slave response generations**

When the master initiates the register access with HSIZE other than WORD, the VIC returns the two clock ERROR response. In all other cases the VIC returns the OK response.

The generation of HREADYOUT depends on the register being accessed. When one wait state registers are accessed, HREADYOUT is pulled low for one clock. For all other registers no wait states are inserted.

During the first clock of an ERROR response, the HREADYOUT is pulled low for one clock when the registers are accessed with HSIZE other than WORD.

#### HRESP generation:

Signal TransferSizeErr is used to generate an ERROR response to the AHB master. This signal becomes high when the VIC is selected (HSELVIC = '1'), HTRANS is either SEQ or NSEQ, HREADYIN is HIGH and HSIZE is other than WORD.

TransferSizeErr is registered two times and TransSizeErrQ[0] (clocked version of TransferSizeErr) and TransSizeErrQ[1] (clocked version of TransSizeErrQ[0]) are generated. When the registers are accessed with HSIZE other than WORD, TransSizeErrQ[1:0] will contain non zero values for the next two clocks. When the TransSizeErrQ[1:0] has non zero value, ERROR response is given to the access.

#### HREADYOUT generation:

When the VICVECTADDR0-31 and VICVECTPRIORITY0-31 registers are accessed, the HREADYOUT is pulled low for one clock. For other register accesses, no wait states are inserted. When the ERROR response is generated, the HREADYOUT is pulled low for one clock period.

When a valid transfer is initiated to the one wait stated register, signal OneWaitSt is generated. This signal is clocked two times and OneWaitSt1Q, OneWaitSt2Q are generated. Then the HREADYOUT is generated as shown in the logic equation.

$$\text{HREADYOUT} = ((\sim(\text{OneWaitSt1Q} \&\& \sim(\text{OneWaitSt2Q}))) \&\& \sim(\text{TransSizeErrQ}[0]))$$

Where TransSizeErrQ[0] is the clocked version of TransferSizeErr. During the first clock of ERROR response TransSizeErrQ[0] will be HIGH. This is used to pull the HREADYOUT LOW during the first clock of the ERROR response.

### 5.2.5 Generation of Control signals

Reading and writing of the VICADDRESS register will trigger the interrupt priority circuit in VicInterrupt block. The control signals IRQSWAck and IRQSWClear are generated for this purpose in the VicAhbif as shown in the **Figure 5.2-5**. A read of this register returns the address of the ISR and sets the current interrupt as being serviced. A read must only be performed while there is an active interrupt.

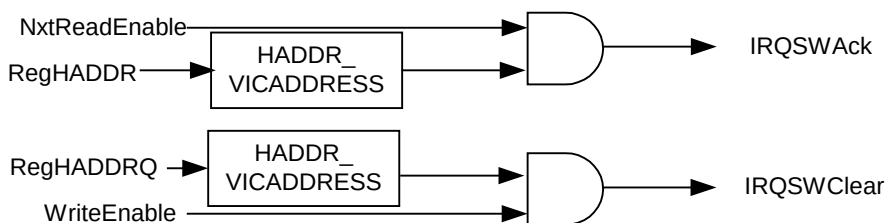
A write of any value to this register clears the current interrupt. A write must only be performed at the end of an interrupt service routine.

IRQSWAck is generated in the address phase of read to the VICADDRESS register, which indicates that the CPU has acknowledged the interrupt. It is generated as shown below

```
assign IRQSWAck = ((NxtReadEnable && (HADDR == `HADDR_VICADDRESS)) ? 1'b1: 1'b0);
```

Similarly, IRQSWClear is generated in the data phase of a write to the VICADDRESS register, which indicates that the interrupt, has already executed. It is generated as shown below,

```
assign IRQSWClear = ((WriteEnable && (HADDRQ == `HADDR_VICADDRESS)) ? 1'b1: 1'b0);
```



**Figure 5.2-5 IRQSWAck and IRQSWClear signal generation**

IntSelect[31:0] is the content of the VICIntSelect register. Each bit corresponds to the type of interrupt of the source. When the bit is 0, the selected interrupt is an IRQ interrupt and when the bit is 1, the selected interrupt is the FIQ interrupt. This is the input to the interrupt request logic where it is used to distinguish between the IRQ and FIQ interrupts. IntEnable[31:0] gives the enabled status of the interrupt i.e. whether the interrupt is enabled or not. It is the content of the VICIntEnable register. SoftwareInt[31:0] gives the status of the software interrupts. Each bit corresponds to a distinct software interrupt. When the bit is high, the corresponding interrupt request is asserted. SoftwareIntClear[31:0] signal contains the value of the VICSoftIntClear register. When '1' is written to the bit of VICSoftIntClear register, the corresponding software interrupt bit (content of VICSoftInt) is cleared. VectPriorityX is the priority of the Xth interrupt source. The VicInterrupt block uses this information. SoftPriorityMask is a 16 bit wide signal. It gives the content of the VICSWPriorityMask register. When any bit in this signal is made zero, the interrupt priority level corresponding to that bit will be masked.

The VicAhbif block also generates the clocked version of HADDR, HWRITE, HSIZE, HTRANS, HSELVIC, and HWDATA. This block also returns the value of the peripheral ID when the corresponding register is read. The PERIPHID registers are read only registers where the values of these registers are version dependent.

The register VICIntSStatus shows the status of the raw interrupts, which can be used to know the status of the interrupts, that are deasserted before they have been disabled. This register is used in the integration test environment.

When ISS bit is high, the VICIntSStatus register is updated with the latest synchronised version of VICINTSOURCE value. The signal generation looks like below,

$$\text{assign NxtIntSStatus} = (\text{IntSStatusQ} \mid \text{VICIntSourceQ2});$$

Where IntSStatusQ is the clocked output of IntSStatus. This equation ensures that, if the interrupt is disabled before being serviced, then the status of that interrupt can be found out with the IntSStatus register.

The value in the IntSStatus register is cleared when the corresponding bit in the VICIntSStatusClear is written with '1' and it is implemented as below,

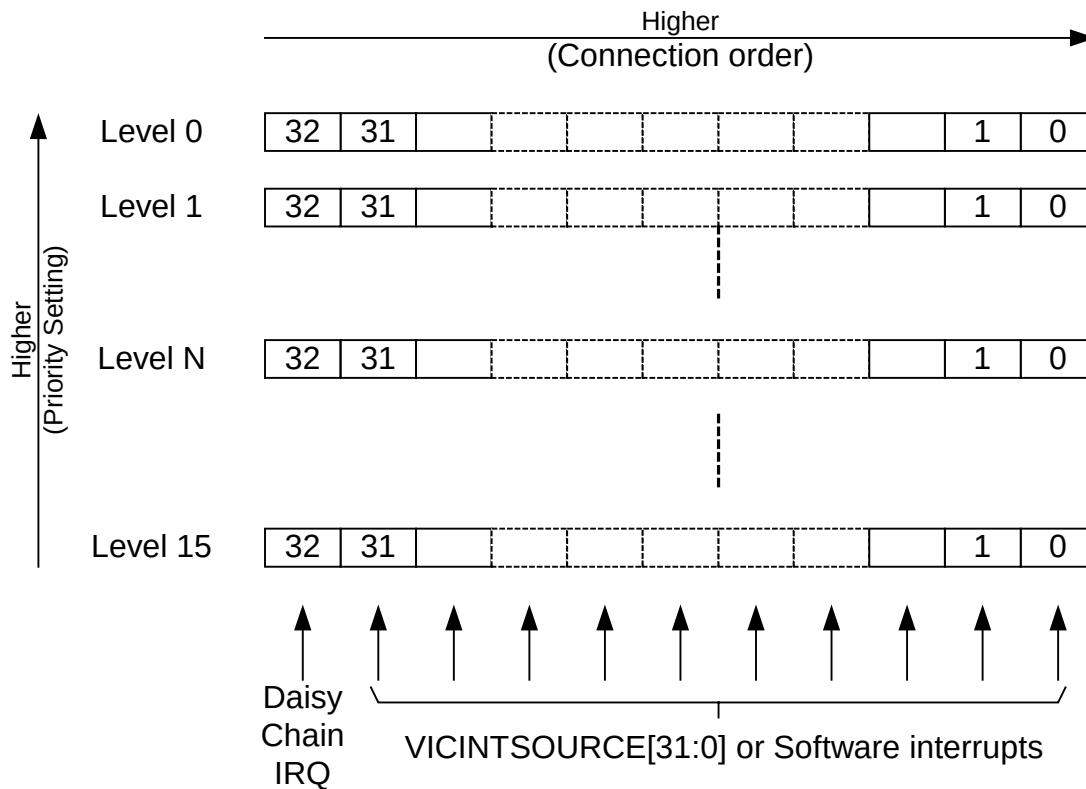
$$\text{NxtIntSStatus} = ((\text{IntSStatusQ} \mid \text{VICIntSourceQ2}) \& \sim(\text{HWDATA}));$$

Force values for the integration testing are generated in this block by connecting the appropriate signal to the integration registers bits.

## 5.3 Interrupt Processing Block (VicInterrupt)

### 5.3.1 Priority Scheme

The VIC PL192 supports 16-levels of programmable priority interrupts. Each interrupt source (including the daisy chained IRQ input) has a Priority setting register. The priority scheme of the design can be viewed as below:



**Figure 5.3-1 Priority scheme for interrupt processor**

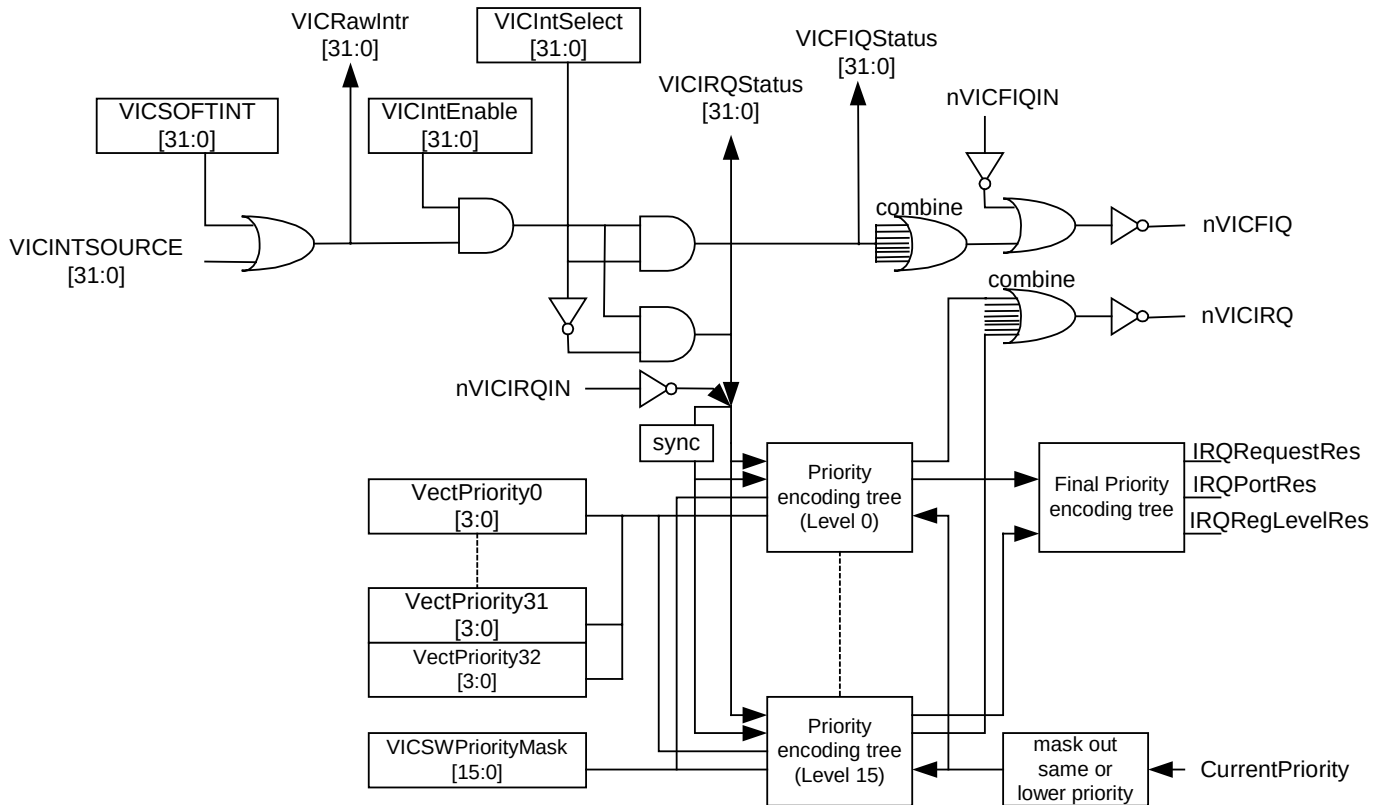
When multiple interrupt sources are asserted at the same time, the interrupt source with the higher priority (level 0 is highest and level 15 is lowest) will be output by the VicInterrupt block. If the interrupt sources have the same priority level, then the interrupt used is determined by the hardware connection arrangement. (Source 0 will have a higher priority and daisy chain is lowest)

The interrupt priority setting of a VIC is local and is not passed onto an upstream VIC. As a result, all interrupts from daisy chain VIC will appear on the primary VIC in the level specified by the VICVectPriorityDaisy of the primary VIC.

Daisy chain interrupts can be masked by the VICSWPriorityMask register.

### 5.3.2 VicInterrupt Block

The VicInterrupt Block is used to handle masking and priority encoding of incoming interrupts. It accepts 32 interrupt sources and the daisy chain interrupt input, and generates interrupt outputs (nVICFIQ and nVICIRQ). From the synchronised interrupt requests, it generates the interrupt request status, the interrupt port information (to indicate which IRQ source is to be accepted) and the priority level of the current interrupt.



**Figure 5.3-2 Interrupt Processing Block**

The block accepts external interrupt sources and combines these with the software interrupt sources. It also accepts the Daisy Chain nVICFIQIN and nVICIRQIN from the Daisy Chain input. These Daisy Chain inputs are then registered if the VICIRQINREG and VICFIQINREG are high. If they are low, nVICFIQIN and nVICIRQIN are inverted and directly used. This block also determines if an interrupt should be handled as FIQ or IRQ and if the interrupt should be masked or not masked, controlled by the settings of the AHB interface block.

For interrupt sources that are marked as FIQ, the interrupts are OR-ed together and then combined with the Daisy Chain FIQ input. Normally in a system only one interrupt source should be marked as FIQ. In this way the interrupt handler can process the FIQ immediately without taking time to determine the FIQ source.

The VIC also provides un-synchronised interrupt sources so that the interrupt request can reach the processor even when the HCLK is off.

For interrupt sources that are marked as IRQ, the signals will enter the priority encoding circuit to determine which interrupt source would be accepted.

The VicInterrupt block contains two sub-modules called VicPriority and VicIntResolver. VicPriority block is the priority encoding tree in each priority level. Each priority encoding tree reads the interrupt status and checks if the priority level is the same as the local setting and then outputs the highest priority interrupt information and its priority (IRQPort and Priority level). The interrupt request output can be disabled if the corresponding bit in the VICSWPriorityMask is not set.

### 5.3.3 VicPriority block

The block diagram of the VicPriority is as shown below,

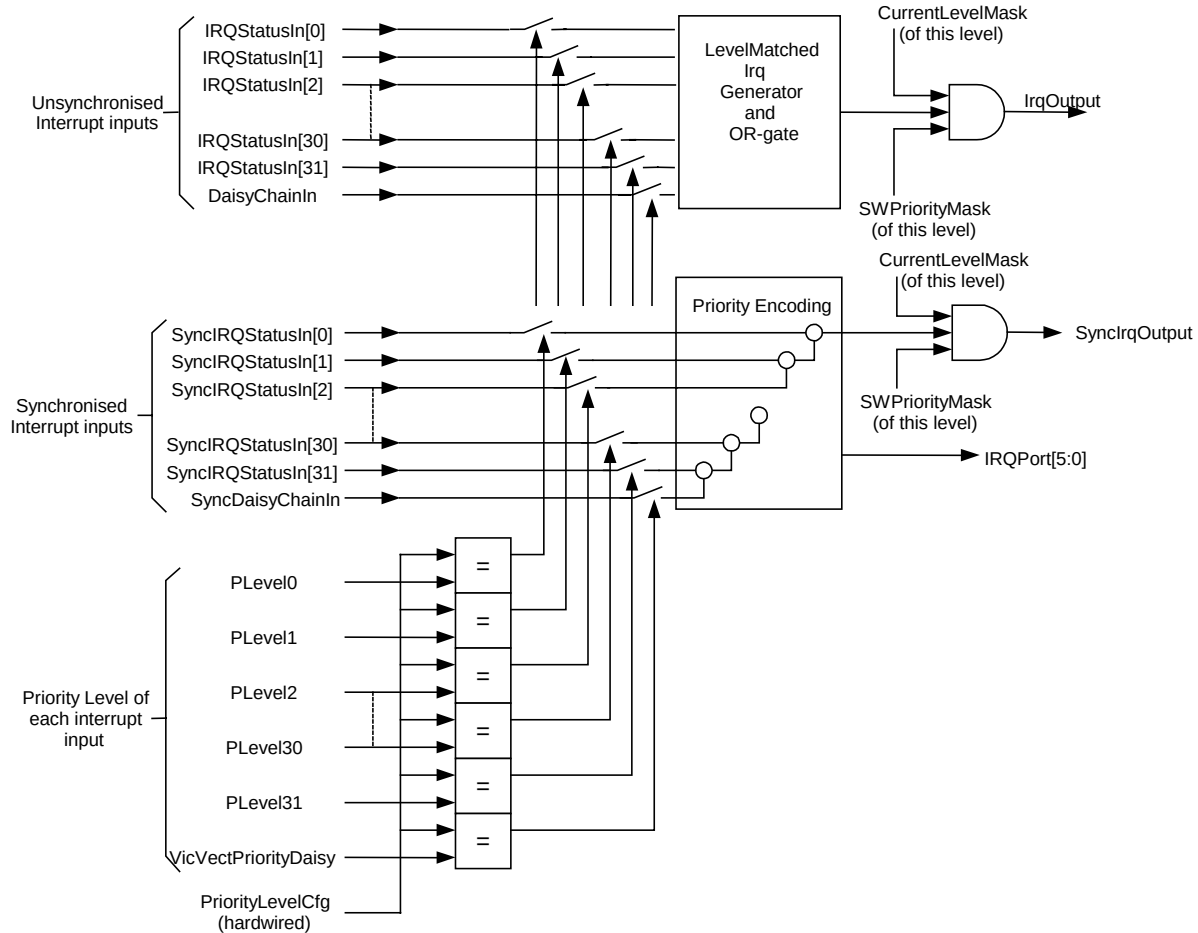


Figure 5.3-3 VicPriority block

In each priority encoder, if there are more than one interrupts with equal priorities, then they are resolved according to the hardware interrupt priority.

When the priority of the interrupt matches the priority level of the priority encoding tree, LevelMatchedIrq[32:0] is updated with the interrupt status. Each bit of LevelMatchedIrq corresponds to each IRQ interrupt (32 IRQ interrupt and one daisy chain interrupt). When LevelMatchedIrq contains non-zero value and if that particular priority level is not masked either by the software or because of the lower priority than the currently serving interrupt then assert the IrqOutput which indicates the interrupt being asserted with priority as the priority level of the current priority encoding tree.

always @(LevelMatchedIrq or SWPriorityMask or CurrentLevelMask)

begin : p\_IrqOutComb

if (((LevelMatchedIrq != 'Zero33) && SWPriorityMask) && CurrentLevelMask)

begin

IrqOutput = 1'b1;

end

else

```

begin
    IrqOutput = 1'b0;
end
end // p_IrqOutComb

```

Similarly, SyncLvMatchedIrq[32:0] in the priority tree is updated when the synchronised interrupt of a particular priority is asserted and the priority level is not masked. Synchronised IRQ is generated as shown below,

```

always @(SyncLvMatchedIrq or SWPriorityMask or CurrentLevelMask)
begin : p_SyncIrqOutComb
    if (((SyncLvMatchedIrq != 'Zero33) && SWPriorityMask) && CurrentLevelMask)
        begin
            SyncIrqOutput = 1'b1;
        end
    else
        begin
            SyncIrqOutput = 1'b0;
        end
    end
end // p_SyncIrqOutComb

```

SyncLvMatchedIrq is used to generate the port information (IRQPort). Each bit of SyncLvMatchedIrq represents the activity of a valid enabled interrupt request. If any bit is high, then the information of that port whose SyncLvMatchedIrq bit is high should be passed to the VicIntResolver.

Port information is generated in a priority encoder tree. Since interrupt source 0 has the highest priority and daisy chain interrupt (interrupt source 33) has the lowest priority, the priority encoder is implemented in such a way that, if more than one interrupt is asserted simultaneously then the highest priority information is passed to the VicIntResolver.

### 5.3.4 VicIntResolver block

The outputs of the VicPriority (16 blocks, each one representing each priority level) are then inputted to the final decoding stage in VicIntResolver block of VicInterrupt. VicIntResolver has a priority tree whose inputs are IRQPort, SyncIrqOutput and IrqOutput from each VicPriority block. This information is used to generate the nVICIRQ, nVICFIQ, CurrentLevelMask, resolved interrupt port information, VICFIQStatus, VICIRQStatus and VICRawIntr.

#### Generation of nVICIRQ and nVICFIQ:

IrqOutput from all VicPriority blocks are clubbed in the signal IrqAsync[15:0]. When any of the bits in the IrqAsync are HIGH, it means that a valid interrupt is being asserted which is to be transferred to the CPU. During integration testing nVICIRQ is to be driven with the value of the registered VICITOP1[7]. So, a multiplexer is used at the generation of the nVICIRQ.

```

assign VICIRQtmux = ((ITEN == 1'b1) ? IRQForceVal : (IrqAsync == 16'b0000000000000000) ? 1'b0 : 1'b1);
assign nVICIRQ = ~(VICIRQtmux);

```

Where IRQForceVal is the value of VICITOP1[7].

ORing the FIQ inputs and the daisy chain nVICFIQIN input generates nVICFIQ. If the VICFIQINREG is HIGH then nVICFIQIN is clocked and used, else non-clocked nVICFIQIN is used. Interrupt enable information and interrupt select information is used to generate the status of FIQ (iFiqStatus[31:0]).

assign iFiqStatus = (MaskedRawIntr & VICIntSelect);

Where, MaskedRawIntr is the enabled interrupt source information.

If the iFiqStatus contains a non zero value then nVICFIQ is asserted.

If the FIQ interrupt from the daisy chain is asserted, then this information is passed as nVICFIQ.

A multiplexer is placed at the end of the logic to select the value of the normal logic or the integration test register value. When ITEN bit is HIGH the multiplexer selects the value of clocked VICITOP1[6].

nVICIRQ and nVICFIQ will be output to processor or to the next VIC.

#### Other control signals:

This block also generates the CurrentLevelMask depending on the Current Priority, which is generated in VicCpuif block. CurrentLevelMask value enables the higher priority interrupts to be generated when lower interrupts are serviced. CurrentLevelMask is generated with the help of the priority encoder.

Resolved interrupt request port number (IRQPortRes) and resolved request level (IRQReqLevelRes) are generated by using the synchronised interrupt information from the sixteen VicPriority blocks by using the priority encoder. If the interrupt with the highest priority programmed is asserted then the information of that port is passed to the VicCpuif block.

The encoded port number output and priority information will be used to generate the vector address.

## 5.4 CPU Interface (VicCpuif)

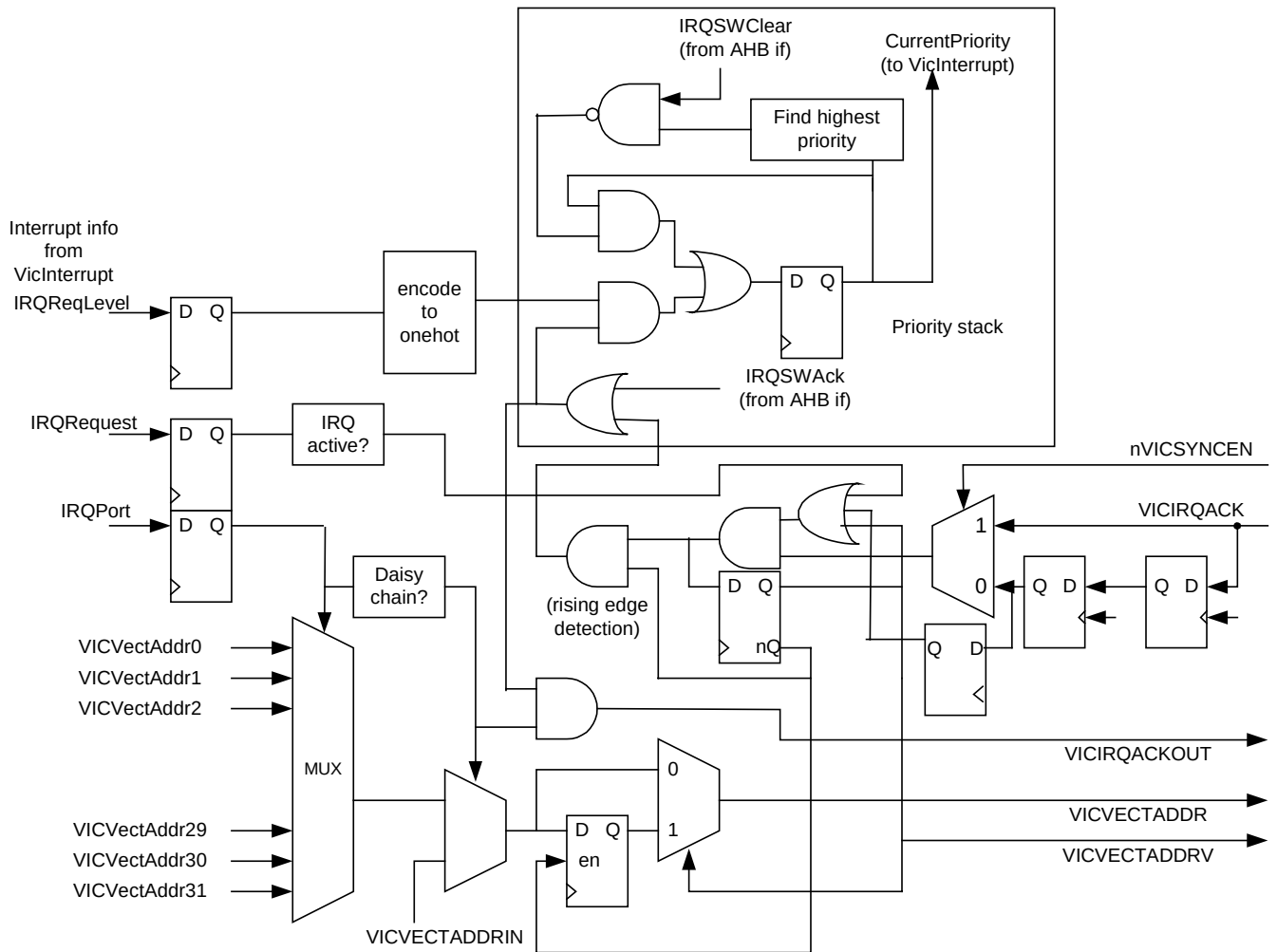
The VIC-CPU interface is used to provide a handshaking signal to the processor and to manage the interrupt priority.

It generates the following signals,

- VICIRQACKOUT signal, as an acknowledge signal for the downstream VIC
- VICVECTADDRV signal which is an address valid signal for the CPU.
- VICVECTADDROUT signal which has the address of the interrupt currently being serviced.
- Current Priority information to the VicInterrupt block,
- Integration test registers values

The diagram of the VicCpuif is as shown below,





**Figure 5.4-1** VIC port interface for CPU handshake signal generation and priority control

#### PriorityStack:

This block has the sixteen bit priority stack to store the priorities which are being serviced or in the pipeline. On reset, the stack contains zeros. If the CPU acknowledges the interrupt by reading the VICADDRESS register or by asserting the VIC port acknowledge signal, the signal StackPush is generated and the current priority interrupt is being stored in the stack. If the CPU completes the interrupt service routine and writes to the VICADDRESS register, StackPop is generated and the highest priority bit is popped and the value of that bit is reset. This is helpful to keep the proper mask value when the two interrupts are acknowledged by the CPU and the ISR of higher priority interrupt is completed and the ISR of second interrupt is not fully executed.

#### Generation of control signals:

The StackPush is generated when either the CPU reads the VICADDRESS register or the internally generated IRQACK is high.

assign StackPush = (IRQSWAckQ | ~(LastIrqAckQ) & LastIrqAckQ);

IRQSWAckQ is the clocked version of the IRQSWAck, which is generated in VicAhbif block. LastIrqAck is the clocked version of the IrqAck and LastIrqAckQ is the delayed version of LastIrqAck. IrqAck is asserted when the VICIRQACK is HIGH and ReadyForAck is HIGH.

When an interrupt is asserted to the CPU, ReadyForAck is generated as shown below,

assign ReadyForAck = (IRQRequestQ | LastIrqAck | Sync3IRQACK);

If this signal is HIGH, then the VIC is ready to take the VICIRQACK signal from the CPU. This signal ensures that the VIC is clocked once before asserting the IrqAck. Syn3IRQACK is VICIRQACK clocked three times this ensures that ReadyForAck is high if the interrupt is asserted for only one clock period. The VIC considers the VICIRQACK signal only when ReadyForAck is high.

StackPop is generated by directly assigning the IRQSWClear to StackPop.

If nVICSYNCE is LOW, then the synchronized VICIRQACK is assigned to IrqAck. If the nVICSYNCE is HIGH then, VICIRQACK is assigned to IrqAck.

CurrentPriority gives the priority of interrupt currently being serviced, which can be decided by considering the value of the PriorityStack.

```
assign CurrentPriority = (LastIrqAck == 1'b0) ? PriorityStack : {16{1'b0}};
```

nVICIRQ signal should be deasserted when the address valid signal (VICVECTADDRV) is deasserted. This can be achieved by the above logic equation for CurrentPriority.

### Generation of CPU and Daisy Chain VIC handshaking signals:

This block generates the VICVECTADDRV signal as an address valid signal to the CPU. This is generated when the ISR address is stable on the VICVECTADDRROUT lines. This is generated as the clocked version of the internally generated acknowledge signal in normal operation. If the ITEN bit is HIGH, then the value of VICITOP1[8] is passed as the VICVECTADDRV value.

```
assign VADDRVtmux = (ITEN ? VADDRVForceVal : LastIrqAck);
```

Assign the VADDRVtmux to VICVECTADDRV signal.

If the VICIRQACK is asserted from the CPU on the very next clock after the interrupt is submitted to the CPU, the VIC is not in a position to handle the VICIRQACK. It takes a minimum of three clock cycles to be ready to receive the acknowledgement from the CPU. I.e. when the ReadyForAck is asserted VICIRQACK is considered. If the ReadyForAck is asserted and if the VICIRQACK is asserted already then, VICVECTADDRV is asserted in the next clock.

When the interrupt being serviced is of the daisy chain connected VIC, then that VIC is informed by asserting the VICIRQACKOUT signal. VICIRQACKOUT is connected to the VICIRQACK line of the daisy chain connected VIC.

During integration test mode, VICIRQACKOUT is connected to the VICITOP1[9] bit. In normal mode the VICIRQACKOUT is generated as shown below,

```
assign VICIRQACKOUT = (IRQSWAck | (IrqAck & ~(LastIrqAck))) & IRQPortBit5Q;
```

Note that IRQSWAck will be high during the address phase of the AHB read to the VICADDRESS register, which makes sure that the priority handling in the daisy chain VIC is synchronised with the VIC nearer to the CPU. IRQPortBit5Q makes sure that the VICIRQACKOUT is asserted only when the daisy chain interrupt is being serviced.

### Generation of VICVECTADDRROUT:

The ISR address is outputted according to the value of IRQPort. At the output there is a multiplexer to select the integration test value or the normal logic value. When any valid interrupt is asserted, the address corresponding to that interrupt is selected. If the interrupt is from the daisy chain, then VICVECTADDRIN is outputted as VICVECTADDRROUT. The ISR address of last selected interrupt is kept on the VICVECTADDRROUT line if there is no new interrupt asserted. It is required not to alter the address when VICVECTADDRV is HIGH. So, when VICVECTADDRV is HIGH, keep the Last vector address is kept on the VICVECTADDRROUT.

### Integration test values:

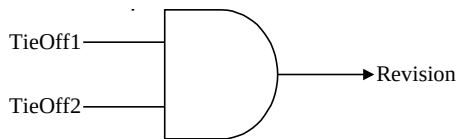
During integration testing, input intra chip signals should have the values of input integration register values. VICIRQACK, VICVECTADDRIN are two intra chip signals that are used in this block. During normal operation, VICITIP registers contain the values of the normal logic. During integration test scenario, these pins are driven with

the integration register values. Therefore it is possible that integration register values can be driven to the normal logic path during integration test.

Similarly, the output pins like VICVECTADDRV, VICIRQACKOUT and VICVECTADDRROUT are multiplexed with the VICITOP register values. During integration mode, VICITOP registers contain the integration test values. And the non-AMBA outputs contain the integration register values.

## 5.5 Revision Designator block (VicRevAnd)

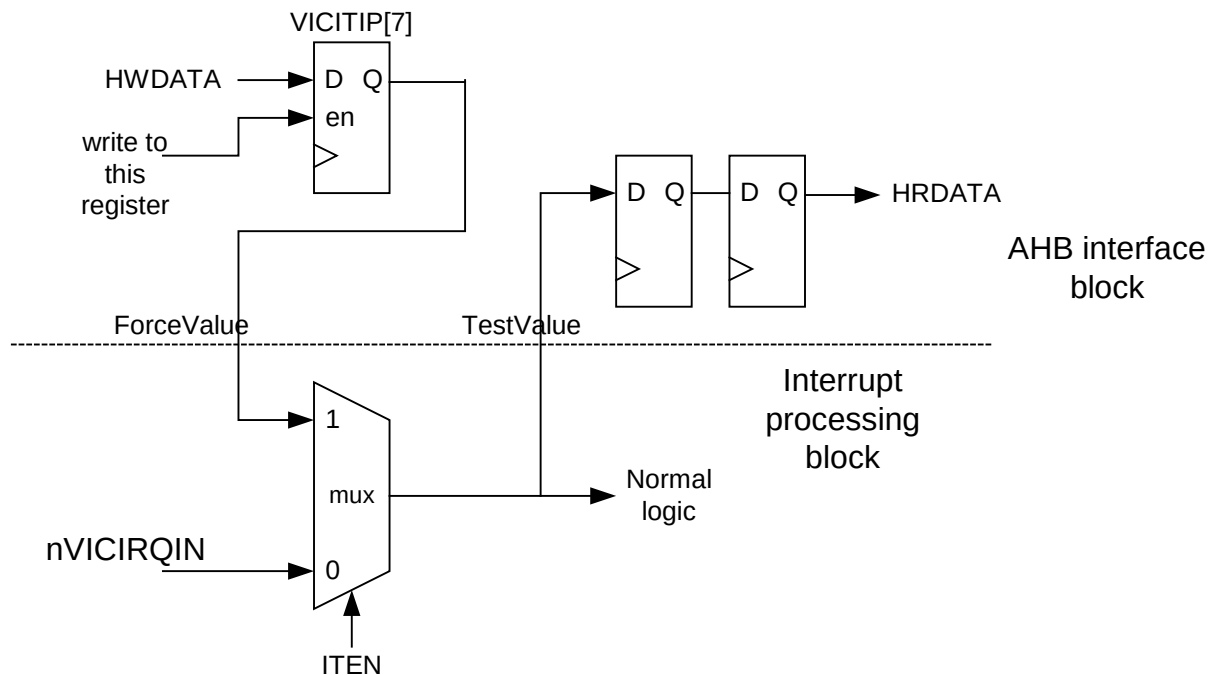
This module contains a single AND gate to be used as a placeholder cell to mark the Revision of the controller. The 2 input pins are tied off at the top level of the hierarchy. These "TieOffs" can be identified during layout and re-wired to "VDD" or "VSS" if needed.



**Figure 5.5-1 Revision AND component**

### 5.5.1 Integration test input registers

The nVICIRQIN, nVICFIQIN and VICIRQACK signals are the intra chip signals, which are outputs of another Daisy Chained VIC. When the ITEN is high, (in integration test mode) the values in VICITIP1 are used in the internal logic.



**Figure 5.5-2 Multiplexing the VICITIP[7] bit and the nVICIRQIN signal**

The figure above shows the multiplexing of the nVICIRQIN signal and the bit 7 of VICITIP1. When the controller is in the integration test mode, the content of VICITIP1[7] is driven as the value of the nVICIRQIN value and will be used internally in the VIC. When a read is performed on this register in test mode, the value of this field is read. When a read is performed on this register when ITEN is 0, it returns the value of nVICIRQIN. The same thing applies to nVICFIQIN bit of VICITIP1 register and the VICVECTADDRIN field of VICITIP2 register.

In test mode, the values of VICFIQINREG and VICIRQINREG are stored in the bit 10 and bit 9 of VICITIP1 register, which are the read only fields.

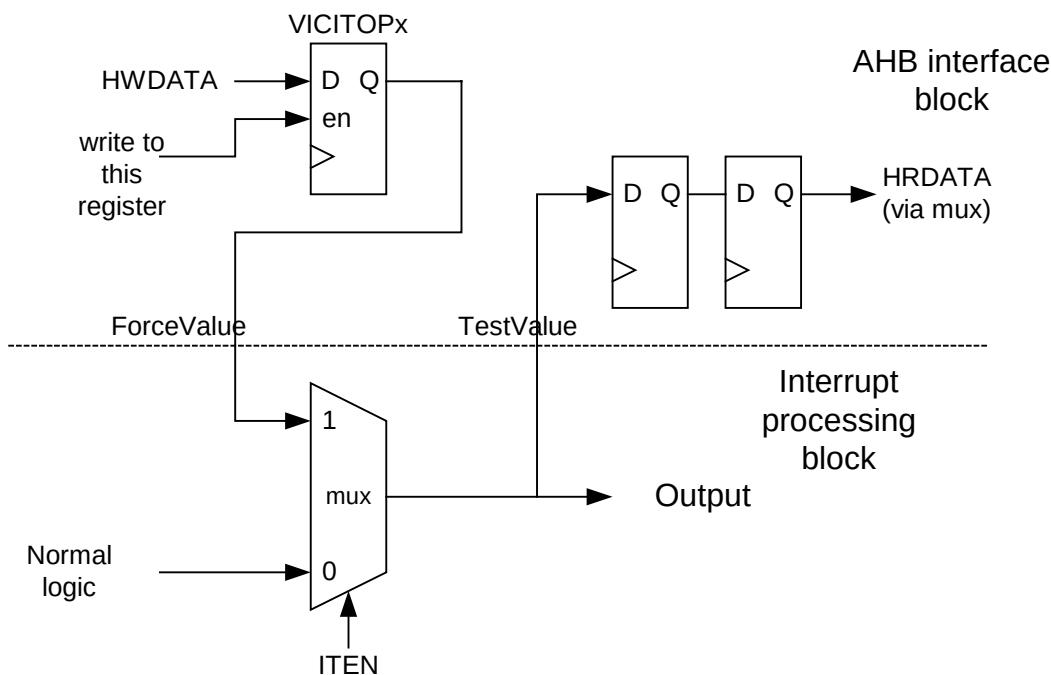
The multiplexing of the integration input registers values will be performed in the VicInterrupt block where all the inputs mentioned above are used.

### 5.5.2 Integration test output registers

There are two output registers VICITOP1 and VICITOP2. VICVECTADDRV, VICIRQ, VICFIQ, VICIRQACKOUT and VICVECTADDRROUT are the outputs of the VIC and are the intra chip signals.

The outputs of the VIC are multiplexed with the values set in the integration registers. During the test mode, the values of the ITOP registers are driven on the output lines. In normal mode, actual values are driven to these fields. When a write is performed on this register, the line corresponding to the bit is driven with the value of the register. When the register is read, it returns the value of this field.

The block diagram shows multiplexing of the VICVECTADDRV output,



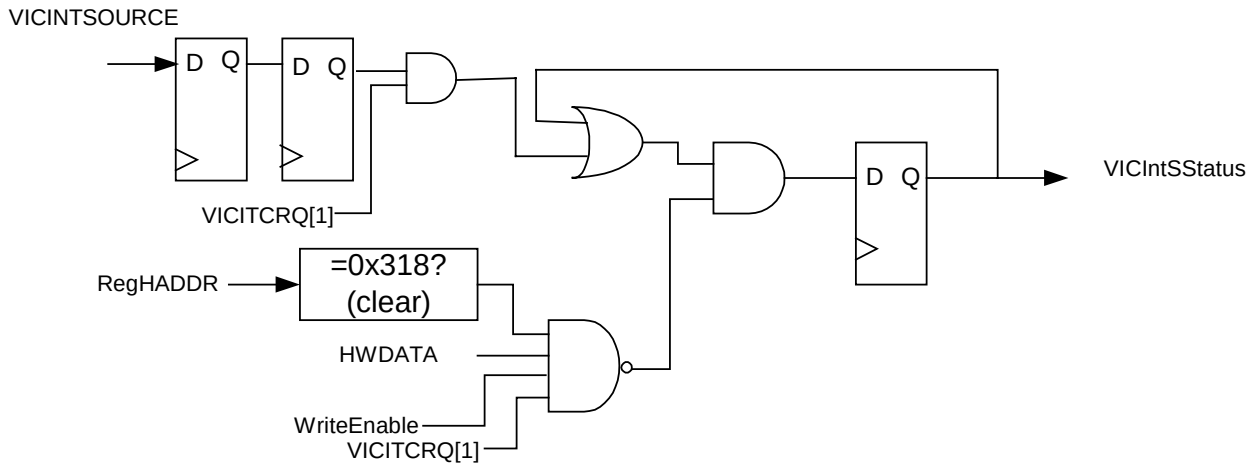
**Figure 5.5-3 Integration test output register MUX**

In the case of nVICIRQ and nVICFIQ signals, the signals generated by the internal logic of VIC and the test register values are multiplexed before the inversion of the VICIRQ and VICFIQ values. During the integration test mode, when the write is performed to the VICITOP1 register, the multiplexer selects the VICITOP1[7] value and drives it on the VICIRQ line. When the register is read during the test mode, the value of this field will be returned. However, in normal mode, the field contains the value of the VICIRQ. The same explanation holds good for the VICFIQ field. These two signals are multiplexed in the VicInterrupt block. However, VICVECTADDRV is multiplexed in the VicCpuif block.

The VIC also has the VICITOP2 register, which has VICVECTADDRROUT as the field. During test mode, when the write is performed on this register, the value of this register is driven as the actual value of the VICVECTADDRROUT lines. When it is read, the value of this field is returned. During normal mode, the value of the VICVECTADDRROUT output is returned when the read operation is performed. This address bus is multiplexed in the VicCpuif block.

### 5.5.3 Integration test output registers

The VICINTSSTATUS register stores the status of the interrupt, which has been asserted irrespective of whether the interrupt has been serviced or not. If the corresponding bit in the VICINTSSTATUSCLEAR register have been written as 1 then the corresponding interrupt source status register bit will be cleared as discussed in the VicAhbif block.



**Figure 5.5-4** VICIntStatus Register value generation

## 6 CUSTOMISATION

The ARM PrimeCell Vectored Interrupt Controller can be modified to suit specific user configurations, but the full benefit of a fast integration will be only obtained when the design is used without modification. In the event of modification, it is the user's responsibility to ensure that the design is fully functional by appropriate amendment of the functional test program and the testbenches where required. Synthesis will have to be rerun to ensure that timing is met.

The following sections describe some of the customisations that are possible in the VIC and estimates of the gate count reduction / addition, the time required and difficulty level of the change and the RTL updates required for each customisation. The gate counts that will be achieved are dependent on the target cell library used, and synthesis timing constraints and command options.

### 6.1 Removing the VIC port

The ARM PrimeCell VIC (PL192) supports VIC port feature in ARM11. If other processors are used it is also possible to remove the VIC port support leaving only software handshaking support.

The following pins are not required, if the VIC port support is to be removed.

- VICIRQACK,
- nVICSYNCEIN,
- VICVECTADDRV

To remove the VIC-port following changes are required in the VicCpuif block,

- Remove IRQACKtmux signal.
- Remove the multiplexer, which selects the VICIRQACK depending on the value of nVICSYNCEIN.
- Generation of VICIRQACKOUT should be modified so that it is devoid of VICIRQACK.
- Remove the logic, which generates VADDRVtmux, which is used as VICVECTADDRV.
- Remove the logic for the generation of ReadyForAck.
- Generation of StackPush depends on the value of VICIRQACK (LastIrqAck). Modify it in such a way that it only depends on software acknowledgement.
- Modify the CurrentPriority generation in such a way that the CurrentPriority always follows the value of PriorityStack value.
- Remove the integration register fields, which are related to the VIC port support.

### 6.2 Removing the Daisy Chain Connection support

The ARM PrimeCell VIC (PL192) supports the daisy chain connection support where more than one VIC can be connected in a Daisy Chain connection fashion.

To remove the daisy chain connection support, following changes are required in the RTL,

Remove the following pins

- nVICFIQIN,
- nVICIRQIN,

- VICVECTADDRIN[31:0],
- VICVECTADDRROUT[31:0],
- VICIRQACKOUT,
- VICIRQINREG,
- VICFIQINREG.

**VicIntResolver block:**

- Remove the generation of DaisyChainIn.
- Modify the generation of nVICFIQ.
- Remove the integration register fields, which are concerned with daisy chain interface.

**VicPriority block:**

- Reduce the size of LevelMatchedIrq[32:0] and SyncLvMatchedIrq[32:0] to LevelMatchedIrq[31:0] and SyncLvMatchedIrq[31:0].
- Remove the logic, which updates the 32<sup>nd</sup> bit of above signals.
- Modify the generation of IRQPort in similar way.

**VicCpuif block:**

- Remove the logic of generation of VICIRQACKOUT.
- Remove the VICVECTADDRIN from the mux, which selects the valid interrupt address.

## 7 VIC VERIFICATION TESTBENCH

The VIC Functional Verification Testbench is an AHB BusTalk Testbench. The VHDL and Verilog BusTalk Testbench files reside in the **verification/vhdl** and **verification/verilog** directories respectively. The test vectors that are applied to the BusTalk Testbench are in the **verification/bustest** directory.

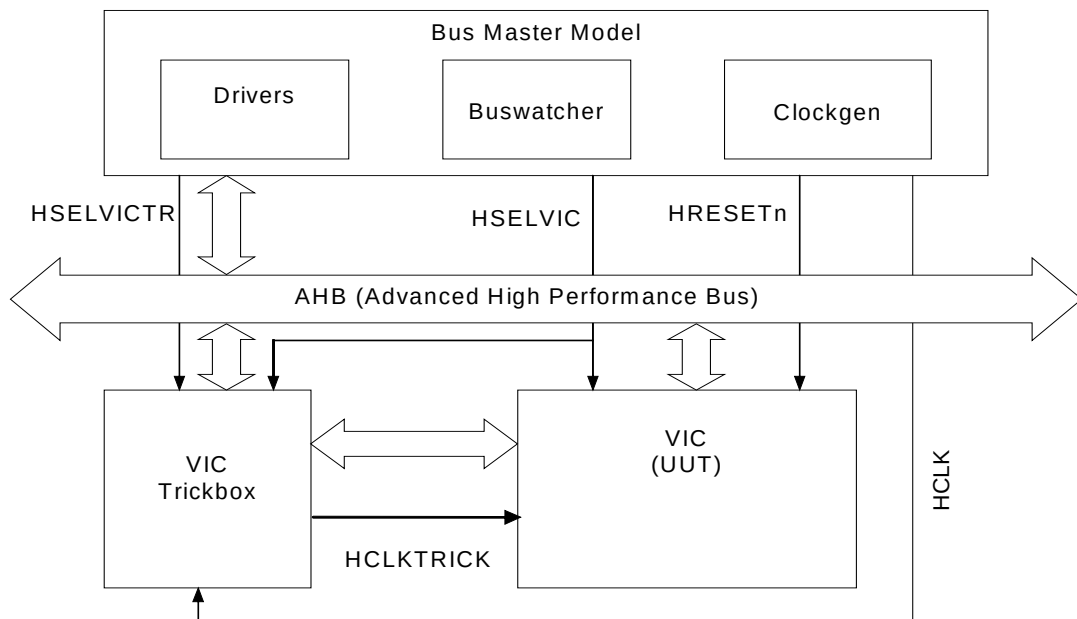
### 7.1 VIC VHDL Verification Testbench

The ARM PrimeCell VIC VHDL test world uses a testbench, which is based on a bus compliance Testbench from the AMBA Compliance Test Suite. For further information, refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref 5].

The Testbench is designed to be used with most common VHDL simulators, and they are suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

The Testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the testbench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific testbench to which it is being targeted.

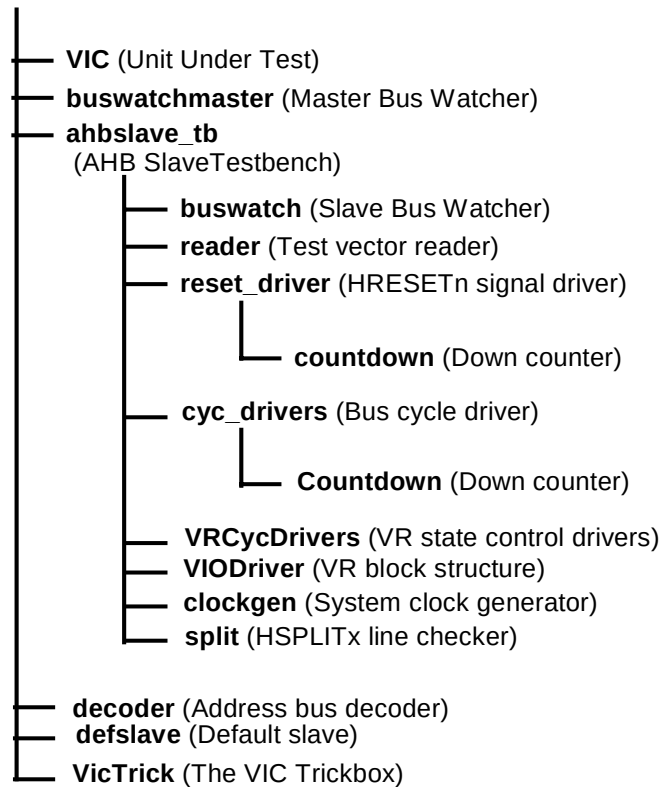
**Figure 7.1-1** illustrates the BusTalk VHDL Testbench for running test vectors.



**Figure 7.1-1** VHDL Functional Verification Testbench

**Figure 7.1-2** illustrates the hierarchy for the VHDL Testbench.



**tbench****Figure 7.1-2 Vhdl Functional Verification Testbench hierarchy**

The testbench, **tbench.vhd** is used for functional testing of the Vectored Interrupt Controller (VIC). This block instantiates the VIC (Unit Under Test), the behavioural model of the Trickbox, the behavioural model of the AHB Master/Arbiter, the decoder, the default slave and the master bus watcher.

The Trickbox drives all the non-AMBA inputs to the VIC. All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the timing.vhd file.

### 7.1.1 Unit Under Test

The Unit Under Test may be one of the following

#### VIC VHDL RTL

#### VIC scaninsert VHDL netlist from VHDL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **verification/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

### 7.1.2 Bus Master Protocol Checker

The Testbench instantiates the master bus watcher, `buswatchmaster.vhd` that checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and will flag warning and error messages if any timing or protocol violations are detected during simulations.

### 7.1.3 AHB Master/Arbiter Model

The UUT is stimulated primarily by an AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.vhd` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the `.bif` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRcycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The Split module verifies that the `HMASTER` number is asserted on the `HSPLITx` lines on the expected clock. The clockgen module generates the system clock, `HCLK`.

The reader reads test vectors from the `infile.bif` residing in the ***verification/bustest/invec*** directory.

### 7.1.4 Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (VIC) is connected to `HSEL [0]` and the Trickbox is connected to `HSEL [1]`.

Any access to the address range

*0x50000000 to 0x5FFFFFFF*

Will access the UUT via the `HSEL [0]` select line.

Accesses to the address range

*0xA0000000 to 0xBFFFFFFF*

Will access the Trickbox via the `HSEL[1]` select line.

Accesses to any address outside the above ranges will select the Default Slave.

### 7.1.5 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its `HREADY` output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

### 7.1.6 Protocol Checker

The Testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.vhd` file in the ***verification/vhdl/tbench*** directory. The buswatch module residing in the ***verification/vhdl/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

### 7.1.7 Trickbox

The functionality of the VIC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the VIC and sample its non-AHB outputs. The Trickbox implements a mirrored model of the VIC within it and continuously compares the locally generated outputs with the actual outputs generated by the VIC.

### 7.1.8 Simulation environment

#### Generics

The VIC Testbench provides some configurability options using generics. All these generics except SuppressOnReset are configurable through the `tbench.vhd` file. SuppressOnReset is configurable through the `ahbslave_tb.vhd` file in the ***verification/vhdl/tbench*** directory.

#### - XonSig

XonSig is used in the `cyc_drivers.vhd` file that resides in ***verification/vhdl/bid***.

This generic is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals transition to 'X' when the line driver is idle.

#### - Verbosity

Verbosity is used in the `reset_driver.vhd` and the `cyc_drivers.vhd`, which reside in ***verification/vhdl/bid***. It is also used in the `buswatch.vhd` file that resides in ***verification/vhdl/buswatcher*** and the `VIODrivers.vhd`, which reside in ***verification/vhdl/vr***.

This generic is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

#### - HaltOnMismatch

This is used in the `cyc_drivers.vhd`, which resides in ***verification/vhdl/bid***, the `buswatch.vhd`, which resides in ***verification/vhdl/buswatcher*** and the `VIODriver.vhd`, which resides in ***verification/vhdl/vr***.

This generic is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged, but the simulation continues.

#### - Databuswidth

Databuswidth is used in the `cyc_drivers.vhd`, which resides in ***verification/vhdl/bid***.

This generic controls the width of the AHB databus. For the VIC Testbench this is set to 32.

#### - SuppressOnReset

SuppressOnReset is used in the `buswatch.vhd`, which resides in ***verification/vhdl/buswatcher***.

This generic is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

## Test Vectors

The test vectors for the verification of the VIC are coded into separate C files, in the **verification/bustest** directory. There are separate test files created for each category of tests mentioned in section 8.

## Running Simulations

The **README** file in the **verification/vhdl/tbench** directory gives step by step instructions for running simulations using the VIC test vectors on the VHDL source code.

Run time logs for all the combinations are available in the **verification/vhdl/Vic\_rtl** and **verification/vhdl/VIC\_scaninsert\_vhdl\_net\_max** directories.

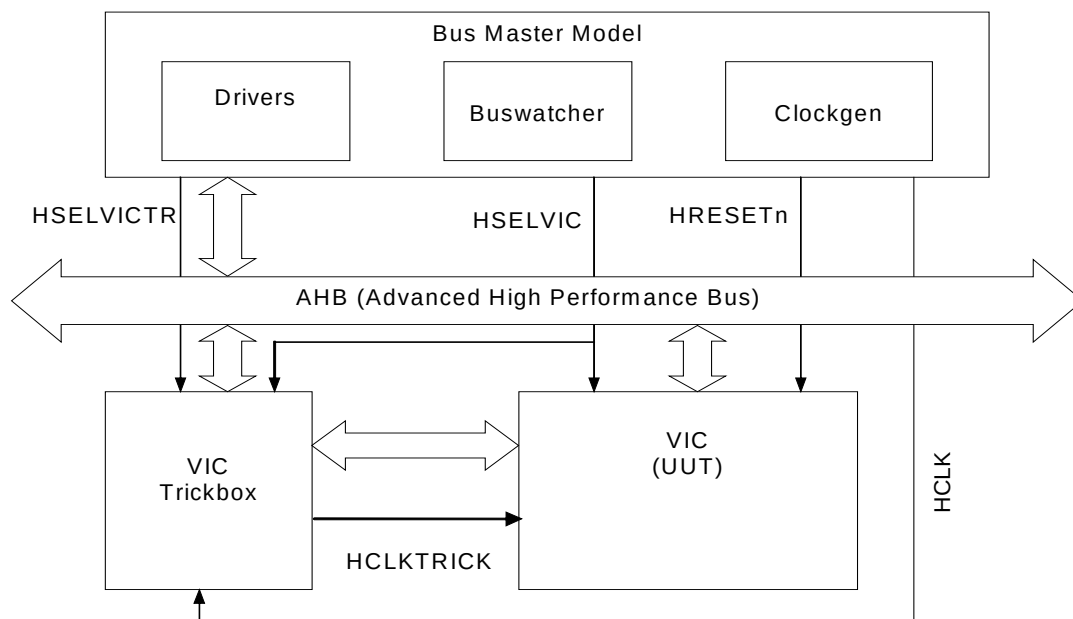
## 7.2 VIC Verilog Verification Testbench

The ARM PrimeCell VIC Verilog test world uses a Testbench, which is based on a bus compliance Testbench from the AMBA Compliance Test Suite. For further information, refer to the ARM Micropack AMBA Compliance Test Suite User Guide [Ref 5].

The testbench is designed to be used with most common Verilog simulators, and it is suited to AMBA system designers who want to ensure the portability of their blocks to any other AMBA designs. This simplifies the ASIC integration task and exchange of peripherals between projects.

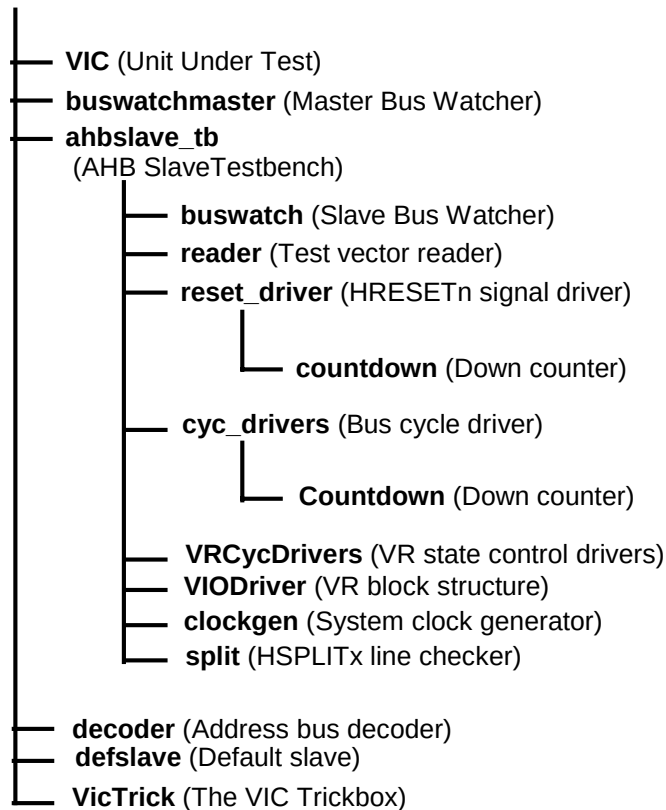
The testbench is “programmable”, meaning that a description of the transfers to be performed on the **Unit Under Test** (UUT) is given without a need to modify the testbench implementation, hence the “generic” property. This transfer description takes the form of a text file that can be read by the specific testbench to which it is being targeted.

**Figure 7.2-1** illustrates the BusTalk Verilog Testbench for simulating test vectors.



**Figure 7.2-1** Verilog Functional Verification Testbench

**Figure 7.2-2** illustrates the hierarchy for the Verilog Testbench.

**tbench****Figure 7.2-2 Verilog Functional Verification Testbench hierarchy****7.2.1 Testbench**

The testbench, **tbench.v** is used for functional testing of the Vectored Interrupt Controller (VIC). This block instantiates the VIC (Unit Under Test), the behavioural model for the AHB masters/arbiters, the behavioural model of the Trickbox, the decoder, the default slave and the master bus watcher.

The Trickbox drives all the non-AMBA inputs to the VIC. All the Scan input pins are tied low to keep them from interfering with the test process.

The HCLK frequency and AHB signal timing budgets are defined in the **timing.v** file.

**7.2.2 Unit Under Test**

The Unit Under Test may be one of the following,

**VIC Verilog RTL****VIC scanready Verilog netlist from Verilog Source****VIC noscan Verilog netlist from VHDL Source**

One out of these three versions may be referenced by creating a link [named **uut**] in the **verification/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

### 7.2.3 Bus Master Protocol Checker

The Testbench instantiates the master bus watcher, `buswatchmaster.v`, which checks the protocol of master accesses initiated via the test code. This module implements the protocol checks for the AHB master's output signals and flags warning and error messages if any timing or protocol violations are detected during simulations.

### 7.2.4 AHB Master/Arbiter Model

The UUT is stimulated primarily by an AHB Master/Arbiter behavioural model. The AHB Master/Arbiter block issues commands on the AHB bus under program control.

The AHB Master/Arbiter model, `ahbslave_tb.v` instantiates the buswatcher, the reader, the reset driver, the cycle drivers, the VR cycle driver, the split checker and the clock generator. The reader block reads the test vectors from the `.sim` file (from the ***verification/bustest/invec*** directory) one line at a time and drives the information about the bus cycle in the form of packets. If the information driven by the reader indicates that the current cycle is a reset cycle, the reset driver drives the `HRESETn` signal with the correct timings. The `cyc_drivers` drive the address, control and data signals for each AHB cycle type, i.e. HSR and HSW etc. The `VRCycdrivers` selects the appropriate VR registers for a read or write operation. The buswatch module implements the protocol checks for the AHB slave's output signals and flags warning and error messages on detection of mismatches. The split module verifies that the `HMASTER` number is asserted on the `HSPLITx` lines on the expected clock. The Clockgen module generates the system clock, `HCLK`.

The reader reads test vectors from the `bif.sim` residing in the ***verification/bustest/invec*** directory.

### 7.2.5 Decoder

This module decodes the addresses and generates the `HSELx` signals for the slaves. It can generate the select signals for a maximum of 16 slaves.

The UUT (VIC) is connected to `HSEL[0]` and the Trickbox is connected to `HSEL[1]`.

Any access to the address range

`0x50000000 to 0x5FFFFFFF`

will access the UUT via the `HSEL[0]` select line.

Accesses to the address range

`0xA0000000 to 0xBFFFFFFF`

will access the Trickbox via the `HSEL[1]` select line.

Accesses to any address outside the above ranges will select the Default Slave.

### 7.2.6 Default Slave

This module drives the response when an access is detected to an unmapped region. The Default slave drives its `HREADY` output high, when other slaves are not selected. It drives a zero wait State OK response for IDLE and BUSY transfers and a two cycle ERROR response for an NSEQ or SEQ transfer access to an unmapped address.

### 7.2.7 Protocol Checker

The Testbench includes a protocol checker, which reports any protocol or timing violations during accesses to any AHB slave. Timing parameters can be set using the `timing.v` file in the ***verification/verilog/tbench*** directory. The buswatch module residing in the ***verification/verilog/buswatcher*** directory checks the Read/Write databus timings and reports any timing violations.

## 7.2.8 Trickbox

The functionality of the VIC is verified via the Trickbox. The Trickbox is a programmable AHB slave designed to drive stimulus on the non-AHB input terminals of the VIC and sample its non-AHB outputs. The Trickbox implements a mirrored model of the VIC within it and continuously compares the locally generated outputs with the actual outputs generated by the VIC.

## 7.2.9 Simulation Environment

### Simulation configuration (Parameters)

The VIC Testbench provides some configurability options via parameters. All the parameters are configurable through the `tbench.v` file.

#### - XonSig

XonSig is used in the `cyc_drivers.v` file that resides in ***verification/verilog/bid***.

This parameter is used to eliminate the 'X's that appear on the address and control signals when these signals are not being actively driven. If XonSig is set to 0 in the top level of the testbench, these signals go to '0' when the corresponding line driver is inactive. If set to 1, the signals transition to 'X' when the line driver is idle.

#### - Verbosity

Verbosity is used in the `reset_driver.v` and the `cyc_drivers.v`, which reside in ***verification/verilog/bid***. It is also used in the `buswatch.v` that resides in ***verification/verilog/buswatcher*** and the `VIODrivers.v`, which reside in ***verification/verilog/vr***.

This parameter is used to control the verbosity of the transcript generated during simulation. If Verbosity is set to 1, every command issued will have a corresponding message in the transcript file. If Verbosity is set to 0, a message is issued only if an error occurs on a read.

#### - HaltOnMismatch

This is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***, the `buswatch.v`, which resides in ***verification/verilog/buswatcher*** and the `VIODriver.v`, which resides in ***verification/verilog/vr***.

This parameter is used to stop the simulation if an error occurs. If set to 1, the simulation halts after an error is reported. If set to 0, the error is flagged but the simulation continues.

#### - SuppressOnReset

SuppressOnReset is used in the `buswatch.v`, which resides in ***verification/verilog/buswatcher***.

This parameter is used to suppress all protocol checks on the slave's output signals when the HRESETn signal is asserted. If SuppressOnReset is TRUE, the set-up and hold timing violations on these signals are not flagged when HRESETn is asserted. If SuppressOnReset is set to FALSE, the set-up and hold timing violations are flagged irrespective of HRESETn. When SuppressOnReset is FALSE the slave's output signals are not checked for 'X'es when HRESETn is asserted.

#### - TestMode

TestMode is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

If the TestMode parameter is set, the Testbench is configured to run in Big Endian mode. By default this parameter is cleared.

#### - Databuswidth

Databuswidth is used in the `cyc_drivers.v`, which resides in ***verification/verilog/bid***.

This parameter controls the width of the AHB databus. For the VIC Testbench this is set to 32.

## Test Vectors

The test vectors for the verification of the VIC are coded into separate C files, in the **verification/bustest** directory. There are separate test files created for each category of tests mentioned in section 8.

## Running Simulations

The **README** file in the **verification/verilog/tbench** directory gives step by step instructions for running simulations using tests on the Verilog source code.

Run time logs for all the combinations are available in the **verification/verilog/Vic\_rtl**, **verification/verilog/VIC\_noscan\_vhdl\_net\_min** and **verification/verilog/VIC\_scanready\_verilog\_net\_typ** directories.

## 7.3 Trickbox Description

The VIC Trickbox is a programmable AHB Slave device designed to provide a means to test the functionality of the VIC. It has one AHB Slave port and can drive the non-AHB input pins of the VIC and sample the non-AHB responses of the same.

The Trickbox core logic (Mirror Logic) mirrors the functionality of the VIC. The outputs of Mirrored Trickbox are compared with outputs of VIC. The Trickbox continuously monitors the output from the VIC UUT and flags error messages whenever a mismatch. The status of the interrupts generated by the VIC can be read back via the Trickbox.

### 7.3.1 Trickbox Signal Description

The VIC Trickbox signals are listed in **Table 7.3-1** and **Table 7.3-2**

| Name        | Type | Source / Destination | Description                                                                                                                                                                 |
|-------------|------|----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HCLK        | IN   | From Testbench       | AHB clock.<br>This clock times all bus transfers. All signal timings are referenced to the rising edge of HCLK.                                                             |
| HRESETn     | IN   | From Testbench       | Bus reset signal, Active Low                                                                                                                                                |
| HADDR[11:2] | IN   | From Testbench       | AHB address Bus                                                                                                                                                             |
| HTRANS[1:0] | IN   | From Testbench       | Indicates the type of the current transfer. The VIC Trickbox does not support the BUSY transfer type. This signal is to be connected to HTRANS [1].                         |
| HSIZE[2:0]  | IN   | From Testbench       | Indicates the size of the current transfer. The VIC Trickbox and the VIC support only WORD transfer size.                                                                   |
| HPROT       | IN   | From Testbench       | Protection Control Signal. This indicates that the transfer to the mirrored register is a supervisor mode or user mode access. This signal is to be connected to HPROT [1]. |
| HWRITE      | IN   | From Testbench       | AHB transfer direction signal, indicates a write access when HIGH, read access when LOW                                                                                     |
| HSELVICTR   | IN   | From Testbench       | Trickbox select signal from decoder. This select signal is used for Trickbox-specific register access.                                                                      |
| HSELVIC     | IN   | From Testbench       | This select signal is used by the Trickbox to access registers that are common to the VIC and the Trickbox.                                                                 |



| Name         | Type | Source / Destination | Description                                                                                                         |
|--------------|------|----------------------|---------------------------------------------------------------------------------------------------------------------|
| HREADYIN     | IN   | From Testbench       | The slave will start the transfer only if HREADY is HIGH.                                                           |
| HWDATA[31:0] | IN   | From Testbench       | AHB Write data bus. During the write operations, HWDATA is used to transfer data from the bus master to the slaves. |
| HREADYOUT    | OUT  | From Trickbox        | HREADYOUT along with an OK response indicates the successful completion of the data transfer.                       |
| HRESP[1:0]   | OUT  | From Trickbox        | The bus transfer response. The Trickbox returns only the OK response.                                               |
| HRDATA[31:0] | OUT  | To Testbench         | AHB read data bus. It is used to transfer from Bus slaves to the master during the read operations.                 |

**Table 7.3-1 AHB signal description**

| Name                   | Type | Source/Destination | Description                                                                                                                                                                                                       |
|------------------------|------|--------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| nVICFIQ                | IN   | From VIC           | nVICFIQ output of the VIC. The value on this input is continuously compared with nVICTrFq signal generated by Mirror Trickbox.                                                                                    |
| nVICIRQ                | IN   | From VIC           | nVICIRQ output of the VIC. The value on this input is continuously compared with nVICTrIrq generated by Mirror Trickbox.                                                                                          |
| VICVECTADDRROUT [31:0] | IN   | From VIC           | VICVECTADDRROUT output of the VIC. The value on this input is continuously compared with VICTrVectAddr generated by Mirror Trickbox.                                                                              |
| VICIRQACKOUT           | IN   | From VIC           | VICIRQACKOUT output of the VIC. The value of the VICIRQACK is passed on this signal. The value on this input is continuously compared with VICTrIrqAckOut signal generated by Mirror Trickbox.                    |
| VICVECTADDRV           | IN   | From VIC           | VICVECTADDRV output of VIC. HIGH on this signal indicates the vector address value is valid and stable. The value on this input is continuously compared with VICTrVectAddrv signal generated by Mirror Trickbox. |
| VICINTSOURCE [31:0]    | OUT  | To VIC             | Interrupt Requests to the VIC are raised via the VICINTSOURCE output of the Trickbox.                                                                                                                             |
| nVICFIQIN              | OUT  | To VIC             | nVICFIQIN Daisy Chain signal to the VIC.                                                                                                                                                                          |
| nVICIRQIN              | OUT  | To VIC             | nVICIRQIN Daisy Chain signal to the VIC.                                                                                                                                                                          |
| VICVECTADDRIN [31:0]   | OUT  | To VIC             | VICVECTADDRIN Daisy chain address to the VIC.                                                                                                                                                                     |
| VICIRQINREG            | OUT  | To VIC             | IRQ Daisy Chain Input configuration.                                                                                                                                                                              |
| VICFICINREG            | OUT  | To VIC             | FIQ Daisy Chain Input configuration.                                                                                                                                                                              |
| VICIRQACK              | OUT  | To VIC             | IRQ acknowledge from the Testbench.                                                                                                                                                                               |
| VICSYNCE               | OUT  | To VIC             | Synchronisation setting                                                                                                                                                                                           |
| HCLKTRICK              | OUT  | To VIC             | Clock to VIC UUT and Mirrored Trickbox. Used to test Clock Off mode.                                                                                                                                              |

**Table 7.3-2 VIC signals**

### 7.3.2 Trickbox Register Description

The base address of the Trickbox is not fixed and may be different for any particular system implementation. However, the offset of any particular register from the base address is fixed.

#### Trickbox Register Summary

Trickbox implements most of the programmable registers of VIC UUT as mirrored registers. The HSELVIC signal is used to access these mirrored registers. HSELVICTR signal is used only when we need to access Trickbox specific registers. When a write is performed on a VIC UUT register, the same is reflected in the Trickbox mirrored register. However when a read is performed on these Trickbox mirrored registers a value zero is returned. Additionally, it does not return responses to bus transfers for which HSELVIC is asserted. This avoids conflict on the AHB databus.

The Trickbox specific registers are shown below

| Address                 | Read Location    | Write Location  |
|-------------------------|------------------|-----------------|
| VIC Trick Base + 0x0050 | --               | VICTrTCR        |
| VIC Trick Base + 0x0054 | --               | VICTrIntSource  |
| VIC Trick Base + 0x0058 | VICTrStatus      | --              |
| VIC Trick Base + 0x005C | VicTrVectAddrOut | --              |
| VIC Trick Base + 0x0060 | --               | VICTrIntIn      |
| VIC Trick Base + 0x0064 | --               | VICTrIntInReg   |
| VIC Trick Base + 0x0068 | --               | VICTrVectAddrIn |
| VIC Trick Base + 0x006C | --               | VICTrSync       |
| VIC Trick Base + 0x0074 | --               | VICTrAckCnt     |
| VIC Trick Base + 0x0078 | --               | VICTrWaitStReg  |

**Table 7.3-3 VIC Trickbox Specific Registers**

The Trickbox mirrored registers are shown below

| Address                  | Read Location      | Write Location     |
|--------------------------|--------------------|--------------------|
| VIC Base + 0x000C        | VICTrIntSelect     | VICTrIntSelect     |
| VIC Base + 0x0010        | VICTrIntEnable     | VICTrIntEnable     |
| VIC Base + 0x0014        | --                 | VICTrIntEnClear    |
| VIC Base + 0x0018        | VICTrSoftInt       | VICTrSoftInt       |
| VIC Base + 0x001C        | --                 | VICTrSoftIntClear  |
| VIC Base + 0x0020        | VICTrProtection    | VICTrProtection    |
| VIC Base + 0x0024        | VICTrSwPriMask     | VICTrSwPriMask     |
| VIC Base + 0x0028        | VICTrVectPriDsy    | VICTrVectPriDsy    |
| VIC Base + 0x0100-0x017C | VICTrVectAddr0-31  | VICTrVectAddr0-31  |
| VIC Base + 0x0200-0x027C | VICTrVectPrity0-31 | VICTrVectPrity0-31 |
| VIC Base + 0x0F00        | VICTrVectAddr      | VICTrVectAddr      |

**Table 7.3-4 VIC Mirrored registers**

Note: A write to the VIC register will automatically update its mirrored register as well. If these mirrored VIC registers are accessed using the VIC Base address, the Trickbox returns zeros on the HRDATA bus. However, if these

registers are accessed using the Trickbox base address (instead of the VIC Base address), their current contents are reflected onto the HRDATA bus.

### 7.3.3 Trickbox Specific Registers Description

All the Trickbox registers are accessible by using Trickbox base address only. A detailed description of each of the VIC Trickbox registers follows:

#### VICTrTCR: Trickbox Test Control Register [6] (+0x050)

This register is used as the test control register. Through this register, the comparison can be enabled or disabled. We can also program this register to test the asynchronous mode of the VIC.

| Bits | Name          | Read/Write | Description                                       |
|------|---------------|------------|---------------------------------------------------|
| 0    | FIQCompEn     | Read/write | nVICFIQ compare enable                            |
| 1    | IRQCompEn     | Read/write | nVICIRQ compare enable                            |
| 2    | VectAdCompEn  | Read/write | VICADDRESS comparison enable                      |
| 3    | AddrvEn       | Read/write | VICVECTADDRV comparison enable                    |
| 4    | AckOutEn      | Read/write | VICIRQACKOUT Acknowledge signal comparison Enable |
| 5    | HCLKOFF       | Read/write | 0 = Clock enable 1 = Clock disable. Reset value 1 |
| 6    | Clock Ratio 0 | Read/write | Clock Ratio Bit 0                                 |
| 7    | Clock Ratio 1 | Read/write | Clock Ratio Bit 1                                 |
| 8    | Clock Ratio 2 | Read/write | Clock Ratio Bit 2                                 |

**Table 7.3-5 VICTrTCR Register**

#### VICTrIntSource: Trickbox Interrupt Source Register [32] (+0x054)

This read/write register is used as the interrupt source for the VIC and Mirror Trick box

| Bits | Name           | Read/Write | Description                                        |
|------|----------------|------------|----------------------------------------------------|
| 31-0 | VICTrIntSource | Read/Write | Interrupt source for the VIC and Mirror Trick box. |

**Table 7.3-6 VICTrIntSource Register**

#### VICTrStatus: Trickbox Status Register [32] (+0x058)

The VICTrStatus register provides the status of nVICIRQ and nVICFIQ outputs from VIC UUT. This register can be read by using the Trickbox base address only.

| Bits | Name    | Read/Write | Function                     |
|------|---------|------------|------------------------------|
| 0    | nVICIRQ | Read       | nVICIRQ status from VIC UUT. |
| 1    | nVICFIQ | Read       | nVICFIQ status from VIC UUT. |

**Table 7.3-7 VICTrIRQStatus Register**

**VICTrVectAddrOut: Trickbox Vector Address Register [32] (+0x05C)**

VICTrVectAddrOut is a read only register, which contains the interrupt service routine vector address of the currently active interrupt from the VIC UUT. This register can be read by using the Trickbox base address only.

| Bits | Name             | Read/Write | Description                                                    |
|------|------------------|------------|----------------------------------------------------------------|
| 31-0 | VICTrVectAddrOut | Read       | Contains the address of the currently active ISR from VIC UUT. |

**Table 7.3-8 VICTrVectAddrOut Register****VICTrIntIn: Trickbox Daisy Interrupt Input Register [2] (+0x060)**

This register generates daisy interrupt inputs nVICIRQIN and nVICFIQIN to VIC UUT and Mirrored Trickbox.

| Bits | Name      | Read/Write | Description                                                          |
|------|-----------|------------|----------------------------------------------------------------------|
| 0    | nVICFIQIN | Read/Write | nVICFIQIN Input to VIC UUT and Mirrored Trickbox. Default value 0x1. |
| 1    | nVICIRQIN | Read/Write | nVICIRQIN Input to VIC UUT and Mirrored Trickbox. Default value 0x1  |

**Table 7.3-9 VICTrIntIn Register****VICTrIntInReg: Trickbox Daisy Interrupt Input Register [2] (+0x064)**

This register generates daisy inputs nVICIRQINREG and nVICFIQINREG to VIC UUT and Mirrored Trickbox. These inputs are used to register the Daisy chain interrupts inputs nVICFIQIN and nVICIRQIN.

| Bits | Name         | Read/Write | Description                                                              |
|------|--------------|------------|--------------------------------------------------------------------------|
| 0    | nVICFIQINREG | Read/Write | nVICFIQINREG Inputs to VIC UUT and Mirrored Trickbox. Default value 0x0. |
| 1    | nVICIRQINREG | Read/Write | nVICIRQINREG Inputs to VIC UUT and Mirrored Trickbox. Default value 0x0. |

**Table 7.3-10 VICTrIntInReg Register****VICTrVectAddrIn: Trickbox Daisy Address Input Register [32] (+0x068)**

This register generates Daisy chain address input to VIC UUT and Mirrored Trickbox.

| Bits | Name            | Read/Write | Description                      |
|------|-----------------|------------|----------------------------------|
| 31-0 | VICTrVectAddrIn | Read/Write | Programmable Daisy chain address |

**Table 7.3-11 VICTrVectAddrIn Register****VICTrSync: Trickbox Synchronization Enable Register [1] (+0x070)**

This register generates nVICSYNEN signal to VIC UUT and Mirrored Trickbox model.

| Bits | Name      | Read/Write | Description                      |
|------|-----------|------------|----------------------------------|
| 0    | VICTrSync | Read/Write | Synchronization Enable register. |

**Table 7.3-12 VICTrSync Register**

**VICTrAckCnt: Trickbox Acknowledge Control register (Daisy Chain) [1] (+0x074)**

This register generates the VICIRQACK signal. The VICIRQACK signal generation is delayed by a value programmed in this register. Default value at reset is 0x00. The VICIRQACK signal generation can be disabled by just programming VICTrAckCnt register with default value.

| Bits | Name        | Read/Write | Description                                           |
|------|-------------|------------|-------------------------------------------------------|
| 2-0  | VICTrAckCnt | Read/Write | Used for delaying the generation of VICIRQACK signal. |

**Table 7.3-13 VICTrAckCnt Register****VICTrWaitStReg: Trickbox Wait State Insertion Register [32] (+0x078)**

The VICTrWaitStReg register inserts n wait states (maximum of 10) when it is accessed. This Non-Zero wait state register has been included to verify the HREADYIN of the VIC going LOW.

| Bits | Name           | Read/Write | Description                                                                         |
|------|----------------|------------|-------------------------------------------------------------------------------------|
| 31-0 | VICTrWaitStReg | Read/Write | Accessible locations with Non-zero wait states. Returns 0x55555555 when it is read. |

**Table 7.3-14 VICTrWaitStReg Register****7.3.4 Trickbox Mirrored Registers Description****VICTrIntSelect: Mirrored Interrupt Select Register [32] (+0x00C)**

The VICTrIntSelect register selects the interrupt request to generate FIQ or IRQ. Since this is a mirrored register, the changes in the corresponding register in the VIC will be reflected here as well.

| Bits | Name           | Read/Write | Function                                                                                                                                                              |
|------|----------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | VICTrIntSelect | Read/Write | Read/Write register used to select whether the corresponding interrupt request generates a FIQ or an IRQ interrupt.<br>0 = IRQ Interrupt(reset)<br>1 = FIQ Interrupt. |

**Table 7.3-15 VICTrIntSelect Register****VICTrIntEnable: Mirrored Interrupt Enable Register [32] (+0x010)**

This register is used to mask all interrupt requests except the External interrupt via the Daisy chain. Write accesses to the corresponding register in the VIC changes the contents of this register.

| Bits | Name           | Read/Write | Function                                                                                                                                                                                                                                                                                                                                                  |
|------|----------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | VICTrIntEnable | Read/Write | Enables the interrupt request lines, which allow the interrupts to reach the processor.<br>Read<br>0 = Interrupt disabled (reset).<br>1 = Interrupt Enabled.<br>The interrupt enable register can only be set using this register. The VICTrIntEnClear register must be used to disable the interrupt enable.<br>0 = no effect.<br>1 = Interrupt enabled. |

| Bits | Name | Read/Write | Function                              |
|------|------|------------|---------------------------------------|
|      |      |            | On reset all interrupts are disabled. |

**Table 7.3-16 VICTrIntEnable Register****VICTrIntEnClear: Mirrored Interrupt Enable Clear Register [32] (+0x014)**

The VICTrIntEnClear register clears bits in the VICTrIntEnable register, and masks out the interrupt sources for the IRQ interrupt.

| Bits | Name            | Read/Write | Description                                                                                                                      |
|------|-----------------|------------|----------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | VICTrIntEnClear | Write      | Clears corresponding bits in the VICTrIntEnable register.<br>0 = no effect<br>1 = interrupt disabled in VICTrIntEnable register. |

**Table 7.3-17 VICTrIntEnClear Register****VICTrSoftInt: Mirrored Software Interrupt Register [32] (+0x0018)**

The VICTrSoftInt register is used to generate software interrupts.

| Bits | Name         | Read/Write | Description                                                                                                                                                                                                                                               |
|------|--------------|------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | VICTrSoftInt | Read/Write | Setting a bit HIGH generates a software interrupt for the selected source before interrupt masking.<br>Read:<br>0 = software interrupt inactive (reset).<br>1 = software interrupt active.<br>Write:<br>0 = no effect.<br>1 = software interrupt enabled. |

**Table 7.3-18 VICTrSoftInt Register****VICTrSoftIntClear: Mirrored Software Interrupt Clear Register [32] (+0x01C)**

The VICTrSoftIntClear write only register is used to clear the VICTrSoftInt Register.

| Bits | Name              | Read/Write | Description                                                                                                                           |
|------|-------------------|------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 31-0 | VICTrSoftIntClear | Write      | Clears corresponding bits in the VICTrIntSoft register.<br>0 = no effect<br>1 = software interrupt disabled in VICTrSoftInt register. |

**Table 7.3-19 VICTrSoftIntClear Register****VICTrProtection: Mirrored Protection Register [1] (+0x020)**

The VICTrProtection register enables and disables protected access, stopping register access when the processor is in user mode.

| Bits | Name            | Read/Write | Description                                    |
|------|-----------------|------------|------------------------------------------------|
| 0    | VICTrProtection | Read/Write | Enables or disables protected register access. |

| Bits | Name | Read/Write | Description                                                           |
|------|------|------------|-----------------------------------------------------------------------|
|      |      |            | 0 = protection mode disabled (reset).<br>1 = protection mode enabled. |

**Table 7.3-20 VICTrProtection Register****VICTrSwPriMask: Mirrored Software Priority Mask register [16] (+0x024)**

The VICTrSwPriorityMask register contains the mask value for the interrupt priority level.

| Bits | Name           | Read/Write | Description                                                                                                                                                                                                                  |
|------|----------------|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 15-0 | VICTrSwPriMask | Read/Write | Contains the mask value for the interrupt priority levels.<br>0 = Priority level is masked.<br>1 = Priority level is not masked (reset).<br>Each bit of the register is applied to each of the 16-interrupt priority levels. |

**Table 7.3-21 VICTrSwPriMask Register****VICTrVectPriDsy: Mirrored Vectored Priority Daisy register [4] (+0x028)**

The VICTrVectPriDsy register selects the interrupt priority level for Daisy IRQ Interrupt. The value can be 0 to 15 with 15 being the lowest priority.

| Bits | Name            | Read/Write | Description                                                                              |
|------|-----------------|------------|------------------------------------------------------------------------------------------|
| 3-0  | VICTrVectPriDsy | Read/Write | This register contains the programmed priority value for the Daisy IRQ interrupt source. |

**Table 7.3-22 VICTrVectPriDsy Register****VICTrVectAddr0-31: Mirrored Vector Address 0-31 Registers [32] (+0x100-0x17C)**

The VICTrVectAddr0-31 registers are read/write registers, which contain ISR vector addresses.

| Bits | Name              | Read/Write | Description                    |
|------|-------------------|------------|--------------------------------|
| 31-0 | VICTrVectAddr0-31 | Read/Write | Contains ISR vector addresses. |

**Table 7.3-23 VICTrVectAddr0-31 Registers****VICTrVectPrity0-31: Mirrored Programmable Priority 0-31 Registers [4] (+0x200-0x27C)**

The VICTrVectPrity0-31 registers selects the interrupt priority levels for the 32 vectored interrupts sources. The value can be from 0 to 15. The default values have all interrupts on the same priority level 15 which is the lowest. This allows any of the vectored interrupts to be promoted to a higher priority with one simple register write.

| Bits | Name               | Read/Write | Description                                 |
|------|--------------------|------------|---------------------------------------------|
| 3-0  | VICTrVectPrity0-31 | Read/Write | Selects vectored interrupt priority levels. |

**Table 7.3-24 VICTrVectPrity0-31 Registers**

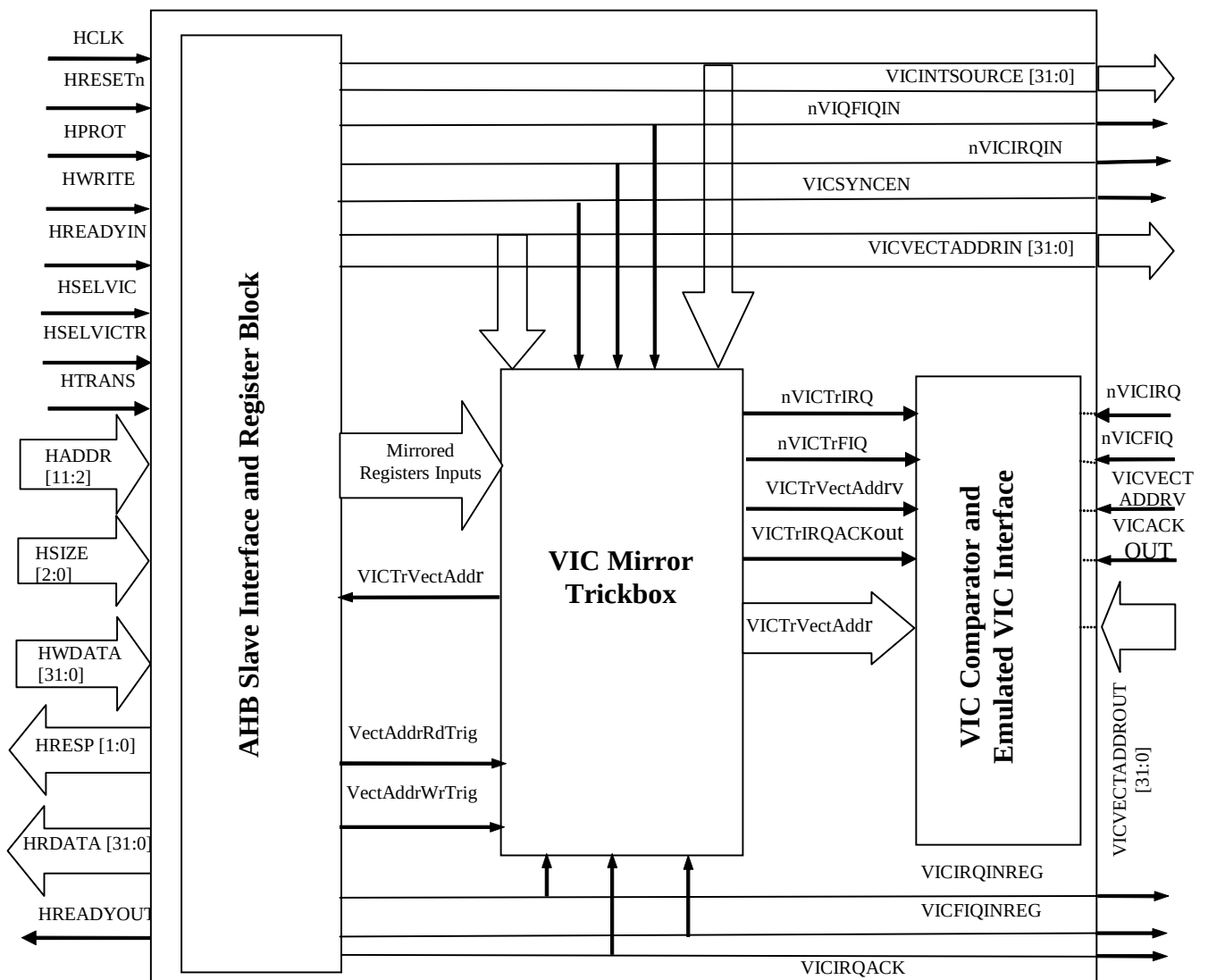
**VICTrVectAddr: Mirrored Vector Address Registers [32] (+0xF00)**

The VICTrVectAddr contains the resolved priority IRQ vector address. When this register is accessed using VIC base address zero is returned. When accessed with Trickbox base address it return the resolved priority IRQ vector address from the mirrored Trickbox.

| Bits | Name          | Read/Write | Description                                       |
|------|---------------|------------|---------------------------------------------------|
| 31-0 | VICTrVectAddr | Read/Write | Returns the resolved priority IRQ vector address. |

**Table 7.3-25 VICTrVectAddr Register****7.3.5 Trickbox Functional Description**

The VIC Trickbox block diagram is given in the **Figure 7.3-1**

**Figure 7.3-1 VIC Trickbox Block Diagram**



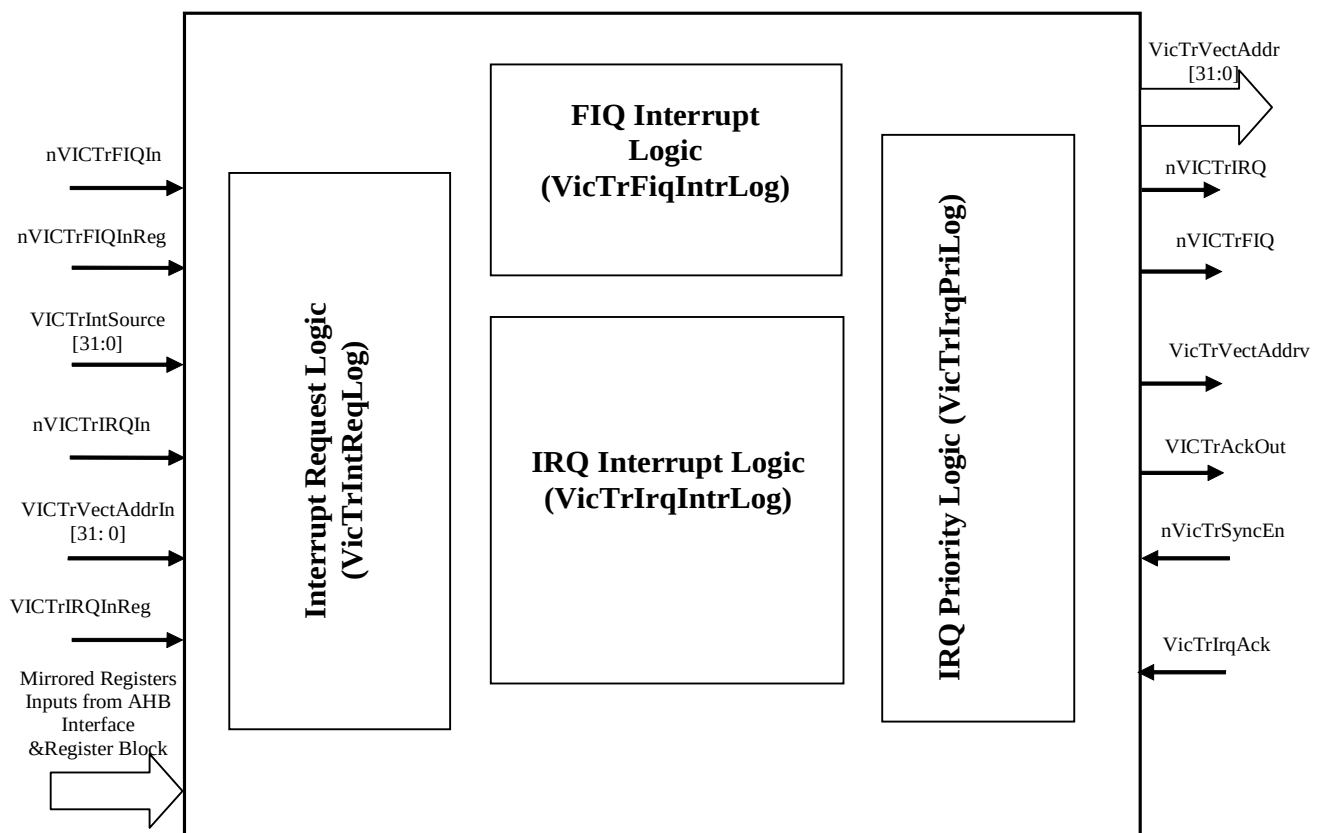
The basic block description is as follows:

### AHB Slave Interface and Register Block (VicTrAhbif)

The AHB slave and Register block implements all the registers of the Trickbox (both Trickbox mirrored registers and Trickbox specific registers). This block decodes the read and write access to internal registers. This block also stores the data written or to be read across the AHB interface. This block also generates the inputs to the VIC UUT and Mirrored Trickbox as shown in the **Figure 7.3-1**. This block also generates the VICIRQACK signal to the VIC UUT and VIC Mirror Trickbox.

### VIC Mirror Trickbox (VicMirTrickbox)

The VIC Mirror Trickbox replicates the mirrored functionality of the VIC. It is used for functional verification of VIC. The outputs of this VIC Mirror Trickbox are compared with the outputs of VIC. The mirrored registers implemented in the AHB Slave Interface and Register block forms inputs to this block shown in the below **Figure 7.3-2**.



**Figure 7.3-2 VIC Mirrored Trick Box Block Level Diagram**

The sub modules of the VIC Mirrored Trick Box are

### Interrupt Request Logic

The interrupt request logic receives the interrupt requests from the peripherals and combines them with the software interrupts requests. It then masks out the interrupt requests that are not enabled. The enabled requests are routed to FIQ interrupt logic or IRQ interrupt logic based on the VICTrIntSelect register. It is implemented as shown in the **Figure 7.3-3**.

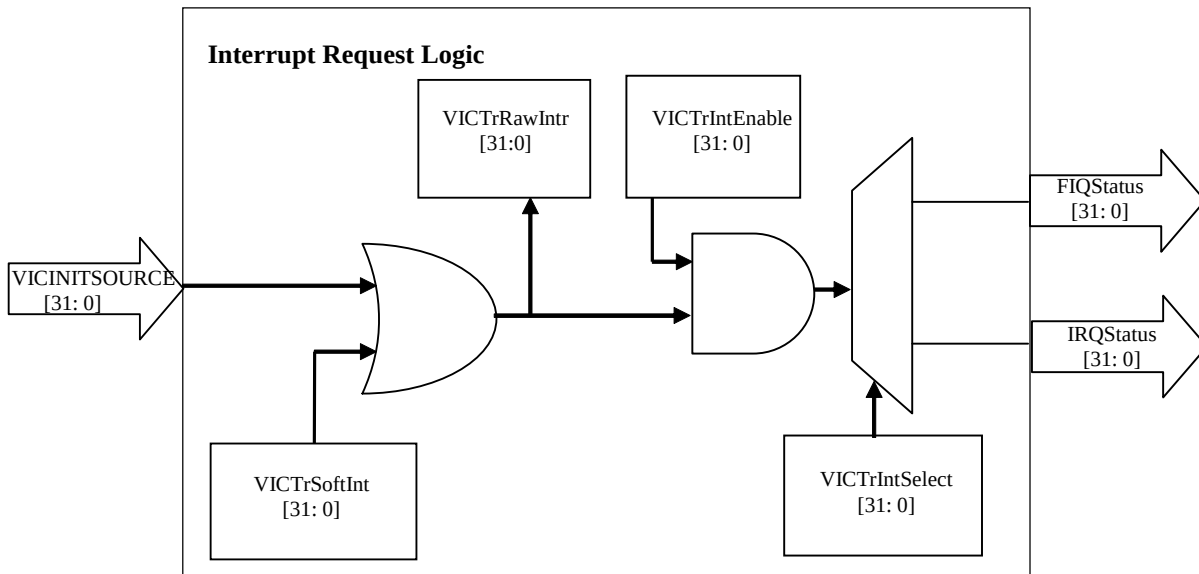


Figure 7.3-3 Interrupt Request Logic

### FIQ Interrupt Logic

The non-vectored FIQ interrupt logic generates the FIQ interrupt signal by combining the FIQ interrupt requests in the interrupt controller with the request from the daisy-chained interrupt controllers. The nVICFIQIN signal from the daisy-chained interrupt controller is clocked if VICFIQINREG signal is high. It is implemented as shown in the **Figure 7.3-4**.

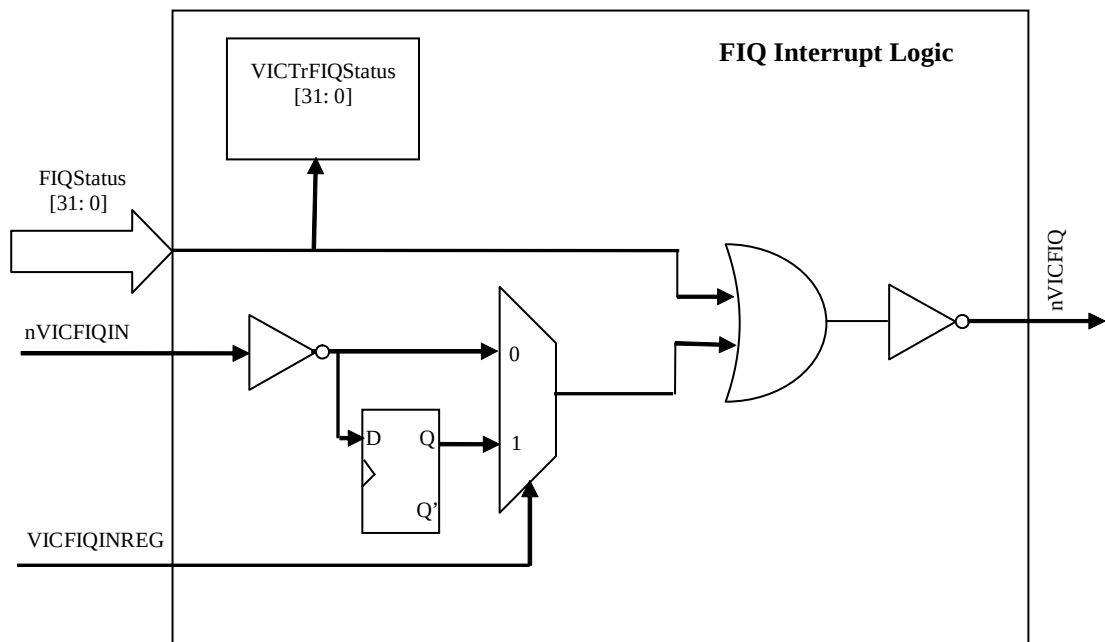
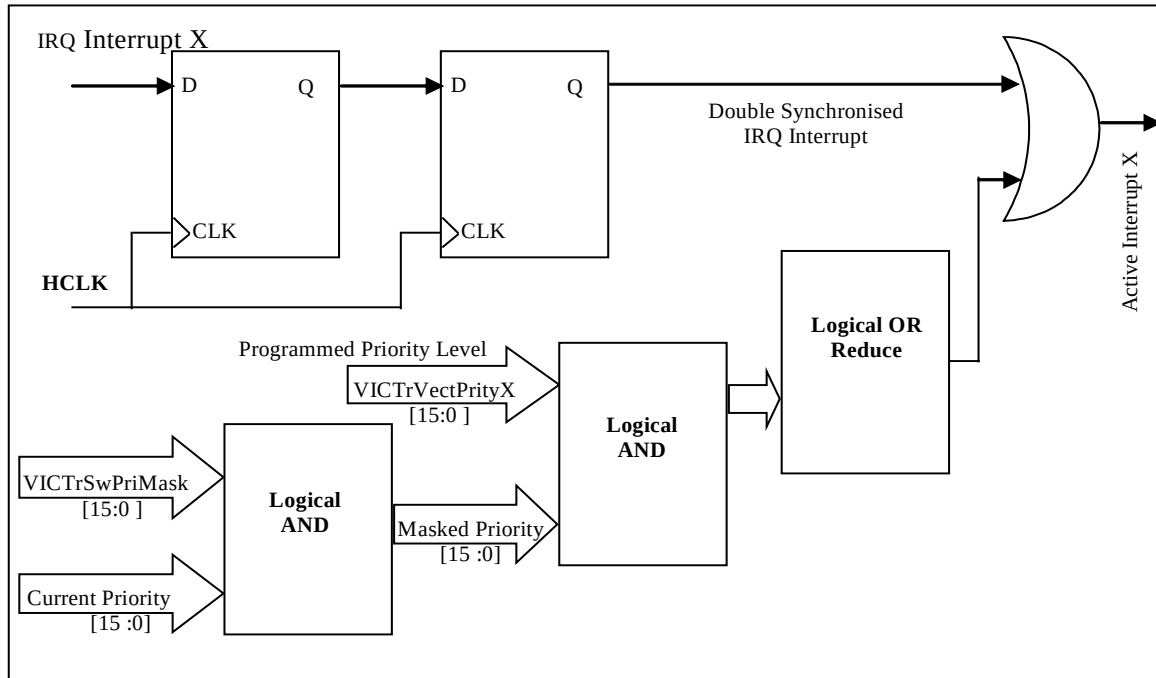


Figure 7.3-4 FIQ Interrupt Logic

### IRQ Interrupt Logic (VicTrIrqIntrLog)

The IRQ Interrupt logic generates the active interrupts by resolving the software priority. First, it masks the interrupts by software priority mask register. Then masks the active interrupts with the current priority level. The current priority level is an input to this module from IRQ Priority Logic (VicTrIrqPriLog). A single IRQ interrupt logic to determine the active interrupt after resolving the software priority as shown in the **Figure 7.3-5**.



**Figure 7.3-5 A single IRQ Interrupt Logic**

### IRQ Priority Logic (VicTrIrqPriLog)

This module resolves the hardware priority of interrupts and decodes the address of the highest priority level interrupt selected. The masking of lower and equal priority interrupts are different for VIC port and Non VIC port. For VIC port lower and equal priority interrupts are masked through a handshake mechanism. For Non VIC port, the lower and equal priority interrupts are masked when a read on VICVectAddr register is performed. When VICVectAddr register is written, equal and lower priority interrupts are unmasked. This is common for both VIC port and Non-VIC port. A stack structure is implemented which has 16 registers each of 16 bits wide. All the registers in stack structure are initialised to 0xFFFF at reset. The contents of register 0 on stack will have a value corresponding to the current priority level selected. The content of register 0 on the stack is used to resolve software priority upon a new IRQ interrupt. The contents of stack are pushed when a read on VICVectAddr register is performed (for NON-VIC port) or when the interrupt is serviced through a handshake mechanism (for VIC port). When the VICVectAddr register is written, the stack contents are popped. This module generates the nTrIrq signal, which is the inverted version of nVICTrIrq signal. It also generates the VicTrVectAddrv signal for the VIC port handshake mechanism

### VIC Comparator and Emulated VIC Interface

The Compare logic compares the locally generated outputs with the corresponding signals from the VIC. Bits [4:0] of the VICTrTCR register controls the duration for which the compare logic is active. When these bits are HIGH, the Trickbox compares the outputs at every rising edge of the HCLK. When there is a mismatch, it flags error messages.

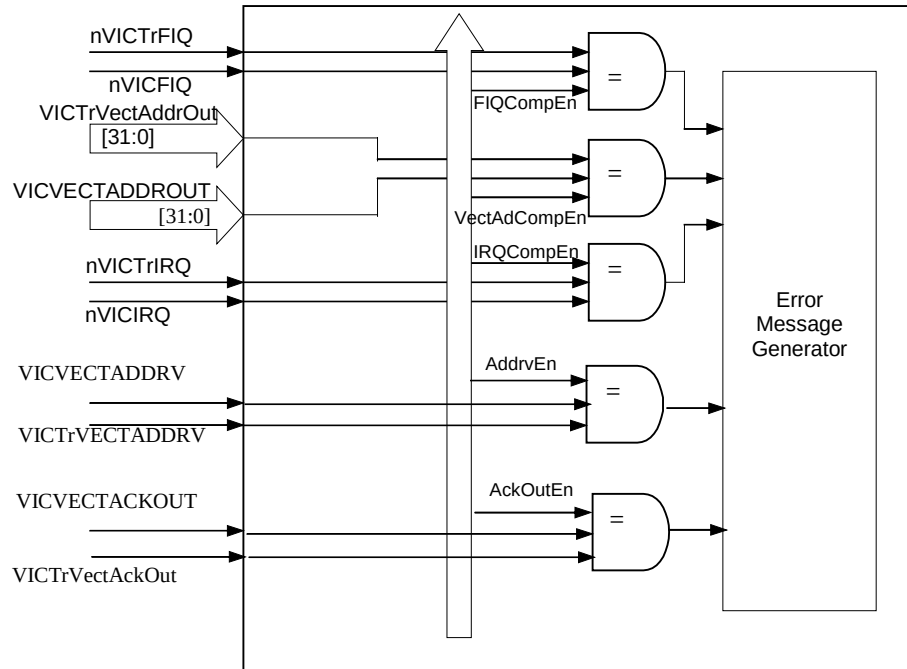


Figure 7.3-6 VIC Comparator and Emulated VIC Interface

## 8 FUNCTIONAL TESTS

The following section describes the test cases to validate the VIC.

It must be noted that the majority of the Interrupt tests rely heavily on the on-line comparisons done in the Trickbox. This method has the advantage that the compare accuracy can be done at a nano-second level if need be [in the actual Trickbox implementation, the compare granularity is kept to a HCLK period to eliminate false mismatches during netlist simulations]. The number of test vectors required to prove functionality is much lesser than a method, which predominantly uses reads of the VIC and Trickbox registers to verify functionality.

### 8.1 Reset Tests (ResetTests)

This test is invoked by the ResetTests() function call.

After a reset, each of the following registers are read to check if they contain the specified reset value.

| VIC Register Name        | Reset Value |
|--------------------------|-------------|
| VICIRQStatus             | 0x00000000  |
| VICFIQStatus             | 0x00000000  |
| VICRawIntr <sup>1</sup>  | 0x00000000  |
| VICIntSelect             | 0x00000000  |
| VICIntEnable             | 0x00000000  |
| VICSoftInt               | 0x00000000  |
| VICProtection            | 0x0         |
| VICSWPriorityMask        | 0xFFFF      |
| VICVectPriorityDaisy     | 0xF         |
| VICVectAddr <sup>2</sup> | 0x00000000  |
| VICITCR                  | 0x00000000  |
| VICITIP1 <sup>3</sup>    | 0x000000C0  |
| VICITIP2 <sup>4</sup>    | 0x00000000  |
| VICITOP1 <sup>5</sup>    | 0X00000000  |
| VICITOP2 <sup>6</sup>    | 0X00000000  |
| VICVectAddr0-31          | 0x00000000  |
| VICVectPriority0-31      | 0xF         |
| VICPeriphID0             | 0x00000092  |
| VICPeriphID1             | 0x00000011  |
| VICPeriphID2             | 0x00000004  |
| VICPeriphID3             | 0x00000000  |

| VIC Register Name | Reset Value |
|-------------------|-------------|
| VICPCellID0       | 0x0000000D  |
| VICPCellID1       | 0x000000F0  |
| VICPCellID2       | 0x00000005  |
| VICPCellID3       | 0x000000B1  |

**Table 8.1-1 Reset values of VIC registers**

1. Note that the VICRawIntr is the logical OR of the VICINTSOURCE and the VICSoftInt bits. The value reflected in the VICRawIntr register will depend on the values driven on the external VICIntSource pins since the VICSoftInt register is reset to 0x00000000. The reset value tests assume that the source of VICINTSOURCE (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICINTSOURCE drives these to a value other than 0x00000000, the test code will have to be modified accordingly.
2. Note that the VICVectAddr is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The value reflected in the VICVectAddr register will depend on the values driven on the external VICVECTADDRIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICVECTADDRIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.
3. Note that the VICITIP1 is derived from the internally generated Interrupts and external Daisy Chained nVICIRQIN and nVICFIQIN interrupts. The value reflected in the VICITIP1 register will depend on the values driven on the external VICIRQACK, nVICIRQIN and nVICFIQIN pins since the internally generated Interrupts will be reset to 0x00000000. The reset value tests assume that the source of VICIRQACK, nVICIRQIN and nVICFIQIN (the Trickbox in the functional verification world) drives these pins inactive after reset. If the source of the VICIRQACK, nVICIRQIN and nVICFIQIN drives these to their active values after reset, the test code will have to be modified accordingly.
4. Note that the VICITIP2 is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The value reflected in the VICITIP2 register will depend on the values driven on the external VICVECTADDRIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICVECTADDRIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.
5. Note that the VICITOP1 is derived from the internally generated Interrupts and external Daisy Chained nVICIRQIN and nVICFIQIN interrupts. The value reflected in the VICITIP1 register will depend on the values driven on the external VICIRQACK, nVICIRQIN and nVICFIQIN pins since the internally generated Interrupts will be reset to 0x00000000. The reset value tests assume that the source of VICIRQACK, nVICIRQIN and nVICFIQIN (the Trickbox in the functional verification world) drives these pins inactive after reset. If the source of the VICIRQACK, nVICIRQIN and nVICFIQIN drives these to their active values after reset, the test code will have to be modified accordingly.
6. Note that the VICITOP2 is derived from the internally generated Vector Address and the external Daisy Chained Vector Address (VICVECTADDRIN). The values reflected in the VICITOP2 register will depend on the values driven on the external VICVECTADDRIN pins since the internally generated Vector Address will be reset to 0x00000000. The reset value tests assume that the source of VICVECTADDRIN (the Trickbox in the functional verification world) drives these pins to 0x00000000 after reset. If the source of the VICVECTADDRIN drives these to a value other than 0x00000000, the test code will have to be modified accordingly.

Initially HRESETn is asserted and the registers specified in the **Table 8.1-1** are read back to verify that the reset values are returned.

This test is invoked by the RegisterTests () function call.

## 8.2 Register Tests (RegisterTests)

### Read-Write Registers

The data patterns 0x55555555, 0x00000000, 0xFFFFFFFF and 0xAAAAAAAA are written to each of the Read-Write registers. After each write, the register contents are read back to see if it contains the written value for the registers VICIntSelect, VICVectAddr0-31, VICVectPriority0-31 and VICITCR. If the returned values are the same as the expected values, then the register is declared writeable and readable and the test passes.

For testing the VICProtection register, a write and a read is performed to it. If the HPROT[1] is low (indicating a user access) during the write-read operation, then this value is temporarily stored and the HPROT[1] is made high for the write-read access. If the pattern read is the same as that was written then the register is declared writeable and readable and the test passes.

To test the VICITIP registers, the VIC is configured in integration test mode by setting the ITEN bit in the VICITCR. A data pattern is written into the VICITIP1 register and read back from the 6 and 7 bits. If the returned values are the same as the expected value, then the register is declared writeable and readable and the test passes. Similarly, the patterns written in the VICITIP2 should be returned while reading back. The VICITIP1 and VICITIP2 registers are cleared before proceeding further. Now, the VIC is configured back into normal mode by clearing the ITEN bit in the VICITCR. At this moment, the pattern coming in through the VICVECTADDRIN is reflected in the VICITIP2. Similarly, the values on the nVICIRQIN and nVICFIQIN inputs will be reflected in the 6 and 7 bits of the VICITIP1 register. The contents of the VICVECTADDRIN are made zero by writing to the VICTrVectAddrIn register of the Trickbox. The nVICIRQIN and nVICFIQIN are set (to indicate no external interrupts) by writing into the VICTrIntIn register. Now, different data patterns are written into the VICITIP1 and VICITIP2 registers. But, the content of these registers should not change. Now, the contents of VICTrVectAddrIn and VICTrIntIn are changed. This change should be reflected in the VICITIP1 and VICITIP2 registers.

To test the VICIntEnable and VICSoftInt registers, a different method is adopted as their contents can be altered by writing into their respective clear registers as well. As the reset value of the VICIntEnable and the VICSoftInt is ZERO, initially writing a pattern of 0x55555555 will return the same pattern on reads. Now a pattern of 0x99999999 is written and the pattern 0xDDDDDDDD is expected while reading. Then, a pattern of 0x22222222 is written and a pattern of 0xFFFFFFFF is expected while reading.

Now, the Write-only registers, VICIntEnClear and VICSoftIntClear can be used to clear the bits in the enable registers. Data pattern 0x33333333 is written into the respective clear registers and the pattern 0xCCCCCCCC is expected from the enable registers. Data pattern 0x55555555 is written into the clear registers and the pattern 0x88888888 is expected from the enable registers. Data pattern 0x88888888 is written into the clear registers and the pattern 0x00000000 is expected from the enable registers.

### Read-only Registers

For read-only registers the data patterns 0x55555555, 0x00000000, 0xFFFFFFFF, 0xAAAAAAAA are written and after each write, read back to see if the default values (reset values) for the registers VICIRQStatus, VICFIQStatus, VICRawIntr, VICPeriphID0, VICPeriphID1, VICPeriphID2, VICPeriphID3, VICPCellID0, VICPCellID1, VICPCellID2 and VICPCellID3 are returned. If the default values are returned on reads, these registers are read-only and the test passes.

### 8.3 Transfer Test (TransferTests)

This test is invoked by the TransferTests() function call. This test is used to verify burst access to VICVectPriority0-31 and VICVectAddr0-31 registers with various AHB transfer types. The functions described below are used to achieve the same.

#### Burst Transfer

This is invoked by the BurstTransfer() function call.

A sequence of writes is performed to the VICVectAddr0-31 and VICVectPriority0-31 registers in a burst. Then the registers are read back in a burst with the expected data same as the written data. If all the registers return the expected data, the test passes.

In addition to this, burst transfers are also done in between the Error Response test, Interrupt test and the Protection Mode test. In the Error Response test, the VICVectPriority0-31 registers are accessed in a burst with HSIZE indicating a half-word transfer type. In the Interrupt test, the status registers and the VICRawIntr are read in a burst. Similarly, the initialisation of the vector and control registers are done using a burst access. In the Protection Mode test, all the burst transfers performed in this function are repeated with the Protection bit set.

#### BUSY Transfer

This is invoked by the BusyTransfer() function call.

A burst write is performed to the VICVectAddr0-31 and VICVectPriority0-31 registers with BUSY cycles added in between. Then, a burst read is performed on these registers with BUSY cycles inserted. The points at which the BUSY cycles are inserted during the burst is varied. If the expected data is the same as the data read, then the test passes.

In addition to this, BUSY transfers are also performed in between the Interrupt test and the Protection Mode test. In the Interrupt test, BUSY cycles are added between the burst write accesses to the VICVectAddr0-31 registers. In the Protection Mode test, all the BUSY transfers performed in this function are repeated with the Protection bit set.

#### IDLE Transfer

This is invoked by the IdleTransfer() function call.

A burst write is performed on the VICVectAddr0-31 and VICVectPriority0-31 registers with IDLE cycles added in between. Then, a burst read is performed on these registers with IDLE cycles added at some other points. If the expected data is the same as the data read, then the test passes.

In addition to this, IDLE transfers are also performed in between the Interrupt test and the Protection Mode test. In the Interrupt test, IDLE cycles are added between the burst write accesses to the VICVectAddr0-31 registers. In the Protection Mode test, all the BUSY transfers performed in this function are repeated with the Protection bit set.

### 8.4 Error Response Test (ErrorRespTests)

This test is invoked by the ErrorRespTests() function call.

A write access is performed to all the registers with the HSIZE as Byte or Half-Word. A burst access is performed to the VICVectPriority0-31 registers with HSIZE as Half-Word. If, in all cases an Error response is returned, the test passes.

### 8.5 Protection Mode Test (ProtectionTests)

This test is invoked by the ProtectionTests() function call.



The functionality being tested is that when the VIC is in 'User' mode, both 'Supervisor' and 'User' mode accesses should be possible and when the VIC is in 'Protected' mode, only 'Supervisor' accesses should be possible.

The VIC is configured in 'Protected' mode by setting the Protection bit high. Now, a write-read test is performed in the 'Supervisor' mode. Data pattern 0x55555555 is written into the VICIntSelect, VICVectPriority0-31, VICVectAddr0-31, VICITCR, VICVectAddr, and VICProtection registers. The data read back should be exactly the same as that was written. The read-only registers VICIRQStatus, VICFIQStatus, VICRawIntr, VICITOP1, VICITOP2, VICPeriphID0-3 and VICPCellID0-3 are also written with 0x55555555, but the expected value while reading is their reset value.

While testing the VICITIP registers, the ITEN bit of the VICITCR is enabled and then disabled.

After the above test, the VIC is accessed in 'User' mode. A write-read access is performed to all the above registers. The data pattern written is 0xAAAAAAAA. This time the expected value while reading is ZERO. Now, it has to be proved that the above access in the 'User' mode didn't change the content of any of the registers. Hence, again a 'Supervisor' mode read access is done on all the registers. The read-only registers should return their reset value and the read-write registers should return the previously written pattern, 0x55555555.

A 'User' mode burst access is done to all the VICVectAddr0-31 and VICVectPriority0-31 registers. The pattern read back should be ZERO. BUSY cycles are added in between the burst access and a ZERO value is expected while reading. Similarly, IDLE cycles are added in between the burst access and a ZERO value is expected while reading.

Half-word or Byte write accesses are done on the VICVectAddr, VICProtection and some of the control and address registers. The expected response is an ERROR response.

## 8.6 Protection Mode Interrupt Test (ProtModelIntTests)

This test is invoked by the ProtModelIntTests() function call.

This test verifies that 'User' mode reads or writes to the VICVectAddr register do not affect the Hardware Priority. The test flow is as follows:

- Interrupt Source 1 - VICINTSOURCE [1] - is configured to raised an IRQ.
- Generate the interrupt via the Trickbox. The VICINTSOURCE is 0x00000002 (i.e., 0b0000\_0000\_0000\_0000\_0000\_0000\_0010). Interrupt source 1 is active.
- The Protection mode bit is set in the VICProtection register.
- A User mode read access is performed to the VICVectAddr register in the VIC. If the access were a protected mode access or if the Protection mode bit were cleared, this read should have resulted in the masking of the currently active nVICIRQ output. However, since the current access is in User mode, bit 1 of the CurrentService register should NOT get set. The nVICIRQ output should continue to remain LOW. The Trickbox compare logic verifies that the nVICIRQ line is indeed low. It also verifies that the Vector Address corresponding to (0x11111111) continues to be present on the VICVECTADDRROUT lines. This proves that the User mode read has had no effect on the internal priority mask.
- Now a Protected mode read access is performed to the VICVectAddr register in the VIC. This should cause bit 1 of the CurrentService register to be set, causing the nVICIRQ line to go inactive. The VICVECTADDRROUT should now reflect the Default Vector Address since no interrupts are active. The trickbox compare logic verifies that the interrupt line status and the vector address values are as expected.
- All interrupt sources are cleared (VICINTSOURCE = 0x00000000)
- Interrupt source 2 is raised (VICINTSOURCE = 0x00000004)
- The Trickbox compare logic continues to verify that the nVICIRQ is HIGH and the vector address is the default vector address. Since bit 1 of the CurrentService register is still set, IRQ interrupt is not raised.
- A User mode write access is performed to the VICVectAddr register. Bit 1 in the CurrentService register should continue to remain set. Consequently, the priority mask should also remain the same. The Trickbox compare logic verifies that the nVICIRQ line remains inactive and the default vector continues to get driven on the VECTADDRROUT lines.

- A Protected mode write access is performed to the VICVectAddr register. Bit 1 in the CurrentService register should get cleared. Since the CurrentService register does not have any set bits in it, all interrupts will be unmasked. The pending interrupt source 2 will be propagated on to the nVICIRQ line along with its corresponding Vector Address (0x22222222). The Trickbox compare logic verifies that the values are as expected.
- The Protection bit is reset using a protected mode access.
- The VICVectAddr register is read, masking the active Interrupt source 2.
- VICINTSOURCE is written with 0x00000000.
- The VICVectAddr register is written, clearing the set bit in the CurrentService register.

The Trickbox compare logic continuously monitors the VICVECTADDROUT, nVICIRQ and nVICFIQ output lines, flagging error messages if there are any mismatches.

## 8.7 Basic Interrupt Tests

The Interrupts tests are divided into two categories:

- Non-VIC Interrupts Tests.
- VIC Port Interrupts Tests (tests preceded with VicEn).

The only difference between VIC port and Non-VIC port test cases is the way in which the interrupts are serviced. In VIC port test cases VICIRQACK signal generation is enabled by programming VICTrAckCnt register.

The Non-VIC Interrupt tests are invoked by the UserOneIntTests(), TwoFourIntTests() and EightAllIntTests() function calls. The VIC port Interrupt tests are invoked by the VicEnUserOneIntTests(), VicEnTwoFourIntTests() and VicEightAllIntTests() function calls.

These tests create various combinations of interrupting conditions. Interrupt functionality is verified by a combination of on-line checking by the Trickbox and by selective reads of Status registers.

The overall flow of the Interrupt test suite as well as functionality of key support functions is described:

- Program the VICIntEnable, VICSoftInt, VICIntSelect, VICTrIntIn, VICProtection, VICTrVectAddrIn, VICVectAddr0-31, VICVectPriority0-31, VICVectPriorityDaisy, VICSWPriorityMask, registers. This is done by the functions InitValue() and Initialise(). Tests are done with 32 different combinations of these register settings.
- A variable D\_TrIntSource is assigned a value equivalent to the number of interrupts to be applied. For e.g., if it is a one interrupt test, then the binary format of the value assigned to the D\_TrIntSource will have exactly one of the bits as 1. The function ApplyIntr() is used to write this value into the register VICTrIntSource of the Trickbox (or the VICSoftInt of the VIC). When the argument of the function ApplyIntr() is 0, VICTrIntSource is used and when it is 1, VICSoftInt is used.
- The function AllIntService() is used to service and clear all the interrupts in the order of their priorities.
- The function ReadStatus() reads the Status registers viz., VICIRQStatus, VICFIQStatus and VICRawIntr expecting the desired value. A variable called CSR is written for keeping track of applied interrupts. Then, according to the value in the register VICFIQStatus, the function FIQServ() services and clears all the interrupt sources requesting the FIQ interrupt, beginning from the 0<sup>th</sup> position. Servicing, in this context, means reading all the status registers. Then IRQs are serviced according to their s/w priority in the function IRQServe(). Here, servicing is described as the time between a read from the VICVectAddr to a write to the VICVectAddr. In between this period, all the status registers are read.

Tests are performed applying one, two, four, eight, sixteen and thirty-two interrupts with or without Daisy Chain interrupt, at a time via the Trickbox or the VICSoftInt register. All these tests employ the procedure explained above. Apart from these some specific interrupt tests are performed and they are described in **Section 8.7.7 Special Interrupt Tests** which is a part of **UserOneIntTests0-7** for Non-VIC port and **VicEnUserOneIntTests0-7** for VIC port.

There are two types of interrupts: Non-Vectored FIQ (maximum 32 in number), Vectored IRQ (maximum 32 in number). Note that the external interrupt is not considered here. So, 2 different types of interrupt requests are possible in the One-Interrupt case. Both the types have been covered. For the Two-Interrupt case, 4 different types of interrupt requests are possible, all of which are covered. For the Four-Interrupt case, all of the 15 different types of interrupt requests are covered. For the Eight-Interrupt case, 27 of the possible 45 types of interrupt requests are covered. For the Sixteen-Interrupt case, 24 out of possible 153 types are covered. For the Thirty-two interrupts case, 12 out of the possible 561 types are covered.

### 8.7.1 One Interrupt Test (UserOneIntTests0-7 or VicEnUserIntTests0-7)

The user one-interrupt tests are split into eight test cases UserOneIntTests0 to UserOneIntTests8 for Non-VIC port and VicEnUserOneIntTests0 to VicEnUserOneIntTests7 for VIC port. These tests can be divided into 2 categories:

- Raising of a FIQ.
- Raising of an IRQ.

In all these categories, the VIC is tested with an active external interrupt as well. In some cases, certain interrupts are disabled by resetting the corresponding bit in the VICIntEnable register. In all, 1024 tests are performed for the One-Interrupt case. Each of the 2 categories is illustrated with one example each:

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0xAAAAAAAA | Decides which of the sources will raise a FIQ                    |
| VICTrIntIn        | 0x00000003 | Indicates no external interrupts                                 |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x1        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               | ...        | ...                                                              |
| VICVectPriority14 | 0x14       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x1        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               | ...        | ...                                                              |

| Register          | Value  | Description                                           |
|-------------------|--------|-------------------------------------------------------|
| VICVectPriority30 | 0x14   | Software priority of 30th interrupt                   |
| VICVectPriority31 | 0x15   | Software priority of 31st interrupt                   |
| VICSWPriorityMask | 0xAAAA | All odd numbered software priority interrupts masked. |

**Table 8.7-1 Initialisation values for an example of One Interrupt Test***Category 1 - FIQ*

In this illustration, the value 0x00000008 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that source 3 is raising an interrupt. Since, the bit 3 of the VICIntSelect is 1, it will be a FIQ interrupt.

To test the VIC the following have to be performed:

- Check the Status registers for the expected values. This marks the ISR entry.
- Check that the nVICFIQ pin is low.
- Clear the bit raising the interrupt in the VICTrIntSource.
- Again check the Status registers.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Since, the status registers are clocked out by the VIC, due to the AHB pipelining and the double synchronisation of the status registers, the change in these registers are visible on the read data bus only after two clock cycles. Hence, in the function AllIntService() and the ReadStatus(), IDLE cycles are added to compensate for this two clocks discrepancy.

Now, the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0x00000000. The VICFIQStatus is read expecting a pattern of 0x00000008. Pattern 0x00000008 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be 0x0000.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function FIQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0<sup>th</sup> position, all the FIQ interrupt sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000000, 0x00000008 and 0x00000008 respectively. The bits of the VICFIQStatus are read from the 0<sup>th</sup> position. As soon as, there is a 1 (here, it is the 3<sup>rd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 3<sup>rd</sup> bit of the VICTrIntSource is cleared. The 0<sup>th</sup> bit of the VICTrIntIn is checked to find any pending external FIQ.

Now, the VICFIQStatus register is checked for any other active FIQs. In this illustration, no FIQs should be pending.

The VICIRQStatus, is checked for any IRQ interrupts.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers respectively.

### Category 2 -IRQ

In this illustration the value 0x00001000 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that source 12 is raising an interrupt. Since bit 12 of the VICIntSelect is 0, it will be an IRQ interrupt. VICVectAddr12 i.e., 0x0000A030.

To test the VIC the following have to be performed:

- Check the Status registers for expected values.
- Check that nVICIRQ pin has gone low.
- Read the VICVectAddr register expecting the required vector address. This marks the beginning of the interrupt service routine.
- Clear the bit raising the interrupt in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine.
- Again check the Status registers.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. The software priorities and the masks are taken into consideration and the active highest priority interrupt is determined in this function. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0x00001000. The VICFIQStatus is read expecting a pattern of 0x00000000. Pattern 0x00001000 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. Here, the interrupt is raised via the 12<sup>th</sup> interrupt source. Hence, the 12<sup>th</sup> bit of the CSR should be set. Thus, the value stored in the CSR should be 0x00001000.

The VICFIQStatus, is checked for any FIQ interrupts.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Since, the CSR is non-zero (indicating the presence of an active IRQ), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as any non-zero bit is seen, the interrupt service routine is started. The routine consists of reading the VICITOP registers, address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed with clearing the serviced interrupt bit.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00001000, 0x00000000 and 0x00001000 respectively. The bits of the CSR are read as per priority order as described above. As soon as there is a 1 (here, it is the 4<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A030. The VICVectAddr is read expecting the address value as 0x0000A030. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, the bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

If in all the above cases, the VIC performs as desired, the test passes.

In addition to the above basic test sequences, the following variants are also tested for the 3 categories:

- External interrupt is raised
- Tests are performed with different software priorities and software priority mask values.
- Tests are performed with the interrupt disabled by clearing various combinations of bits in the VICIntEnable register.

### 8.7.2 Two Interrupt Test (TwoFourIntTests or VicEnTwoFourIntTests)

This test is invoked by the TwoFourIntTests() function call for Non-VIC port and VicEnTwoFourIntTests() function call for VIC port. These tests can be divided into 3 categories:

- Raising of two FIQs.
- Raising of two IRQs.
- Raising of one FIQ and one IRQ.

In all these categories, the VIC is tested with the external interrupt also. In some cases certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 1024 tests are performed for the Two-Interrupt case, 512 with VICSoftInt and 512 with TrIntSource. One category is illustrated with an example. In this example, the external interrupt is also active.

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0x12345678 | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn        | 0x00000000 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x33333333 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               | ...        | ...                                                              |
| VICVectPriority14 | 0x15       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x0        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               | ...        | ...                                                              |

| Register          | Value  | Description                                            |
|-------------------|--------|--------------------------------------------------------|
| VICVectPriority30 | 0x15   | Software priority of 30th interrupt                    |
| VICVectPriority31 | 0x15   | Software priority of 31st interrupt                    |
| VICSWPriorityMask | 0x1111 | All multiple of 4 software priority interrupts masked. |

**Table 8.7-2 Initialisation values for an example of Two Interrupt Tests***Two IRQs*

In this illustration, the value 0xC0000000 is written into the VICTrIntSource register. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 30 and 31 are raising interrupts. Since, the bits 30 and 31 of the VICIntSelect are 0, they will raise an IRQ interrupt. The priority of the interrupt source 30 is greater than that from source 31.

To test the VIC the following have to be performed:

- Check the Status registers for expected values.
- Check that the nVICIRQ and nVICFIQ pins had gone low.
- Clear the bit raising the external FIQ. This is the 0<sup>th</sup> bit of VICTrIntIn.
- Again check the Status registers for expected values.
- Check that the nVICIRQ pin had gone low.
- Read the VICVectAddr register expecting the content of the VICVectAddr30. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the first interrupt (bit 30) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers for expected values.
- Check the nVICIRQ pin for a low.
- Read the VICVectAddr expecting the content of the VICVectAddr31. This marks the beginning of interrupt service routine of the second interrupt.
- Clear the bit raising the second interrupt (bit 31) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers. They should contain zeroes.
- Check the nVICIRQ for a low.
- Read the VICVectAddr expecting the content of the VICTrVectAddrIn.
- Clear the bit raising the external IRQ. This is the 1<sup>st</sup> bit of VICTrIntIn.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0xC0000000. The VICFIQStatus is read expecting a pattern of 0x00000000. The pattern 0xC0000000 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. Here, the interrupts are raised via the 1<sup>st</sup> and 2<sup>nd</sup> vector interrupt. Hence, the 1<sup>st</sup> and 2<sup>nd</sup> bits of the CSR should be set. Thus, the value stored in the CSR should be 0x0006.

The `VICFIQStatus` register is checked for any FIQ interrupts. The 0<sup>th</sup> bit of the `VICTrIntIn` is checked for any external FIQ. Now, the `FIQServ()` function is called. In this function, the Status registers are read. The expected values are 0xC0000000, 0x00000000 and 0xC0000000 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` registers. The 0<sup>th</sup> bit in the `VICTrIntIn` is set to indicate that the external FIQ is cleared.

Now, the `VICIRQStatus` is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. The function `IRQServ()` is accessed with the argument zero (indicating that it is from the `VICTrIntSource`). This function clears all the IRQ interrupts in the order of their priorities. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of reading the `VICITOP` registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are 0xC0000000, 0x00000000 and 0xC0000000 respectively. The CSR is read bit by bit as per the order of priority. As soon as there is a 1 (here, it is the 30<sup>th</sup> bit), the `VICITOP1` is read with the expected value as 0x00000060. The `VICITOP2` is read expecting the vector address, 0x0000A078. The `VICVectAddr` is read expecting the address value as 0x0000A078. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 30 of the `VICTrIntSource` is made 0. Now, a write access is done to the `VICVectAddr` register to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the status registers are read again. Expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are 0x80000000, 0x00000000 and 0x80000000 respectively. The next bits of the CSR are read. As soon as there is a 1 (here, it is the 31<sup>st</sup> bit), the `VICITOP1` is read with the expected value as 0x00000060. The `VICITOP2` is read expecting the vector address, 0x0000A07C. The `VICVectAddr` is read expecting the address value as 0x0000A07C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, the bit 31 of the `VICTrIntSource` is made 0. Now, a write access is done to the `VICVectAddr` register to mark the end of the interrupt service routine. The content of the CSR is cleared.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` registers, respectively.

The other categories of the two interrupts case are tested in a similar fashion.

### 8.7.3 Four Interrupt Test (TwoFourIntTests or VicEnTwoFourIntTests)

This test is invoked by the `TwoFourIntTests()` function call for Non-VIC port and `VicEnTwoFourIntTests()` function call for VIC port. This test can be divided into 5 categories:

- Raising of four FIQs.
- Raising of four IRQs.
- Raising of three FIQs and one IRQ.
- Raising of two FIQs and two IRQs.
- Raising of one FIQ and three IRQs.

In all these categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the `VICIntEnable` register. In all, 384 tests are performed for the Four-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.



| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0x55555555 | Decides which of the sources will raise a FIQ                    |
| VICTrIntIn        | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               |            | ...                                                              |
| VICVectPriority14 | 0x7        | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x7        | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x8        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x8        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               |            | ...                                                              |
| VICVectPriority30 | 0x15       | Software priority of 30th interrupt                              |
| VICVectPriority31 | 0x15       | Software priority of 31st interrupt                              |
| VICSWPriorityMask | 0x8888     | Software priority mask                                           |

**Table 8.7-3 Initialisation values for an example of Four Interrupt Tests***Two FIQ, two IRQs*

In this illustration, the value 0x0000F00 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 8, 9, 10 and 11 are raising interrupts. Since the bits 8 and 10 of the VICIntSelect are 0, they will be raising IRQ interrupts and bits 9 and 11 are one so sources 9 and 11 will be raising FIQ interrupts.

To test the VIC, the following have to be performed:

- Check the Status registers for expected values.
- Check the nVICIRQ and nVICFIQ pins, whether they had gone low.
- Clear the bit raising the external FIQ. This is the 0<sup>th</sup> bit of VICTrIntIn.
- Again check the Status registers, whether they contain the required values.

- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 0<sup>th</sup> bit of VICTrIntIn.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the interrupt 8.
- Clear the bit raising the higher priority IRQ (bit 8) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the interrupt 8.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr10. This marks the beginning of the interrupt service routine of the interrupt 10.
- Clear the bit raising the IRQ (bit 10) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of interrupt 10.
- Again check the Status registers, whether they are cleared.

To perform the above tasks, the function AllIntService() is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. The VICIRQStatus is read expecting a pattern of 0x00000500. The VICFIQStatus is read expecting a pattern of 0x00000A00. The pattern 0x00000F00 is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vector interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be 0x00000500.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function FIQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0<sup>th</sup> position, all the FIQ interrupt sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000500, 0x00000A00 and 0x00000F00 respectively. The bits of the VICFIQStatus are read from the 0<sup>th</sup> position. As soon as there is a 1 (here, it is the 9<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9<sup>th</sup> bit of the VICTrIntSource is cleared. Again the reading of the VICFIQStatus register is continued. Now, at bit 11 again a 1 is present. The VICITOP1 is read with the expected value as 0x00000040. Now, the 11<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x00000500, 0x00000000 and 0x00000500 respectively. The CSR is read bit by bit as per the order of priority. As soon as there is a 1 (here, it is the 8<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A020. The VICVectAddr is read expecting the address value as 0x0000A020. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 8 of the VICTrIntSource is made 0. Now, a write access is done to the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x00000400.

Same procedure is repeated for interrupt 10. The vector address is expected to be 0x0000A028. The second IRQ interrupt is also serviced similar to first IRQ interrupt.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the four interrupts case are tested in a similar fashion.

### 8.7.4 Eight Interrupt Test (EightAllIntTests or VicEnEightAllIntTests)

This test is invoked by the EightAllIntTests() function call for Non-VIC port and VicEnEightAllIntTests() function call for VIC port. This test can be divided into 9 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 256 tests are performed for the Eight-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0x0000FFFF | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn        | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               | ...        | ...                                                              |
| VICVectPriority14 | 0x15       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |

| Register          | Value  | Description                                            |
|-------------------|--------|--------------------------------------------------------|
| VICVectPriority17 | 0x0    | Software priority of 17 <sup>th</sup> interrupt        |
| ...               | ...    | ...                                                    |
| VICVectPriority30 | 0x15   | Software priority of 30th interrupt                    |
| VICVectPriority31 | 0x15   | Software priority of 31st interrupt                    |
| VICSWPriorityMask | 0x1111 | All multiple of 4 software priority interrupts masked. |

**Table 8.7-4 Initialisation values for an example of Eight Interrupt Tests***Eight IRQs*

In this illustration, the value 0xFF000000 is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 24, 25, 26, 27, 28, 29, 30 and 31 are raising interrupts. Since, all of these bits of the VICIntSelect are 0, they will be raising an IRQ interrupt.

To test the VIC the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr24. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the Vectored IRQ (bit 24) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr25. This marks the beginning of the interrupt service routine of the second interrupt.
- Clear the bit raising the Vectored IRQ (bit 25) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr26. This marks the beginning of the interrupt service routine of the third interrupt.
- Clear the bit raising the Vectored IRQ (bit 26) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr27. This marks the beginning of the interrupt service routine of the fourth interrupt.
- Clear the bit raising the Vectored IRQ (bit 27) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
- Again check the Status registers, whether they contain the required values.

- Read the VICVectAddr expecting the content of the VICVectAddr28. This marks the beginning of the interrupt service routine of the fifth interrupt.
- Clear the bit raising the Vectored IRQ (bit 28) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr29. This marks the beginning of the interrupt service routine of the sixth interrupt.
- Clear the bit raising the Vectored IRQ (bit 29) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr30. This marks the beginning of the interrupt service routine of the first Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 30) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr31. This marks the beginning of the interrupt service routine of the second Non-Vectored interrupt.
- Clear the bit raising the Non-Vectored IRQ (bit 31) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second Non-Vectored interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the `VICTrIntSource` register of the Trickbox. When the interrupts are raised via the `VICSoftInt` register, the argument is 1.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of `0xFF000000`. The `VICFIQStatus` is read expecting a pattern of `0x00000000`. The pattern `0xFF000000` is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be `0xFF00`.

The `VICFIQStatus`, is checked for any FIQ interrupts.

Now, the `VICIRQStatus` is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. The function `VectIRQServ()` is accessed with the argument zero (indicating that it is from the `VICTrIntSource`). This function clears all the IRQ interrupts in the order of their priorities. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as any non-zero bit occurs the interrupt service routine is started. The routine consists of reading the `VICITOP` registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` are `0xFF000000`, `0x00000000` and `0xFF000000` respectively. The CSR is read bit by bit as per the

order of software priority. As soon as there is a 1 (here, it is the 24<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A060 (the content of the VICVectAddr24). The VICVectAddr is read expecting the address value as 0x0000A060. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this, bit 24 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0xF800.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 25<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A064 (the content of the VICVectAddr25). The VICVectAddr is read expecting the address value as 0x0000A064. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 25 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 26<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A068 (the content of the VICVectAddr26). The VICVectAddr is read expecting the address value as 0x0000A068. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 26 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 27<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A06C (the content of the VICVectAddr27). The VICVectAddr is read expecting the address value as 0x0000A06C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 27 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 28<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A070 (the content of the VICVectAddr28). The VICVectAddr is read expecting the address value as 0x0000A070. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 28 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 29<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A074 (the content of the VICVectAddr29). The VICVectAddr is read expecting the address value as 0x0000A074. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 29 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 30<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A078 (the content of the VICVectAddr30). The VICVectAddr is read expecting the address value as 0x0000A078. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 30 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 31<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A07C (the content of the VICVectAddr31). The VICVectAddr is read expecting the address value as 0x0000A07C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 31 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the eight interrupts case are tested in a similar fashion.

### 8.7.5 Sixteen Interrupt Test (EightAllIntTests or VicEnEightAllIntTests)

This test is invoked by the EightAllIntTests() function call for Non-VIC port and VicEnEightAllIntTests() function call for VIC port. This test can be divided into 17 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 64 tests are performed for the Sixteen-Interrupt case. One category is illustrated with an example.

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0x0F0F0F0F | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn        | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               |            | ...                                                              |
| VICVectPriority14 | 0x15       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x0        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               | ...        | ...                                                              |
| VICVectPriority30 | 0x15       | Software priority of 30th interrupt                              |
| VICVectPriority31 | 0x15       | Software priority of 31st interrupt                              |
| VICSWPriorityMask | 0x0000     | no software priority of interrupts masked.                       |

**Table 8.7-5 Initialisation values for an example of Sixteen Interrupt Tests**

*Eight FIQs, eight IRQs*

In this illustration, the value 0x0000FFFF is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 0 to 15 are raising interrupts. Since, the bits 0, 1, 2, 3, 8, 9, 10 and 11 of the VICIntSelect are 1, these sources will be raising FIQs. Rest of the interrupts will be raising IRQ interrupts.

To test the VIC the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 0<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 1<sup>st</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 2<sup>nd</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 3<sup>rd</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 8<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 9<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 10<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 11<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr4. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the bit raising the Vectored IRQ (bit 4) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr5. This marks the beginning of the interrupt service routine of the second interrupt.



- Clear the bit raising the Vectored IRQ (bit 5) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr6. This marks the beginning of the interrupt service routine of the third interrupt.
- Clear the bit raising the Vectored IRQ (bit 6) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr7. This marks the beginning of the interrupt service routine of the fourth interrupt.
- Clear the bit raising the Vectored IRQ (bit 7) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the fifth interrupt.
- Clear the bit raising the Vectored IRQ (bit 8) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr13. This marks the beginning of the interrupt service routine of the sixth interrupt.
- Clear the bit raising the Vectored IRQ (bit 13) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
- Read the VICVectAddr expecting the content of the VICVectAddr14. This marks the beginning of the interrupt service routine of the seventh interrupt.
- Clear the bit raising the Vectored IRQ (bit 14) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the seventh interrupt.
- Read the VICVectAddr expecting the content of the VICVectAddr15. This marks the beginning of the interrupt service routine of the eighth interrupt.
- Clear the bit raising the Vectored IRQ (bit 15) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the eighth interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the ReadStatus() function is called. Here the VICIRQStatus is read expecting a pattern of 0x0000F0F0. The VICFIQStatus is read expecting a pattern of 0x00000F0F. The pattern 0x0000FFFF is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function CurServReg() is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 16-bit variable called CSR. the value stored in the CSR should be 0x0000F0F0.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function FIQServ() is accessed with the argument zero (indication that it is from the VICTrIntSource). This function clears all FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now, beginning from the 0<sup>th</sup> position, all the FIQ interrupts sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x0000F0F0, 0x00000F0F and 0x0000FFFF respectively. The bits of the VICFIQStatus are read from the 0<sup>th</sup> position. As soon as, there is a 1 (here, it is the 0<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 0<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 1<sup>st</sup> bit position. As soon as, there is a 1 (here, it is the 1<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 1<sup>st</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 2<sup>nd</sup> bit position. As soon as, there is a 1 (here, it is the 2<sup>nd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 2<sup>nd</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 3<sup>rd</sup> bit position. As soon as, there is a 1 (here, it is the 3<sup>rd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 3<sup>rd</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 8<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 8<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 8<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 9<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 9<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 10<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 10<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 10<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 11<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 11<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 11<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 12<sup>th</sup> position. All further bits of VICFIQStatus are zero.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR reflects VICIRQStatus. Since, the CSR is non-zero (indicating the presence of active IRQs), the function IRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as any non-zero bit is read, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read and the expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x0000F0F0, 0x00000000 and 0x0000F0F0 respectively. The CSR is read bit by bit as per the order of priority. As soon as, there is a 1 (here, it is the 4<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A010 (the content of the VICVectAddr4). The VICVectAddr is read expecting the address value as 0x0000A040. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 4 of the VICTrIntSource is made 0. Now, a write

access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 5<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A014 (the content of the VICVectAddr5). The VICVectAddr is read expecting the address value as 0x0000A014. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 5 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 6<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A018 (the content of the VICVectAddr6). The VICVectAddr is read expecting the address value as 0x0000A018. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 6 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0380.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 7<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A01C (the content of the VICVectAddr7). The VICVectAddr is read expecting the address value as 0x0000A01C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 7 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 8<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A020 (the content of the VICVectAddr8). The VICVectAddr is read expecting the address value as 0x0000A020. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 8 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 12<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A030 (the content of the VICVectAddr12). The VICVectAddr is read expecting the address value as 0x0000A030. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 13<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A034 (the content of the VICVectAddr13). The VICVectAddr is read expecting the address value as 0x0000A034. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 13 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 14<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A038 (the content of the VICVectAddr14). The VICVectAddr is read expecting the address value as 0x0000A038. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 14 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 15<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A03C (the content of the VICVectAddr15). The VICVectAddr is read expecting the address value as 0x0000A03C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 15 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the sixteen interrupts case are tested in a similar fashion.

### 8.7.6 Thirty-Two Interrupt Test (AllIntTest or VicEnAllIntTest)

This test is invoked by the AllIntTests() function call for Non-VIC port and VicEnAllIntTests() function call for VIC port. This test can be divided into 33 categories.

In all the categories, the VIC is tested with the external interrupt also. In some cases, certain interrupts are disabled by resetting the bit in the VICIntEnable register. In all, 32 tests are performed for thirty two-Interrupt. One category is illustrated with an example.

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0xF5A00A5F | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn        | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               | ...        | ...                                                              |
| VICVectPriority14 | 0x15       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x0        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               | ...        | ...                                                              |
| VICVectPriority30 | 0x15       | Software priority of 30th interrupt                              |
| VICVectPriority31 | 0x15       | Software priority of 31st interrupt                              |
| VICSWPriorityMask | 0x0000     | no interrupts masked.                                            |

**Table 8.7-6 Initialisation values for an example of Thirty-Two Interrupt Tests**

### Sixteen FIQs and sixteen IRQs

In this illustration, the value 0xFFFFFFFF is written into the VICTrIntSource. This is done by invoking the function ApplyIntr() with the argument as 0. This indicates that sources 0 to 31 are raising interrupts. Since, the bits 0, 1, 2, 3, 4, 6, 9, 11, 21, 23, 24, 26, 28, 29, 30 and 31 of the VICIntSelect are 1, these sources will be raising FIQs. Rest of the interrupts will be raising IRQ interrupts.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 0<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 1<sup>st</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 2<sup>nd</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 3<sup>rd</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 4<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 6<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 9<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 11<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 21<sup>st</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 23<sup>rd</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 24<sup>th</sup> bit of VICTrIntSource.

- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 26<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 28<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 29<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 30<sup>th</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ and nVICFIQ pins, whether they are low.
- Clear the bit raising the external FIQ. This is the 31<sup>st</sup> bit of VICTrIntSource.
- Again check the Status registers, whether they contain the required values.
- Check the nVICIRQ pin, whether it is low.
- Read the VICVectAddr expecting the content of the VICVectAddr16. This marks the beginning of the interrupt service routine of the first interrupt (Interrupt 16 – highest soft priority 0).
- Clear the bit raising the IRQ (bit 16) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr17. This marks the beginning of the interrupt service routine of the second interrupt (software priority 1).
- Clear the bit raising the IRQ (bit 17) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the second interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr18. This marks the beginning of the interrupt service routine of the third interrupt (software priority 2).
- Clear the bit raising the IRQ (bit 18) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the third interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr19. This marks the beginning of the interrupt service routine of the fourth interrupt (software priority 3).
- Clear the bit raising the IRQ (bit 19) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourth interrupt.
- Again check the Status registers, whether they contain the required values.

- Read the VICVectAddr expecting the content of the VICVectAddr20. This marks the beginning of the interrupt service routine of the fifth interrupt (software priority 4).
- Clear the bit raising the IRQ (bit 20) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr5. This marks the beginning of the interrupt service routine of the sixth interrupt (software priority 5).
- Clear the bit raising the IRQ (bit 5) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr22. This marks the beginning of the interrupt service routine of the seventh interrupt (software priority 6).
- Clear the bit raising the IRQ (bit 22) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the seventh interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr7. This marks the beginning of the interrupt service routine of the eighth interrupt (software priority 7).
- Clear the bit raising the IRQ (bit 7) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the eighth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr8. This marks the beginning of the interrupt service routine of the ninth interrupt.
- Clear the bit raising the IRQ (bit 8) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the ninth interrupt (software priority 8).
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr25. This marks the beginning of the interrupt service routine of the tenth interrupt (software priority 9).
- Clear the bit raising the IRQ (bit 25) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the tenth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr10. This marks the beginning of the interrupt service routine of the eleventh interrupt (software priority 10).
- Clear the bit raising the IRQ (bit 10) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the eleventh interrupt.
- Again check the Status registers, whether they contain the required values.

- Read the VICVectAddr expecting the content of the VICVectAddr27. This marks the beginning of the interrupt service routine of the twelfth interrupt (software priority 11).
- Clear the bit raising the IRQ (bit 27) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the twelfth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr12. This marks the beginning of the interrupt service routine of the thirteenth interrupt (software priority 12).
- Clear the bit raising the IRQ (bit 12) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the thirteenth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr13. This marks the beginning of the interrupt service routine of the fourteenth interrupt (software priority 13).
- Clear the bit raising the IRQ (bit 13) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fourteenth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr14. This marks the beginning of the interrupt service routine of the fifteenth interrupt (software priority 14).
- Clear the bit raising the IRQ (bit 14) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the fifteenth interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the VICVectAddr expecting the content of the VICVectAddr15. This marks the beginning of the interrupt service routine of the sixteenth interrupt (software priority 15).
- Clear the bit raising the IRQ (bit 15) in the VICTrIntSource.
- Perform a write access to the VICVectAddr. This marks the end of the interrupt service routine of the sixteenth interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `AllIntService()` is invoked. This function is used to service and clear all the interrupts in the order of their priorities. When the argument of this function is 0, it indicates that the interrupts are being raised by the VICTrIntSource register of the Trickbox. When the interrupts are raised via the VICSoftInt register, the argument is 1.

Now the `ReadStatus()` function is called. Here the VICIRQStatus is read expecting a pattern 0x0A5FF5A0. The VICFIQStatus is read expecting a pattern 0xF5A00A5F. Pattern 0xFFFFFFFF is expected from the VICRawIntr register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be 0xFA5005AF.

Now, the VICFIQStatus is checked. If there is a non-zero bit, it indicates that there is an active FIQ interrupt. Hence, the function `FIQServ()` is accessed with the argument zero (indication that it is from the VICTrIntSource). This function clears all FIQ interrupts. By comparing with the VICFIQStatus register, it is possible to determine which bits of the VICTrIntSource are currently requesting a FIQ. Now all the FIQ interrupts sources are cleared by writing 0s in the corresponding positions in the VICTrIntSource register of the Trickbox.



In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0xFA5005AF, 0x05AFFA50 and 0xFFFFFFFF respectively. The bits of the VICFIQStatus are read from the 0<sup>th</sup> position. As soon as, there is a 1 (here, it is the 0<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 0<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 1<sup>st</sup> bit position. As soon as, there is a 1 (here, it is the 1<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 1<sup>st</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 2<sup>nd</sup> bit position. As soon as, there is a 1 (here, it is the 2<sup>nd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 2<sup>nd</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 3<sup>rd</sup> bit position. As soon as, there is a 1 (here, it is the 3<sup>rd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 3<sup>rd</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 4<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 4<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 4<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 5<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 6<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 6<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 7<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 9<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 9<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 10<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 11<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 11<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 12<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 21<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 21<sup>st</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 22<sup>nd</sup> bit position. As soon as, there is a 1 (here, it is the 23<sup>rd</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 23<sup>rd</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 24<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 24<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 24<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 25<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 26<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 26<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 27<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 28<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 28<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 29<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 29<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 29<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 30<sup>th</sup> bit position. As soon as, there is a 1 (here, it is the 30<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 30<sup>th</sup> bit of the VICTrIntSource is cleared.

Now, the bits of the VICFIQStatus are further read from the 31<sup>st</sup> bit position. As soon as, there is a 1 (here, it is the 31<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000040. Now, the 31<sup>st</sup> bit of the VICTrIntSource is cleared.

Now, the VICIRQStatus is checked. If there is a non-zero bit, it indicates that there is an active IRQ interrupt. Now the value of CSR decides whether the IRQ is vectored or non-vectored. Since, the CSR is non-zero (indicating the

presence of active vectored IRQs), the function VectIRQServ() is accessed with the argument zero (indicating that it is from the VICTrIntSource). This function clears all the IRQ interrupts in the order of their priorities. The priority of the interrupts goes on decreasing when we move from 0 to 15<sup>th</sup> bit in the CSR. The CSR is read from the 0<sup>th</sup> bit. As soon as any non-zero bit occur, the interrupt service routine is started. The routine consists of the reading the VICITOP registers, the address register and then clearing the interrupt and writing to the address register. At the same time the content of the CSR is also changed.

In the present illustration, the status registers are read. Expected data from the VICIRQStatus, VICFIQStatus and VICRawIntr are 0x05AFFA50, 0x00000000 and 0x05AFFA50 respectively. The hardware priority of the interrupts goes on decreasing when we move from the 0<sup>th</sup> to the 31<sup>st</sup> bit in the CSR. The software priority of all the interrupts is stored in array. The software priority is considered first to find the highest priority interrupt. If two or more interrupts have same software priority, the hardware priority is taken into consideration. The CSR is read bit by bit as per the order of priority. As soon as, there is a 1 (here, it is the 1<sup>st</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the vector address, 0x0000A040 (the content of the VICVectAddr16). The VICVectAddr is read expecting the address value as 0x0000A040. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this the bit 16 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A5EF5A0.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 17<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A044 (the content of the VICVectAddr17). The VICVectAddr is read expecting the address value as 0x0000A044. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 17 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A5CF5A0.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 18<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A048 (the content of the VICVectAddr18). The VICVectAddr is read expecting the address value as 0x0000A048. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 18 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A58F5A0.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 19<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A04C (the content of the VICVectAddr19). The VICVectAddr is read expecting the address value as 0x0000A04C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 19 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A50F5A0.

Now, the bits of the CSR are read further from the 20<sup>th</sup> position. As soon as there is a 1 (here, it is the 20<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A050 (the content of the VICVectAddr20). The VICVectAddr is read expecting the address value as 0x0000A050. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 20 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A40F5A0.

Now, the bits of the CSR are read further from the 5<sup>th</sup> position. As soon as there is a 1 (here, it is the 5<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A014 (the content of the VICVectAddr5). The VICVectAddr is read expecting the address value as 0x0000A014. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 5 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A40F580.

Now, the bits of the CSR are read further from the 22<sup>nd</sup> position. As soon as there is a 1 (here, it is the 22<sup>nd</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A058 (the content of the VICVectAddr22). The VICVectAddr is read expecting the address value as 0x0000A058. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this

bit 22 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A00F580.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 7<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A01C (the content of the VICVectAddr7). The VICVectAddr is read expecting the address value as 0x0000A01C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 7 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A00F500.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 8<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A020 (the content of the VICVectAddr8). The VICVectAddr is read expecting the address value as 0x0000A020. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 8 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0A00F400.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 25<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A064 (the content of the VICVectAddr11). The VICVectAddr is read expecting the address value as 0x0000A064. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 25 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0800F400.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 10<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A028 (the content of the VICVectAddr10). The VICVectAddr is read expecting the address value as 0x0000A028. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 10 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0800F000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 10<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A028 (the content of the VICVectAddr10). The VICVectAddr is read expecting the address value as 0x0000A028. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 10 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0800F000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 27<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A06C (the content of the VICVectAddr27). The VICVectAddr is read expecting the address value as 0x0000A06C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 27 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000F000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 12<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A030 (the content of the VICVectAddr12). The VICVectAddr is read expecting the address value as 0x0000A030. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 12 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000E000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 13<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A034 (the content of the VICVectAddr13). The VICVectAddr is read expecting the address value as 0x0000A034. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 13 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x0000C000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 14<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A038 (the content of the VICVectAddr14). The VICVectAddr is read expecting the address value as 0x0000A038. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 10 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x00008000.

Now, the bits of the CSR are read further according to the software priority. As soon as there is a 1 (here, it is the 15<sup>th</sup> bit), the VICITOP1 is read with the expected value as 0x00000060. The VICITOP2 is read expecting the address value as the 0x0000A03C (the content of the VICVectAddr15). The VICVectAddr is read expecting the address value as 0x0000A03C. This indicates the beginning of the interrupt service routine. Now, the interrupt is to be cleared. For this bit 15 of the VICTrIntSource is made 0. Now, a write access is done on the VICVectAddr to mark the end of the interrupt service routine. The content of the CSR is updated to 0x00000000.

Now, the Status registers are read again expecting 0x00000000, 0x00000000 and 0x00000000 from the VICIRQStatus, VICFIQStatus and VICRawIntr registers, respectively.

The other categories of the Thirty Two interrupts case are tested in a similar fashion.

### 8.7.7 Special Interrupt Tests

Apart from the Basic Interrupt tests, where the interrupts are raised simultaneously and cleared one-by-one according to the priority of the interrupts, three other cases are considered.

- Interrupts are raised simultaneously, but the order of servicing and clearing is predefined.
- Interrupts are raised non-simultaneously.
- Interrupts are cleared simultaneously.

#### Interrupts serviced and cleared in a predefined order

Four cases each from the One-Interrupt, Two-Interrupt, Four-Interrupt, Eight-Interrupt, Sixteen-Interrupt and Thirty-Two-Interrupt are considered. In all, 640 tests are performed of this type. One test is illustrated with an example.

Program the following initialising values in the following registers.

| Register         | Value      | Description                                                      |
|------------------|------------|------------------------------------------------------------------|
| VICIntEnable     | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt       | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect     | 0x00000000 | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn       | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn  | 0x00000000 | External Vector Address                                          |
| VICProtection    | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0     | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1     | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...              | ...        | ...                                                              |
| VICVectAddr31    | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0 | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |

| Register          | Value  | Description                                     |
|-------------------|--------|-------------------------------------------------|
| VICVectPriority1  | 0x0    | Software priority of 1 <sup>st</sup> interrupt  |
| ...               | ...    | ...                                             |
| VICVectPriority14 | 0x7    | Software priority of 14 <sup>th</sup> interrupt |
| VICVectPriority15 | 0x7    | Software priority of 15 <sup>th</sup> interrupt |
| VICVectPriority16 | 0x8    | Software priority of 16 <sup>th</sup> interrupt |
| VICVectPriority17 | 0x8    | Software priority of 17 <sup>th</sup> interrupt |
| ...               | ...    | ...                                             |
| VICVectPriority30 | 0x15   | Software priority of 30 <sup>th</sup> interrupt |
| VICVectPriority31 | 0x15   | Software priority of 31 <sup>st</sup> interrupt |
| VICSWPriorityMask | 0x0000 | No software priority interrupts masked.         |

**Table 8.7-7 Initialisation values for an example of Special Interrupt Tests**

In this illustration interrupt sources 12 and 16 are made active by invoking the function `ApplyIntr()` with the argument as 0. Since none of the bits of the `VICIntSelect` are 1, these sources will be raising IRQs. The software priorities of the interrupts are such that interrupt 12 will be serviced before interrupt 16. This Now the function `ServiceTwo()` is called with the arguments 16 followed by 12. This indicates that the interrupt source 16 is to be cleared first followed by the source 12.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the `nVICIRQ` pin, whether it is low.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr12`. This marks the beginning of the interrupt service routine of the first interrupt.
- Clear the 16<sup>th</sup> bit of the interrupt source.
- Check the Status registers, whether they contain the required values.
- Clear the 12<sup>th</sup> bit of the interrupt source.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `ServiceTwo()` is invoked. This function is used to clear the interrupts in the order the arguments are passed to it.

Since, the status registers are clocked, the change in these registers are visible only after one clock cycle. Hence, in the function `ServiceTwo()`, an IDLE cycle is added to compensate for this one clock discrepancy.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00011000. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00011000 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be 0x00011000.

Now, another function called `UserIntService()` is called with the argument 16. This indicates that the interrupt to be cleared is 16. But, the interrupt to be serviced will be the highest priority one (here, it is source 12).

While clearing the interrupt, once the `VICVectAddr` is read it should not be read until this particular interrupt bit is cleared. To track this a 32-bit global variable called `servreg` is used. Here, the `VICVectAddr` will be the content of `VICVectAddr12` (corresponding to the interrupt from 12<sup>th</sup> bit). But, the interrupt to be cleared is that from the 16<sup>th</sup> bit. Hence, as soon as the `VICVectAddr` is read, the 12<sup>th</sup> bit of the `servreg` is set. Hence, `servreg` is now, 0x00001000. Now, the 16<sup>th</sup> interrupt is cleared. Now, interrupt number 12 is to be cleared. But, before reading the `VICVectAddr` register, the `servreg` is checked whether it is zero. Now, after clearing the 12<sup>th</sup> interrupt, the `servreg` is cleared. When the interrupt is an FIQ, the address is not read.

In this illustration, the 16<sup>th</sup> bit of the `VICFIQStatus` is checked for any FIQ at the 16<sup>th</sup> bit position.

Now, the program control returns to the `ServiceTwo()` function. There another `UserIntService()` function call is made with argument as 12.

The 12<sup>th</sup> bit of the `VICFIQStatus` is checked for any FIQ at the 12<sup>th</sup> bit position.

Then the 12<sup>th</sup> bit of `VICIRQStatus` is checked for any IRQ. The CSR is read from the 12<sup>th</sup> bit. As soon as a 1 is read (in this case, it is at bit 12) the `VICVectAddr` is read. This time, the expected address is the content of `VICVectAddr12`. This Now, the CSR is updated to 0x0000 and the `servreg` is changed to 0x0000.

### Interrupts are raised non-simultaneously

In all, 128 tests are performed of this type. One test is illustrated as an example.

Program the following initialising values in the following registers.

| Register                       | Value      | Description                                                                   |
|--------------------------------|------------|-------------------------------------------------------------------------------|
| <code>VICIntEnable</code>      | 0xFFFFFFFF | All the interrupts are enabled                                                |
| <code>VICSoftInt</code>        | 0x00000000 | None of the interrupts can occur through the <code>VICSoftInt</code> register |
| <code>VICIntSelect</code>      | 0x00000000 | Decides which of the sources will raise an FIQ                                |
| <code>VICTrIntIn</code>        | 0x00000003 | Indicates both the external interrupts                                        |
| <code>VICTrVectAddrIn</code>   | 0x00000000 | External Vector Address                                                       |
| <code>VICProtection</code>     | 0x00000000 | VIC is in the 'User' mode                                                     |
| <code>VICVectAddr0</code>      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                                         |
| <code>VICVectAddr1</code>      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                                         |
| ...                            |            | ...                                                                           |
| <code>VICVectAddr31</code>     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                                        |
| <code>VICVectPriority0</code>  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                                |
| <code>VICVectPriority1</code>  | 0x0        | Software priority of 1 <sup>st</sup> interrupt                                |
| ...                            | ...        | ...                                                                           |
| <code>VICVectPriority14</code> | 0x7        | Software priority of 14 <sup>th</sup> interrupt                               |
| <code>VICVectPriority15</code> | 0x7        | Software priority of 15 <sup>th</sup> interrupt                               |
| <code>VICVectPriority16</code> | 0x8        | Software priority of 16 <sup>th</sup> interrupt                               |

| Register          | Value  | Description                                     |
|-------------------|--------|-------------------------------------------------|
| VICVectPriority17 | 0x8    | Software priority of 17 <sup>th</sup> interrupt |
| ...               | ...    | ...                                             |
| VICVectPriority30 | 0x15   | Software priority of 30th interrupt             |
| VICVectPriority31 | 0x15   | Software priority of 31st interrupt             |
| VICSWPriorityMask | 0x0000 | No software priority interrupts masked.         |

**Table 8.7-8 Initialisation values for an example of Special Interrupt Tests**

In this illustration, interrupt source 2 is made active by invoking the function `ApplyIntr()` with the argument as 0. The servicing of this interrupt is started. Now, interrupt source 20 is made active by invoking the function `ApplyIntr()`. As interrupt 20 has lower priority the address on `VICVectAddr` lines will not change. Now, the 2<sup>nd</sup> bit of the interrupt source is cleared. Then the 20<sup>th</sup> bit is cleared.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the `nVICIRQ` pin, whether it is low.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr2`. This marks the beginning of the interrupt service routine of the first interrupt.
- The interrupt from the 20<sup>th</sup> bit is applied.
- Check the Status registers, whether they contain the required values.
- Clear the 2<sup>nd</sup> bit of the interrupt source.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine of the first interrupt.
- Again check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr20`. This marks the beginning of the interrupt service routine of the second interrupt.
- Clear the 20<sup>th</sup> bit of the interrupt source.
- Perform a write access to the `VICVectAddr`. This marks the end of the interrupt service routine of the second interrupt.

To perform the above tasks, the function `TwoDiffInt()` is invoked with the arguments as 2 and 20. This function is used to raise the interrupts in the order the arguments are passed. The interrupt from the 2<sup>nd</sup> bit is applied.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00000004. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00000004 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Then the vector address is read expecting the contents of `VICVectAddr2`. This marks the beginning of the interrupt service routine for interrupt 2.

At this point, the second interrupt from bit 20 is applied. Interrupt 20 has lower priority than interrupt 2.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x00100004. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x00100004 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

If any of the interrupts applied is a FIQ, then it is just cleared. In the present illustration, this case is not applicable.

Now, in the present as the first interrupt has higher priority than second, the second interrupt will remain masked. Then the first interrupt is cleared. This is done by writing a 0 in the 2<sup>nd</sup> bit of the VICTrIntSource. To mark the end of the interrupt service routine, a write access is performed to the VICVectAddr register.

Any pending IRQs are checked. If there is one, then the value of CSR is read. If the CSR is not zero (as is the case here), it looks whether the interrupt service routine of this interrupt has already been started. Here, the VICITOP registers are read. Then the VICVectAddr is read expecting the contents of VICVectAddr20, to indicate the beginning of the interrupt service routine. Then the 20<sup>th</sup> bit in the VICTrIntSource is cleared to clear the interrupt source. To mark the end of the service routine, a write is conducted on VICVectAddr register.

### Interrupts Cleared Simultaneously

In all, 128 tests are performed of this type. One test is illustrated as an example.

Program the following initialising values in the following registers.

| Register          | Value      | Description                                                      |
|-------------------|------------|------------------------------------------------------------------|
| VICIntEnable      | 0xFFFFFFFF | All the interrupts are enabled                                   |
| VICSoftInt        | 0x00000000 | None of the interrupts can occur through the VICSoftInt register |
| VICIntSelect      | 0x00000000 | Decides which of the sources will raise an FIQ                   |
| VICTrIntIn        | 0x00000003 | Indicates both the external interrupts                           |
| VICTrVectAddrIn   | 0x00000000 | External Vector Address                                          |
| VICProtection     | 0x00000000 | VIC is in the 'User' mode                                        |
| VICVectAddr0      | 0x0000A000 | Address for interrupt 0 is 0x0000A000                            |
| VICVectAddr1      | 0x0000A004 | Address for interrupt 1 is 0x0000A004                            |
| ...               | ...        | ...                                                              |
| VICVectAddr31     | 0x0000A07C | Address for interrupt 31 is 0x0000A07C                           |
| VICVectPriority0  | 0x0        | Software priority of 0 <sup>th</sup> interrupt                   |
| VICVectPriority1  | 0x1        | Software priority of 1 <sup>st</sup> interrupt                   |
| ...               | ...        | ...                                                              |
| VICVectPriority14 | 0x14       | Software priority of 14 <sup>th</sup> interrupt                  |
| VICVectPriority15 | 0x15       | Software priority of 15 <sup>th</sup> interrupt                  |
| VICVectPriority16 | 0x0        | Software priority of 16 <sup>th</sup> interrupt                  |
| VICVectPriority17 | 0x1        | Software priority of 17 <sup>th</sup> interrupt                  |
| ...               | ...        | ...                                                              |
| VICVectPriority30 | 0x14       | Software priority of 30th interrupt                              |
| VICVectPriority31 | 0x15       | Software priority of 31st interrupt                              |
| VICSWPriorityMask | 0x0000     | No software priority interrupts masked.                          |

**Table 8.7-9 Initialisation values for an example of Special Interrupt Tests**



In this illustration, interrupt sources 6, 8, 23 and 24 are made active by invoking the function `ApplyIntr()` with the argument as 0. Since, none of the bits of the `VICIntSelect` are 1, these sources will be raising IRQs. In the present case, it is interrupt number 6. But, while clearing the interrupt number 6, the second highest priority interrupt is also cleared. In this case, the interrupt numbers 6 and 23 are cleared simultaneously. The rest of the interrupts are serviced and cleared according to their priorities.

To test the VIC, the following have to be performed:

- Check the Status registers, whether they contain the required values.
- Check the `nVICIRQ` pin, whether it is low.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr6`. This marks the beginning of the interrupt service routine of the interrupt from source 6.
- Identify the next highest priority interrupt, i.e., source 23.
- Clear both the interrupts viz., source 6 and source 23.
- Check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr8`.
- Clear the interrupt source number 8.
- Check the Status registers, whether they contain the required values.
- Read the `VICVectAddr` expecting the content of the `VICVectAddr24`.
- Clear the interrupt source number 24.
- Again check the Status registers, whether they contain the required values.

To perform the above tasks, the function `SimulClear()` is invoked with the arguments as 6, 8, 23 and 24. This function is used to raise the interrupts through the interrupt source 6, 8, 23 and 24.

Now the `ReadStatus()` function is called. Here the `VICIRQStatus` is read expecting a pattern of 0x08800840. The `VICFIQStatus` is read expecting a pattern of 0x00000000. The pattern 0x08800840 is expected from the `VICRawIntr` register. To read these three registers, a burst access is done.

Next, the function `CurServReg()` is invoked. The details of the interrupt sources raising vectored interrupts are stored in a 32-bit variable called CSR. The value stored in the CSR should be 0x08800840.

A variable `intnum` is initialised. This keeps track of the number of interrupts cleared. The fundamental idea is that, when the value of `intnum` is 1, no read or write transactions take place. When the next highest priority interrupt is determined, the value of `intnum` is incremented by 1. Thus, two bits - the highest and second highest priority interrupts - are cleared simultaneously.

First, the `VICFIQStatus` is checked for any FIQ.

If the `intnum` is not equal to 1 (at present it is 0), the status registers are again read. The values expected are 0x08800840, 0x00000000 and 0x08800840 from the `VICIRQStatus`, `VICFIQStatus` and `VICRawIntr` respectively.

The CSR is checked according to software priority array. At the 6<sup>th</sup> bit position, a 1 is encountered. Now, the vector address of the highest priority interrupt is read from the `VICVectAddr`, i.e., 0x0000A018. This is the beginning of the servicing of the interrupt from source 6. The CSR is updated.

Now, the `intnum` is made 1. The variable, which is used to store the data in the `D_VICTrIntSource`, is changed. The 6<sup>th</sup> bit in it is made 0.

Since, the value of `intnum` is 1, no write is made on the `VICTrIntSource`.

Again the checking of the CSR is continued as per software priorities. The 23<sup>rd</sup> bit is 1. Now, `intnum` is incremented to 2. The variable, which is used to store the data in the `D_VICTrIntSource`, is changed. The 23<sup>rd</sup> bit is also made 0. Since, the value of `intnum` is not 1, the value stored by the `D_VICTrIntSource` is written to the `VICTrIntSource` register. The data in the `VICTrIntSource` is 0x08000800.

To mark the end of the servicing of source 6, a write access is done to the VICVectAddr register.

The CSR is updated to 0x08000800.

Since the software priority of both 8<sup>th</sup> and 24<sup>th</sup> interrupt is same, the hardware priority is checked. So a read is done to VICVectAddr expecting contents of VICVectAddr8. Then the 8<sup>th</sup> bit is cleared and a write access is done to VICVectAddr register. Similarly the 24<sup>th</sup> interrupt is also serviced.

### 8.7.8 Priority Consistency Test (PriConsistency)

This test checks whether resolved priority vector address is reflected on the VICVECTADDRROUT line when interrupts are raised every clock cycle. The test case is illustrated below

- Program the Vector Address registers with different addresses.
- Program the software priority for interrupt sources 0 through 8, with interrupt source 0 programmed at highest priority (priority 0).
- Now apply interrupts through VICTrIntSource register in descending order.
- Perform a read on the VICVectAddr register when the 4 interrupt is raised.
- Perform a write on VICVectAddr register after all the remaining interrupts are raised.

### 8.7.9 Nested Interrupts Tests

The Stack16Test, VicEnStack16Test and VicEnDaisyPriTest checks for nested interrupts. The only difference between Stack16Test and VicEnStack16Test is the manner in which the interrupts are serviced as discussed above. In VicEnDaisyPriTest the Daisy chain interrupt has the highest priority. The Stack16Test is illustrated below.

- All the registers are initialised to their initial values.
- Interrupts are raised through VICTrIntSource and VICTrSoftInt register alternatively for each iteration.
- The test has sixteen iterations. Each iteration considers 15 interrupt sources at a time. In the first iteration, the first 15 interrupt sources [14 to 0] are considered. The priority levels of each of these interrupt sources are programmed with respect to their interrupt source value. For example, Priority Level programmed for interrupt source 14 is 0xE. Interrupts source 0 is programmed for highest priority.
- Interrupts are raised one after another starting from 14 interrupt until interrupt 0.
- In between two interrupts a read on VICVectAddr register is performed. This masks the equal or lower priority interrupts to be masked.
- Finally, 15 successive writes on VICVectAddr register is performed to unmask equal and lower priority interrupts.
- In the second iteration the interrupts source from 15 to 1 are considered. Now the 16 interrupt is programmed at level 0xE and interrupt source 1 is programmed for highest priority.
- Interrupts are raised one after another starting from 15 interrupt until interrupt 0.
- In between two interrupts a read on VICVectAddr register is performed. This masks the equal or lower priority interrupts to be masked.
- Finally, 15 successive writes on VICVectAddr register is performed to unmask equal and lower priority interrupts.
- The above steps are repeated for remaining iterations.

Note: In VicEnDaisyPriTest, the Daisy chain interrupt is programmed for highest priority for all 16 iterations.

### 8.7.10 Asynchronous Mode Tests

The ClockOffTest and VicSyncEnTest are used to test asynchronous mode of VIC. These tests are illustrated below.

#### 8.7.10.1 ClockOffTest

The HCLK is turned off by programming VICTrTCR register. Raise the interrupts through VICTrIntSource or VICTrSoftInt register. The nVICIRQ and nVICFIQ output lines should be asserted. After some time, switch on the HCLK by programming the VICTrTCR register. The rest of the test is similar to the Thirty Two Interrupts Test (AllIntTest).

#### 8.7.10.2 VicSyncEnTest

In this test, the nVICSYNCE input signal to VIC is tied to low. This is done by programming the VICTrSync register. The VIC will now operate in asynchronous mode. The VICIRQACK signal from testbench is generated asynchronously with respect to HCLK. The rest of the test is similar to Thirty Two Interrupt Test (AllIntTest).

### 8.7.11 Random Test

In this test, registers are initialised with random values and random numbers of interrupts are applied. This test resolves the priority of the interrupts and the address of the highest priority interrupt resolved is updated locally. The randomly applied interrupts are serviced. Protocol checker compares the outputs from VIC UUT and Mirror Trickbox.

## 8.8 Verification Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **verification/bustest** directory will create **.bif** formatted and **.sim** formatted vectors in the **verification/bustest/invec** directory. The makefile in **verification/bustest** directory can be referred to for the make options.

## 9 INTEGRATION TESTBENCH

The VIC integration test vector suite allows the integrator to verify that the VIC has been connected into the target system correctly. These vectors test 2 classes of signals:

- AMBA signals (VIC signals connected directly to the AHB)
- Intra-Chip signals (VIC signals connected to the ARM or to other daisy chained VICs)

For more details on Integration testing, please refer to the PrimeCell Integration Vector Strategy document [Ref 4.].

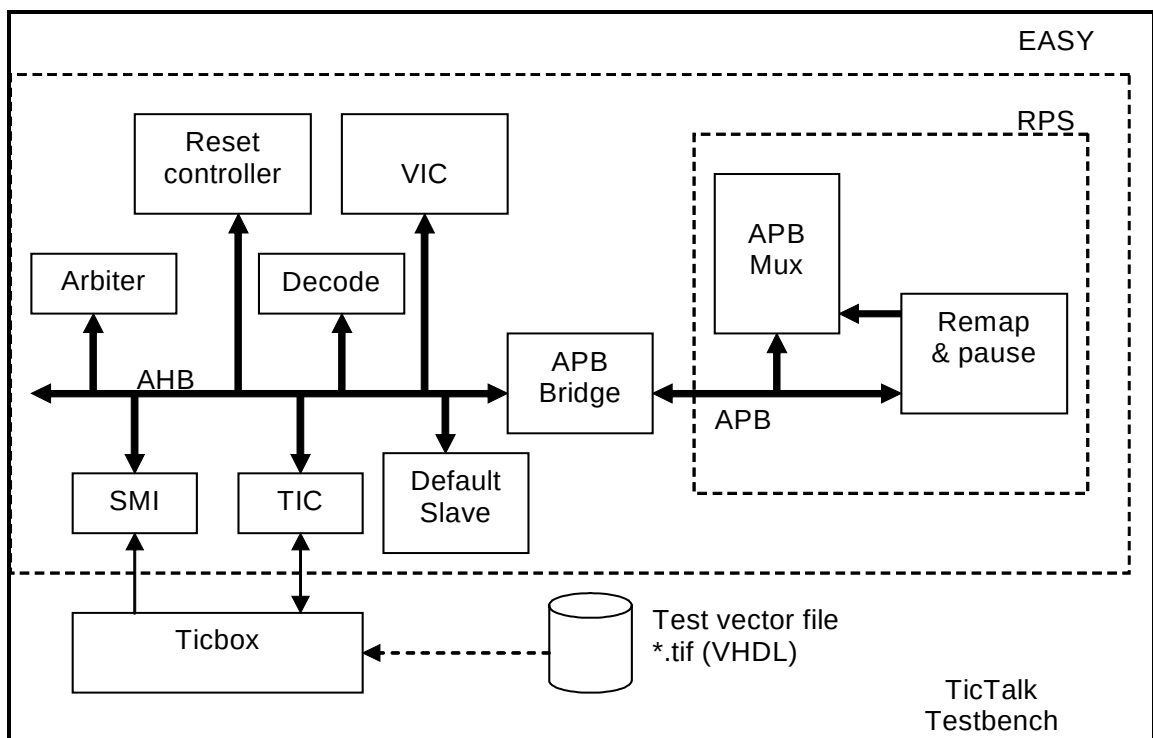
The VIC Integration Testbench is based on an AHB TICTalk Testbench. The VHDL and Verilog TICTalk Testbenches are used to verify that the VIC has been connected into the target system correctly. The VHDL and Verilog TICTalk Testbench files reside in the *integration/vhdl* and *integration/verilog* directories respectively. The test vectors that are applied to the TICTalk Testbenches are in the *integration/bustest* directory. The test vectors for integration testing are written in BusTalk and then converted into *.tif* format to be applied to the TICTalk integration Testbench.

### 9.1 VIC VHDL Integration Testbench

The AMBA TICTalk VHDL Testbench used for integration testing of the VIC is located in the *integration/vhdl/tbench* directory. The Integration test vectors are located in the *integration/bustest* directory.

The test vectors used with this Testbench can be applied to the VIC when it is part of an AMBA system design.

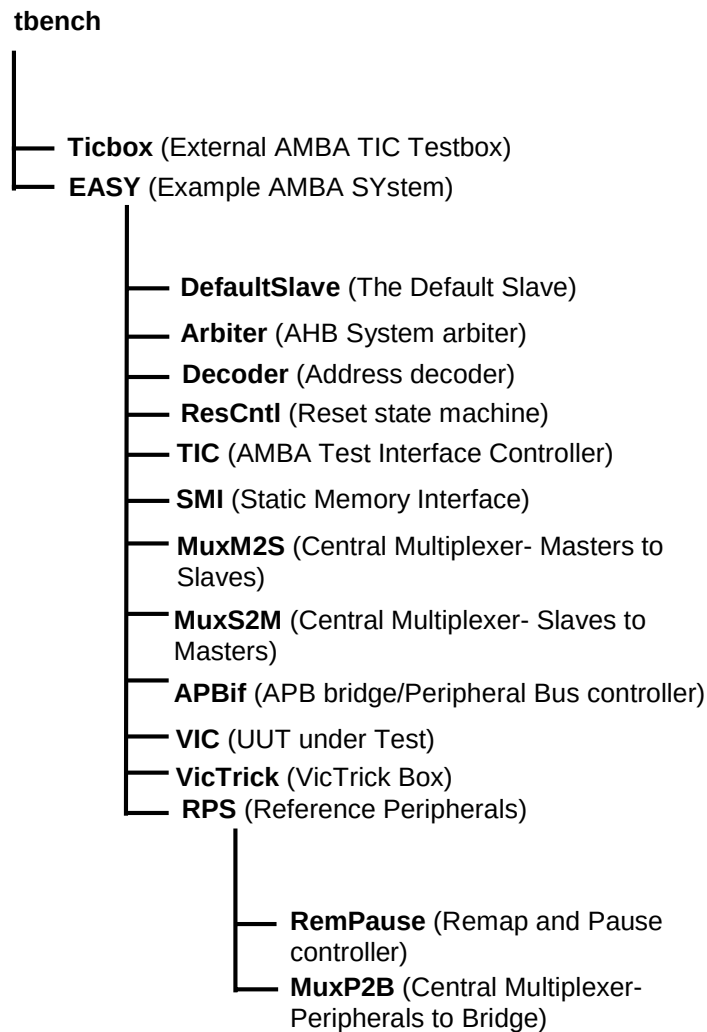
**Figure 9.1-1** illustrates the VHDL Testbench to apply TICTalk test vectors.



**Figure 9.1-1** VHDL Testbench for simulating TICTalk test vectors

The VIC VHDL Integration Testbench is a stripped down version of the standard EASY system Testbench. For further information refer to the Example AMBA System user guide [Ref 3] which is part of ARM Micropack documentation.

**Figure 9.1-2** illustrates the hierarchy for the TICTalk VHDL Testbench.



**Figure 9.1-2 VHDL Testbench hierarchy for simulating TICTalk test vectors**

The test vectors are applied to **tbench.vhd**, the top-level of the VIC Integration Testbench. This module instantiates the model of the EASY Microcontroller and the Ticbox. The VIC is instantiated in the EASY.vhd. All the Scan input pins are tied low to keep them from interfering with the test process. The VICINTSOURCE, VICVECTADDRIN, nVICFIQIN and nVICIRQIN lines are tied to their inactive levels. [During system integration, these lines will have to be re-wired].

### 9.1.1 Unit Under Test

The Unit Under Test may be one of the following

#### VIC VHDL RTL

#### VIC scaninsert VHDL netlist from VHDL Source

One out of these two versions may be referenced by creating a link [named **uut**] in the **integration/vhdl** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].

### 9.1.2 VIC Trickbox (VicTrickbox)

The VIC Trickbox is used for testing the intra chip input signals nVICIRQINREG, nVICFIQINREG and VICINTSOURCE signals. The output signals from VIC nVICIRQ, nVICFIQ and VICVECTADDROUT are inverted and fed back to nVICIRQINREG, nVICFIQINREG and VICINTSOURCE signals.

### 9.1.3 Test Vectors

The test vectors for the integration testing of the VIC are coded into separate C files for testing accesses to all the VIC registers and for Integration testing. Make options are provided to run the tests together or individually.

### 9.1.4 Running Simulations

The **README** file in the *integration/vhdl/tbench* directory gives step by step instructions for running simulations using the VIC Integration test vectors on the VHDL source code.

Integration tests are done with the VIC in stand-alone mode with the internal test paths from the intra-chip inputs pins to the VIC core logic being selected.

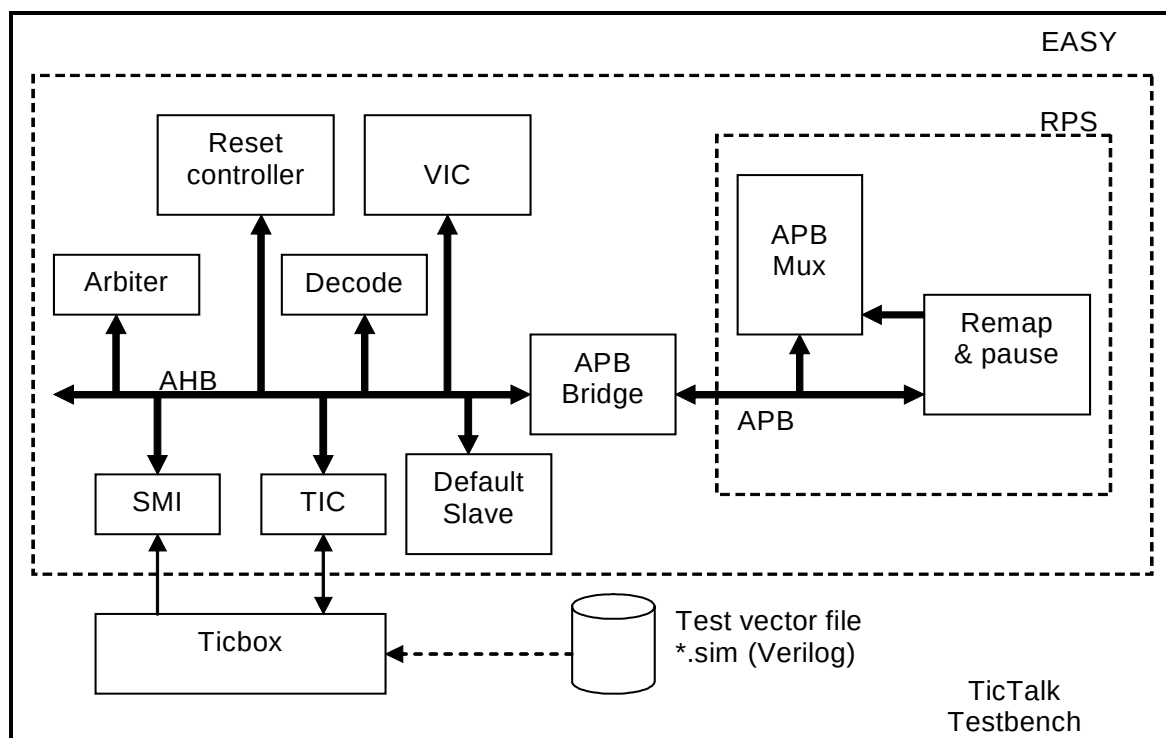
Run time logs for all the combinations are available in the *integration/vhdl/Vic\_rtl* and *integration/vhdl/VIC\_scaninsert\_vhdl\_net\_max* directories.

## 9.2 VIC Verilog Integration Testbench

The AMBA TICTalk Verilog Testbench used for integration testing of the VIC is located in the *integration/verilog/tbench* directory. The Integration test vectors are located in the *integration/bustest/invec* directory.

The test vectors used with this Testbench can be applied to the VIC when it is part of an AMBA system design.

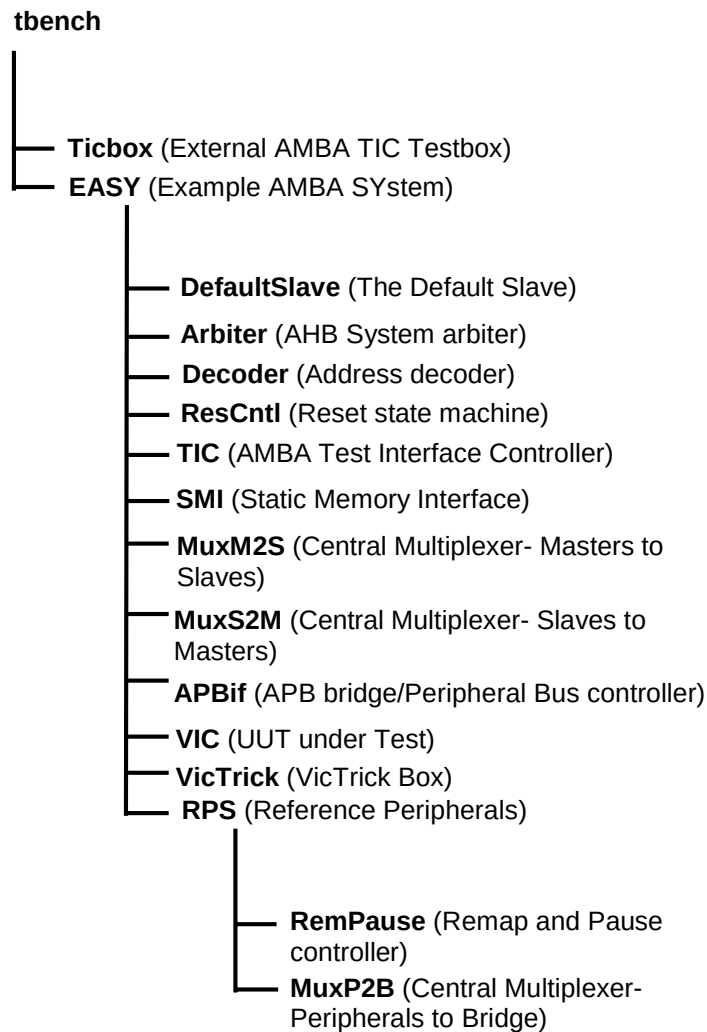
**Figure 9.2-1** illustrates the Verilog Testbench to apply TICTalk test vectors.



**Figure 9.2-1 Verilog Testbench for simulating TICTalk test vectors**

The VIC Verilog Integration Testbench is a stripped down version of the standard EASY system Testbench. For further information refer to the Example AMBA System user guide [Ref 3] which is part of ARM Micropack documentation.

**Figure 9.2-2** illustrates the hierarchy for the TICTalk Verilog Testbench.



**Figure 9.2-2 Verilog Testbench hierarchy for simulating TICTalk test vectors**

The test vectors are applied to **tbench.v**, the top-level of the VIC Integration Testbench. This module instantiates the model of the EASY Microcontroller and the Ticbox. The VIC is instantiated in the EASY.v. All the Scan input pins are tied low to keep them from interfering with the test process. The VICINTSOURCE, VICVECTADDRIN, nVICFIQIN and nVICIRQIN lines are tied to their inactive levels. [During system integration, these lines will have to be re-wired].

### 9.2.1 Unit Under Test

The Unit Under Test may be one of the following

- **VIC Verilog RTL**
- **VIC scanready Verilog netlist from Verilog Source**
- **VIC noscan Verilog netlist from VHDL Source**

One out of these three versions may be referenced by creating a link [named **uut**] in the **integration/verilog** directory to the corresponding source directory. [Note that this link will be created automatically by the simulation makefiles].



### 9.2.2 VIC Trickbox (VicTrickbox)

The VIC Trickbox is used for testing the intra chip input signals nVICIRQINREG, nVICFIQINREG and VICINTSOURCE signals. The output signals from VIC nVICIRQ, nVICFIQ and VICVECTADDROUT are inverted and fed back to nVICIRQINREG, nVICFIQINREG and VICINTSOURCE signals.

### 9.2.3 Test Vectors

The test vectors for the integration testing of the VIC are coded into separate C files for testing accesses to all the VIC registers and for Integration testing. Make options are provided to run the tests together or individually.

### 9.2.4 Running Simulations

The **README** file in the *integration/verilog/tbench* directory gives step by step instructions for running simulations using the VIC Integration test vectors on the VERILOG source code.

Run time logs for all the combinations are available in the following files in the *integration/verilog/Vic\_rtl*, *integration/verilog/Vic\_noscan\_vhdl\_net\_min* and *integration/verilog/Vic\_scanready\_verilog\_net\_typ* directories.

*Vic\_rtl\_modelsim.log*

*Vic\_rtl\_LDV.log*

*Vic\_noscan\_vhdl\_verilog\_net\_min\_modelsim.log*

*Vic\_scanready\_verilog\_verilog\_net\_typ\_LDV.log*

## 10 INTEGRATION TESTS

When the VIC is used in the AMBA system, it must be ensured that all the I/O pins are correctly connected. Integration vectors allow the user to verify that the VIC has been wired into the system correctly.

Integration testing is performed by the RegAccessTests() and the IntegrationTest() function calls.

### 10.1 Integration Testing of AHB pins

The RegAccessTests() function call is used to test the connectivity of the AHB signals via a sequence of register accesses.

- HCLK and HRESETn are verified to be toggling (implicitly) by verifying that the other tests, which follow pass. Note that the test sequence starts with an assertion and then a de-assertion of the HRESETn line.
- HADDR: 1's and 0's are made to appear on these signals by performing a read access to all the internal registers, following the de-assertion of HRESETn.
- HTRANS[1]: A '1' and a '0' are made to appear on this AHB signal by initiating an NSEQ (HTRANS[1] = 1) and an IDLE (HTRANS[1] = 0) transfer.
- HWRITE: Read and Write accesses are performed to the VIC to toggle this signal.
- HSIZE: The VIC supports only WORD (HSIZE = 2) accesses. 1's and 0's are made to appear on these lines by performing WORD and HALFWORD accesses to the internal registers.
- HPROT: A '1' and a '0' is made to appear on this AHB signal by making 'User' and 'Supervisor' mode accesses. Toggling of this line is verified using the following steps. Configure the VIC in 'PROTECTED' mode. Then access the VIC in Supervisor mode for writing into one of VICVectAddr registers with a data value of 0x55555555. Access the VIC in User mode to modify this register. Check the register contents to prove that it remains unmodified.
- HWDATA and HRDATA: These lines are toggled by performing write and read accesses with different data patterns (0x55555555 and 0xAAAAAAAA) and their connectivity are verified by validating those accesses.
- HSELVIC: A '1' and a '0' is made to appear on this line by writing to a location with HADDR[11:2] same as the offset address of a VIC register (VICIntSelect) and with a base address different from that of the VIC. The connectivity of HSELVIC is proven by checking for unmodified data in the above register.

### 10.2 Integration Testing of intra-chip inputs

Signals: VICIRQACK, nVICFIQIN, nVICIRQIN, nVICFIQINREG, nVICIRQINREG, VICVECTADDRIN and VICINTSOURCE.

When Integration tests are run with the VIC in a stand-alone test setup:

- Write a '1' to the ITEN bit in the VICITCR register. This selects the test path from the VICITIP registers to the internal signals.
- Write 1's and then 0's to the VICITIP registers and read the same register to ensure that the value written is read out.

When Integration tests are run with the VIC as part of an integrated system:

- Write a '0' to the ITEN bit in the VICITCR register. This selects the normal path from the intra-chip inputs to the internal VIC signals.

- '1's and '0's are made to appear on these inputs by programming their sources appropriately.
- For the VICIRQACK, nVICFIQIN, nVICIRQIN and VICVECTADDRIN inputs in a system where multiple VICs are connected in a Daisy Chain, these signals are controlled by writing into the internal registers of the first VIC in the Daisy Chain and the connection is verified by reading out from the VICITIP1-2 registers. In case the source of these signals is not another VIC, the integrator will have to suitably program the relevant controller.
- For the nVICFIQINREG, nVICIRQINREG and VICINTSOURCE inputs, the VicTrickbox inverts the nVICFIQ, nVICIRQ and VICVECTADDRROUT signals and are feedback on nVICFIQINREG, nVICIRQINREG and VICINTSOURCE signals. The VICINTSOURCE inputs are verified by writing various combinations in to VICITOP2 register and reading the status of VICRawIntr register. nVICFIQINREG and nVICIRQINREG inputs are verified by writing various combinations to VICITOP1 register bits 6 and 7 and reading the VICITIP1 register bits 10 and 9.
- The **IntegrationTest.c** file in the **integration/bustest** directory is embedded with NOTES, which direct the system integrator to those commands that need to be replaced with suitable equivalents.

### 10.3 Integration Testing of intra-chip outputs

Signals: VICIRQACKOUT, nVICFIQ, nVICIRQ and VICVECTADDRROUT

When Integration tests are run with the VIC in a stand-alone test setup:

- Program the VIC such that the all intra-chip outputs are set to both '0' and '1'.
- Read the expected values back from the VICITOP1 and VICITOP2 registers. Note that for the nVICIRQ and the nVICFIQ outputs, a '1' on the output pin will be reflected as a '0' in the VICITOP1 register [and vice versa]

When Integration tests are run with the VIC as part of an integrated system:

- Program the VIC such that the all intra-chip outputs are set to both 0's and then '1's.
- In a system where multiple VICs are connected in a Daisy Chain, the connection is verified by reading out the expected values from the next VIC's internal registers (VICITIP1-2). Note that for the nVICIRQ and the nVICFIQ outputs, a '1' on the output pin will be reflected as a '1' in the next VIC's internal registers.

### 10.4 Integration Test Vector Generation

A **make sourcefilename** (without the .c extension) in the **integration/bustest** directory will create .tif formatted and .sim formatted vectors in the **integration/bustest/invec** directory. The makefile in the **integration/bustest** directory can be referred to for the make options.

The **make all** option can be used to create a test vector file including all the tests.